Contents

Study-Hub: The Complete Beginner's Tutorial **4**
Files in This Tutorial Series . . . 4
How to Read This . . . 5

Section 01 — The Big Picture **5**
1.1 What Is Code? . . . 5
1.2 How the Web Works — The Full Journey . . . 5
1.3 The Two Sides of Every Web App . . . 6
1.4 What Is Study-Hub? . . . 7
1.5 The Technology Stack . . . 8
1.6 What You Will Build, Step by Step . . . 8
1.7 A Note on Making Mistakes . . . 8

Section 02 — Tools & Setup **9**
2.1 The Tools You Need . . . 9
2.2 Node.js . . . 9
2.3 PostgreSQL . . . 10
2.4 VS Code (Visual Studio Code) . . . 11
2.5 Git . . . 11
2.6 An OpenRouter API Key . . . 12
2.7 Setting Up the Project . . . 12
2.8 Your Environment is Ready . . . 14

Section 03 — HTML Foundations **14**
3.1 What Is HTML? . . . 14
3.2 How HTML Tags Work . . . 15
3.3 Attributes . . . 15
3.4 The Anatomy of an HTML Document . . . 15
3.5 The Common Tags Used in Study-Hub . . . 17
3.6 The `data-*` Attributes in Study-Hub . . . 19
3.7 Study-Hub's HTML Structure at a High Level . . . 19
3.8 Comments in HTML . . . 20
3.9 Key Takeaways . . . 20

Section 04 — CSS Foundations **20**
4.1 What Is CSS? . . . 20
4.2 How CSS Works — The Basic Syntax . . . 20
4.3 Selectors — How CSS Finds Elements . . . 21
4.4 The Box Model — How Space Works . . . 22
4.5 CSS Variables (Custom Properties) . . . 22
4.6 Flexbox — The Layout System . . . 24
4.7 CSS Grid . . . 25
4.8 Positioning . . . 25
4.9 Transitions and Animations . . . 26
4.10 Glassmorphism . . . 27
4.11 The CSS Reset . . . 28
4.12 Responsive Design . . . 28
4.13 The `hidden` Class . . . 28
4.14 Study-Hub's CSS File Architecture . . . 29
4.15 Key Takeaways . . . 29

Section 05 — JavaScript Foundations **29**
5.1 What Is JavaScript? . . . 29

5.2 Where JavaScript Runs . . . 30
5.3 Variables — Storing Information . . . 30
5.4 Functions — Reusable Instructions . . . 31
5.5 Control Flow — Making Decisions . . . 32
5.6 Arrays — Working with Lists . . . 33
5.7 Objects — Working with Named Data . . . 34
5.8 The DOM — JavaScript Talking to the Page . . . 35
5.9 Events — Responding to User Actions . . . 36
5.10 Async JavaScript — Promises and async/await . . . 37
5.11 The Module Pattern (IIFEs) . . . 39
5.12 localStorage — Persisting Data in the Browser . . . 39
5.13 JSON — The Language of APIs . . . 40
5.14 Error Handling . . . 40
5.15 Debounce — Throttling Rapid Events . . . 41
5.16 Key Takeaways . . . 41

Section 06 — Project Structure **41**
6.1 The Full Folder Tree . . . 41
6.2 Why This Structure? . . . 43
6.3 Load Order Matters . . . 44
6.4 The `node_modules` Folder . . . 44
6.5 The `package.json` File . . . 44
6.6 Environment Variables and the `.env` File . . . 45
6.7 The `uploads/` Folder . . . 45
6.8 The `render.yaml` File . . . 45

Section 07 — Backend: Node.js & Express **46**
7.1 What Is a Server? . . . 46
7.2 What Is Express? . . . 46
7.3 server.js — The Entry Point, Line by Line . . . 47
7.4 Middleware — The Request Pipeline . . . 52
7.5 How Express Routes Work . . . 52
7.6 Request and Response Objects . . . 53
7.7 Error Handling Middleware . . . 53

Section 08 — Database & PostgreSQL **53**
8.1 What Is a Database? . . . 54
8.2 SQL — The Language of Databases . . . 54
8.3 database.js — The Connection Module . . . 54
8.4 The Schema — Study-Hub's Data Design . . . 56
8.5 Relationships Between Tables . . . 60
8.6 Key Takeaways . . . 61

Section 09 — Authentication **61**
9.1 The Problem: Proving Identity . . . 61
9.2 Password Hashing with bcrypt . . . 61
9.3 JWT — Staying Logged In . . . 63
9.4 The Full Auth Flow . . . 65
9.5 How the Frontend Handles Auth . . . 66
9.6 Input Validation . . . 67
9.7 Key Takeaways . . . 67

Section 10 — API Design **67**
10.1 What Is an API? . . . 67
10.2 REST — The Design Style . . . 68

10.3 Study-Hub's Complete API Reference . . . 68
10.4 The Standard Response Format . . . 74
10.5 HTTP Status Codes Used in Study-Hub . . . 74
10.6 Ownership Checks — Security Detail . . . 74
10.7 Key Takeaways . . . 75

Section 11 — Frontend Architecture **75**
11.1 What Is a Single-Page Application (SPA)? . . . 75
11.2 The Module System . . . 75
11.3 storage.js — The localStorage Wrapper . . . 76
11.4 helpers.js — Utility Functions . . . 77
11.5 api.js — The HTTP Client . . . 80
11.6 app.js — The Orchestrator . . . 82
11.7 The Data Flow Pattern . . . 84
11.8 Key Takeaways . . . 85

Section 12 — The Notes Feature **85**
12.1 What the Notes Feature Does . . . 85
12.2 The Backend: notesController.js . . . 85
12.3 The Frontend: notes.js . . . 88
12.4 Pinning and Archiving . . . 92
12.5 Full-Text Search . . . 92
12.6 The Complete Data Flow for Creating a Note . . . 93
12.7 Key Takeaways . . . 93

Section 13 — The PDF Feature **93**
13.1 What the PDF Feature Does . . . 93
13.2 File Upload Architecture . . . 93
13.3 middleware/upload.js — Multer Configuration . . . 94
13.4 PDF Text Extraction — pdfService.js . . . 95
13.5 pdfController.js — The Upload Handler . . . 96
13.6 Frontend: pdfs.js . . . 98
13.7 File Storage Security . . . 100
13.8 Deleting PDFs . . . 100
13.9 Key Takeaways . . . 101

Section 14 — The AI Chat Feature **101**
14.1 What the Chat Feature Does . . . 101
14.2 How AI APIs Work . . . 101
14.3 llmService.js — The Complete AI Service . . . 102
14.4 chatController.js — The Send Message Handler . . . 105
14.5 Frontend: chat.js — The Chat Interface . . . 109
14.6 The Complete Flow for Sending a Message with Context . . . 114
14.7 Key Takeaways . . . 115

Section 15 — System Design **115**
15.1 The Big Picture Architecture . . . 115
15.2 Request Lifecycle — The Full Journey . . . 116
15.3 Data Flow Diagram — Notes Feature . . . 118
15.4 Major Design Decisions and Why . . . 119
15.5 Security Model . . . 121
15.6 Performance Considerations . . . 121
15.7 The Data Model — Relationship Summary . . . 122
15.8 Key Takeaways . . . 122

Section 16 — Deployment **123**
16.1 What Deployment Means . . . 123
16.2 What Changes in Production . . . 123
16.3 Setting Up Git . . . 123
16.4 Deploying to Render.com . . . 124
16.5 Environment Variables in Production . . . 125
16.6 The Deployment Pipeline . . . 126
16.7 The Free Tier Limitations . . . 126
16.8 Checking Your Deployment . . . 126
16.9 Custom Domain (Optional) . . . 127
16.10 Making Your First Code Change After Deploy . . . 127
16.11 Key Takeaways . . . 127

Section 17 — Glossary **127**

Study-Hub: The Complete Beginner's Tutorial

From "What is code?" to a fully working AI-powered web app

Who this is for: Someone who has never written a single line of code. Every word, every concept, every file is explained from scratch. Nothing is assumed.

What you will build: Study-Hub — a full-stack web application where users can write notes, upload PDFs, and chat with an AI that knows their documents.

How long this takes: Reading alone is 5–10 hours. Building along the way may take a weekend or more. That's fine. Go slowly. Understanding one thing completely beats skimming ten things.

Files in This Tutorial Series

File	What It Covers
`00_INDEX.md`	This file — the map
`01_BIG_PICTURE.md`	What is coding? How the web works. What we're building and why.
`02_TOOLS_SETUP.md`	Every tool you need, how to install it, and why it exists
`03_HTML_FOUNDATIONS.md`	HTML from zero — every tag used in Study-Hub explained
`04_CSS_FOUNDATIONS.md`	CSS from zero — styling, layout, themes, animations
`05_JAVASCRIPT_FOUNDATIONS.md`	JavaScript from zero — variables, functions, events, async
`06_PROJECT_STRUCTURE.md`	The folder layout explained — why files live where they do
`07_BACKEND_NODE_EXPRESS.md`	What is a server? Node.js, Express, routing — all explained
`08_DATABASE_POSTGRESQL.md`	What is a database? Tables, queries, the Study-Hub schema
`09_AUTHENTICATION.md`	Passwords, hashing, JWT tokens — how login works
`10_API_DESIGN.md`	What is an API? REST, HTTP methods, every route in the app
`11_FRONTEND_ARCHITECTURE.md`	How the frontend JS is organized — SPA, modules, state
`12_NOTES_FEATURE.md`	Building the Notes system end-to-end
`13_PDF_FEATURE.md`	Building the PDF upload and viewer system
`14_CHAT_AI_FEATURE.md`	Building the AI chat with context injection
`15_SYSTEM_DESIGN.md`	Architecture diagrams, data flow, design decisions explained
`16_DEPLOYMENT.md`	Putting it on the internet with Render.com
`17_GLOSSARY.md`	Every technical word used, explained in plain English

How to Read This

Do not skip sections. Each section builds on the last.

If you hit a word you don't understand, check `17_GLOSSARY.md` first. If it's not there, search for it online — that skill (looking things up) is itself one of the most important things a developer learns.

Copy the code. Don't just read it. Type it out yourself, even if you copy it. Your hands learn differently than your eyes.

Break things on purpose. After you get something working, change a value, delete a line, see what breaks. Error messages are not failures — they are explanations.

Let's begin.

Section 01 — The Big Picture

What is code? How does the web work? What are we building?

1.1 What Is Code?

Code is instructions written for a computer to follow.

That's it. There's no magic. A computer is an extremely fast machine that follows instructions literally. It does exactly what you tell it — nothing more, nothing less. If it does something wrong, it's because the instructions were wrong.

Think of it like a recipe. A recipe tells a cook: "Add two eggs. Mix for 3 minutes. Bake at 180°C for 25 minutes." The cook follows those steps in order. Code is the same thing — a sequence of steps that a computer follows.

The difference is that computers speak different languages than humans. So instead of writing "Add two eggs," you write in a language the computer understands. In this project, we use three such languages:

- **HTML** — describes *what* is on a page (a button, a text box, a heading)
- **CSS** — describes *how it looks* (the button is purple, the heading is large)
- **JavaScript** — describes *what happens* (when the user clicks the button, send the message)

And on the server side, we use: - **JavaScript again** (via Node.js) — runs on the server, handles requests - **SQL** — talks to the database ("give me all notes belonging to this user")

1.2 How the Web Works — The Full Journey

When you type `google.com` into your browser and press Enter, here is exactly what happens. Understanding this is the foundation of everything else in this tutorial.

Step 1: Your browser asks "where is google.com?"

Your browser doesn't know where Google is. It only knows an address system called **IP addresses** — numbers like `142.250.80.46`. It asks a special server called a **DNS server** (Domain Name System) to translate the human-friendly name `google.com` into a machine-friendly IP address.

This is like asking directory assistance for a phone number.

Step 2: Your browser connects to that address

Now that it has the IP address, your browser makes a connection to Google's computer (called a **server**) over the internet. This connection uses a protocol called **HTTP** (or **HTTPS** — the secure version).

Protocol just means "agreed-upon rules for how to communicate." HTTP is a set of rules for how browsers and servers talk to each other.

Step 3: Your browser sends a Request

Your browser sends a message to the server. This message is called an **HTTP Request**. It says something like:

```
GET / HTTP/1.1
Host: google.com
```

This is saying: "Hey Google's server, please give me the main page (`/`) of your site."

`GET` is a **method** — it means "I want to read something." There are other methods too: - `POST` — "I want to send you something new" (like submitting a form) - `PUT` — "I want to update something" - `DELETE` — "I want to delete something"

You will use all of these in Study-Hub.

Step 4: The server sends a Response

The server (Google's computer) receives the request, figures out what to send back, and returns an **HTTP Response**. This includes:

1. A **status code** — a number indicating success or failure
   - `200` means "OK, here's what you asked for"
   - `404` means "Not found — that page doesn't exist"
   - `401` means "Unauthorized — you need to be logged in"
   - `500` means "Internal Server Error — something went wrong on our end"
2. The actual **content** — in this case, HTML, CSS, and JavaScript files

Step 5: Your browser renders the page

Your browser receives the HTML, CSS, and JavaScript. It reads the HTML to understand what's on the page, applies the CSS to make it look right, and runs the JavaScript to make things interactive. This whole process is called **rendering**.

1.3 The Two Sides of Every Web App

Every web application — from Google to Facebook to Study-Hub — has two sides:

The Frontend (Client Side)

This is everything that runs **in the user's browser**. It's what the user sees and interacts with: buttons, text boxes, menus, animations.

The frontend is built with: - **HTML** — the structure - **CSS** — the appearance - **JavaScript** — the behaviour

The frontend files (HTML, CSS, JS) are downloaded to the user's computer and run there. This is important: the user's browser does the work, not a remote server.

The Backend (Server Side)

This is everything that runs **on a remote computer** (a server). The user never sees this directly. It handles: - Storing and retrieving data (from a database) - Authentication (checking if a user is who they say they are) - Business logic (complex rules about what users can do) - Communicating with external services (like an AI API)

The backend is built with **Node.js** (JavaScript on the server) and stores data in **PostgreSQL** (a database).

How They Talk to Each Other

The frontend and backend communicate through the same HTTP mechanism described above. The frontend sends requests (e.g., "give me this user's notes"), and the backend sends responses (e.g., the actual list of notes as JSON data).

This communication channel — the rules for how the frontend asks and the backend responds — is called an **API** (Application Programming Interface). We'll cover this in detail in Section 10.

1.4 What Is Study-Hub?

Study-Hub is a **personal knowledge management system** with an AI assistant.

Here's what it does:

Notes — Users can write notes in markdown format (a simple formatting language), organize them into folders, pin important ones, search through them, and archive old ones.

PDFs — Users can upload PDF files. The app extracts the text from the PDF so the AI can read it. Users can view PDFs directly in the browser.

AI Chat — Users can have conversations with an AI. The key feature: the AI has access to the user's notes and PDFs. You can attach a note or PDF to a conversation, and the AI will read it and answer questions about it. The AI can also create and save new notes directly to your library.

Authentication — Each user has their own account. Your notes are private — only you can see them.

Themes — The app has four visual modes: Minimal Dark, Minimal Light, Glass Dark, Glass Light.

1.5 The Technology Stack

A “stack” is the collection of technologies used to build an application. Here is Study-Hub’s stack explained in plain language:

Technology	What it is	Why we use it
HTML	The skeleton of web pages	Without HTML there is no page
CSS	The appearance of web pages	Without CSS everything looks like a 1990s website
JavaScript (browser)	Makes pages interactive	Without JS the page is just a static document
Node.js	JavaScript that runs on a server	Lets us use one language (JS) for both frontend and backend
Express	A framework that makes building servers with Node.js easier	Without it, you’d write hundreds of extra lines just for basic things
PostgreSQL	A database	Stores all user data permanently
JWT	JSON Web Tokens — a way of proving identity	Keeps users logged in securely
bcryptjs	A library for hashing passwords	Stores passwords safely so even we can’t read them
Multer	A library for handling file uploads	Makes it easy to accept PDF files from users
pdf-parse	A library for reading text from PDFs	Extracts text so the AI can read it
OpenRouter API	A service that gives access to AI models	Powers the chat feature
Git	Version control — tracks changes to your code	Industry standard for managing code

---

1.6 What You Will Build, Step by Step

Here’s the order in which we’ll build Study-Hub:

1. Set up your tools and computer
2. Create the folder structure
3. Build the database schema (design the data tables)
4. Build the backend server and API
5. Build the frontend HTML shell
6. Add CSS styling
7. Write the JavaScript that makes it all work
8. Test everything locally
9. Deploy to the internet

By the end, you will have a real, working web application accessible from any browser in the world.

1.7 A Note on Making Mistakes

You will make mistakes. That is not a problem — it is the main way you learn.

When you see an error message, don't panic. Read it carefully. Error messages are the computer trying to tell you what went wrong. They are written in technical language, but you'll quickly learn to decode them.

The most common beginner mistakes are: - A missing closing bracket `}` or parenthesis `)` - A typo in a variable name (`userName` vs `username`) - Forgetting a comma between items in a list - Using the wrong type of quotes

All of these are fixable. None of them are disasters.

Now let's set up your tools.

Section 02 — Tools & Setup

Every tool you need, how to install it, and why it exists

2.1 The Tools You Need

Before you write a single line of code, you need the right tools installed. Here is every tool, what it does, and how to get it.

2.2 Node.js

What is it?

JavaScript was originally designed to run only inside web browsers. **Node.js** is a program that lets JavaScript run outside the browser — specifically on a server (a computer that serves requests).

When you install Node.js, you get two things: - `node` — the program that runs JavaScript files - `npm` — **Node Package Manager**, a tool for installing JavaScript libraries (called "packages")

Why do we need it?

Study-Hub's server is written in JavaScript. Without Node.js, there's no way to run that server code.

How to install it

Go to **https://nodejs.org** and download the **LTS** version (Long Term Support — the stable one). Install it like any regular program.

After installing, open your **Terminal** (on Mac: search for "Terminal" in Spotlight; on Windows: search for "PowerShell" or install "Windows Terminal") and type:

```
node --version
```

You should see something like `v20.11.0`. If you see a version number, it worked.

Then check npm:

```
npm --version
```

You should see something like `10.2.4`.

2.3 PostgreSQL

What is it?

PostgreSQL (often called "Postgres") is a database management system. A database is a program specifically designed to store and retrieve data very efficiently and reliably.

Think of it like a very powerful, structured Excel spreadsheet — but one that can handle millions of rows, multiple tables that relate to each other, and thousands of simultaneous users.

Why do we need it?

Study-Hub needs to save user accounts, notes, PDFs, and chat messages permanently. If we just kept this data in JavaScript variables, it would disappear every time the server restarts. A database keeps it safe.

How to install it

Mac: The easiest way is with Homebrew (a package manager for Mac).

First install Homebrew (if you don't have it) by running in Terminal:

```
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

Then install PostgreSQL:

```
brew install postgresql@16
brew services start postgresql@16
```

Windows: Download the installer from **https://www.postgresql.org/download/windows/** and run it. During installation, you'll be asked to set a password for the `postgres` user — remember this password.

Linux (Ubuntu/Debian):

```
sudo apt update
sudo apt install postgresql postgresql-contrib
sudo systemctl start postgresql
```

Verify it works

```
psql --version
```

Should output something like `psql (PostgreSQL) 16.2`.

Create the Study-Hub database

After installing, you need to create the database that Study-Hub will use:

```
psql -U postgres -c "CREATE DATABASE studyhub;"
```

If it asks for a password, use the one you set during installation.

2.4 VS Code (Visual Studio Code)

What is it?

A **code editor** — a text editor designed specifically for writing code. Unlike Notepad or TextEdit, it understands code and helps you write it:

- It colors different parts of code differently (called **syntax highlighting**) so you can read it more easily
- It auto-completes common patterns
- It shows errors before you run the code
- It has a built-in file explorer

Why do we need it?

You could technically write code in Notepad. But a proper code editor makes you 10x more productive and catches many mistakes before they happen.

How to install it

Go to **https://code.visualstudio.com** and download the version for your operating system. Install it normally.

Recommended extensions

Open VS Code, click the Extensions icon (looks like four squares) in the left sidebar, and install:

- **Prettier** — automatically formats your code to look consistent
- **ESLint** — highlights JavaScript errors as you type
- **PostgreSQL** (by Chris Kolkman) — helps with SQL files

2.5 Git

What is it?

Git is version control software. It tracks every change you make to your code over time. If you break something, you can go back to the last working version. If you want to try a risky change, you can do it safely in a separate "branch" without breaking your main code.

Every professional developer uses Git. It's not optional for serious work.

Why do we need it?

- To deploy the app to the internet (Render.com, which we'll use, reads directly from Git)
- To track your changes and never lose work
- To collaborate with others

How to install it

Mac: Git is usually already installed. Check with:

```
git --version
```

If not, running that command will prompt you to install it.

Windows: Download from **https://git-scm.com/download/win** and install.

Linux:

```
sudo apt install git
```

Configure Git with your name and email

```
git config --global user.name "Your Name"
git config --global user.email "your@email.com"
```

2.6 An OpenRouter API Key

What is it?

Study-Hub's AI chat feature connects to **OpenRouter** — a service that gives access to powerful AI language models (like Meta's Llama 3.3 or Google's Gemma). An API key is like a password that proves you have permission to use their service.

How to get it

1. Go to **https://openrouter.ai**
2. Click "Sign Up" and create an account (it's free)
3. Once logged in, go to **Keys** in your dashboard
4. Click "Create Key" and give it a name like "Study-Hub"
5. Copy the key — it looks like `sk-or-v1-abc123...`
6. Save it somewhere safe — you won't see it again

The free tier gives you access to several powerful models with no cost.

2.7 Setting Up the Project

Now let's create the project and install all its dependencies.

Step 1: Create a folder

```
mkdir study-hub
cd study-hub
```

This creates a folder called `study-hub` and moves into it. (`mkdir` = make directory, `cd` = change directory)

Step 2: Create the package.json

`package.json` is a file that describes your project and lists all the libraries it depends on. Create it:

```
npm init -y
```

The `-y` flag says "yes to all default questions." This creates a basic `package.json`.

Step 3: Install all dependencies

```
npm install express cors helmet morgan express-rate-limit express-validator bcryptjs jsonwebtoken multer
```

Let's understand what each of these is:

Package	What it does
express	The web server framework — handles incoming requests and sends responses
cors	Cross-Origin Resource Sharing — controls which websites can talk to your server
helmet	Adds security headers to every response — protects against common attacks
morgan	Logs every request to the console so you can see what's happening
express-rate-limit	Limits how many requests a user can make (prevents abuse)
express-validator	Validates that data sent to the server is correct before processing it
bcryptjs	Hashes (scrambles) passwords so they're never stored as plain text
jsonwebtoken	Creates and verifies JWT tokens — how users stay logged in
multer	Handles file uploads — specifically the PDF upload feature
node-fetch	Makes HTTP requests from Node.js (to talk to the OpenRouter AI API)
pg	PostgreSQL client — lets Node.js talk to the database
pdf-parse	Extracts text from PDF files
uuid	Generates unique IDs (like ac2c0621-89c1-4925-...)
dotenv	Reads configuration from a .env file

Step 4: Install development dependencies

```
npm install --save-dev nodemon
```

nodemon is a development tool that automatically restarts your server whenever you change a file. Without it, you'd have to manually stop and restart the server every time you make a change. The --save-dev flag means "only needed during development, not in production."

Step 5: Create the .env file

Create a file called .env in the root of your project (the main folder):

```
NODE_ENV=development
PORT=3000

DB_HOST=localhost
DB_PORT=5432
DB_NAME=studyhub
DB_USER=postgres
DB_PASSWORD=your_postgres_password_here

JWT_SECRET=make_this_a_long_random_string_at_least_32_characters
```

```
JWT_EXPIRES_IN=7d

GEMINI_API_KEY=your_openrouter_key_here

UPLOAD_DIR=./uploads/pdfs
MAX_FILE_SIZE_MB=50

ALLOWED_ORIGINS=http://localhost:3000
```

Replace `your_postgres_password_here` with your actual PostgreSQL password and `your_openrouter_key_here` with your OpenRouter API key.

What is a .env file?

`.env` stands for "environment variables." It's a file containing configuration values that: 1. Change between environments (your laptop vs. the production server) 2. Must be kept secret (passwords, API keys)

Never commit your `.env` file to Git. It contains secrets. Create a `.gitignore` file that tells Git to ignore it:

```
echo ".env" >> .gitignore
echo "node_modules/" >> .gitignore
echo "uploads/" >> .gitignore
```

The >> operator appends text to a file (creating it if it doesn't exist). `node_modules/` is the folder where all your installed packages live — it's huge (often thousands of files) and doesn't need to be in Git because anyone can regenerate it with `npm install`.

2.8 Your Environment is Ready

At this point you have: - □ Node.js installed (to run JavaScript on the server) - □ PostgreSQL installed and running (to store data) - □ VS Code installed (to write code) - □ Git installed (to track changes) - □ An OpenRouter API key (for AI features) - □ A project folder with all packages installed

In the next section, we start building — beginning with HTML.

Section 03 — HTML Foundations

The skeleton of every web page, explained from zero

3.1 What Is HTML?

HTML stands for HyperText Markup Language. That's a complicated name for a simple idea: HTML is a way of *labelling* content to tell the browser what it is.

Imagine you write a document. You might underline a title, put some text in a box, add a list. HTML does the same thing, but with special labels called **tags** that the browser understands.

HTML is not a programming language — it doesn't do logic or make decisions. It simply describes structure. It's the skeleton of a web page.

3.2 How HTML Tags Work

An HTML tag looks like this:

```
<tagname>content goes here</tagname>
```

There's an **opening tag** (`<tagname>`) and a **closing tag** (`</tagname>`). The `/` before the name is what makes it a closing tag. The content between them is affected by that tag.

For example:

```
<h1>Welcome to Study-Hub</h1>
```

This creates a large heading that says "Welcome to Study-Hub."

Some tags don't wrap content — they stand alone. These are called **self-closing tags**:

```
<input type="text" placeholder="Search..." />
```

This creates a text input box. There's no content inside it, so there's no closing tag (the `/` at the end is the self-close marker).

3.3 Attributes

Tags can have **attributes** — extra information attached to them. They always go inside the opening tag:

```
<input type="email" id="login-email" class="form-input" placeholder="you@example.com" required />
```

Let's read each attribute: - `type="email"` — tells the browser this input expects an email address (it will show a keyboard with `@` on mobile) - `id="login-email"` — gives this element a unique name so JavaScript can find it - `class="form-input"` — groups it with other elements so CSS can style them all together - `placeholder="you@example.com"` — shows faint hint text when the field is empty - `required` — the form won't submit unless this field is filled in

---

3.4 The Anatomy of an HTML Document

Every HTML file has a standard structure. Here is Study-Hub's `index.html` (simplified):

```
<!DOCTYPE html>
<html lang="en" data-theme="dark">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>Study-Hub</title>
  <link rel="stylesheet" href="/css/variables.css" />
</head>
<body>
  <!-- Page content goes here -->
</body>
</html>
```

Let's explain every line:

```
<!DOCTYPE html>
```

This is a declaration (not a tag) that tells the browser "this is an HTML5 document." It must be the very first line of every HTML file. Without it, browsers may enter a weird "quirks mode" and render things incorrectly.

```
<html lang="en" data-theme="dark">
```

This is the root element — everything else goes inside it. All HTML documents start and end with this tag.

- `lang="en"` — tells the browser (and screen readers) that the content is in English. This helps accessibility tools.
- `data-theme="dark"` — a **custom data attribute**. Attributes starting with `data-` are for storing custom information. Study-Hub uses this to know which theme is active, and CSS reads it to apply different colors. When you switch themes, JavaScript changes this attribute, and CSS instantly reacts.

```
<head>
```

The `<head>` section contains information *about* the page that the browser needs, but which isn't directly shown to the user. Things like: - The page title (shown in the browser tab) - Links to CSS files - Links to JavaScript files - Meta information (character encoding, mobile settings)

```
<meta charset="UTF-8" />
```

Character encoding tells the browser how to interpret the bytes in the file as text. UTF-8 can represent characters from virtually every language — emoji, Arabic letters, Chinese characters, accented letters. Always use UTF-8.

```
<meta name="viewport" content="width=device-width, initial-scale=1.0" />
```

This is the **viewport meta tag** — essential for mobile-friendly websites.

Without it, mobile browsers would zoom out to show the full desktop version of a page (because they're trying to be helpful but it looks terrible). This tag tells the browser: "make the page as wide as the device screen, and don't zoom in or out by default."

`width=device-width` — make the layout width equal to the device's screen width `initial-scale=1.0` — start at 100% zoom (no zoom)

```
<title>Study-Hub</title>
```

The text shown in the browser's tab and in search engine results. Simple and important.

```
<link rel="stylesheet" href="/css/variables.css" />
```

This connects a CSS file to the HTML. The browser will download and apply that CSS. - `rel="stylesheet"` — tells the browser this linked file is a CSS stylesheet - `href="/css/variables.css"` — the path to the file (`/` means "from the root of the website")

Study-Hub links 11 CSS files in order. Order matters — later files can override earlier ones.

<body>

Everything visible on the page goes inside <body>. Every button, text, image, input, panel — all of it.

3.5 The Common Tags Used in Study-Hub

Headings: <h1> through <h6>

HTML has six levels of headings. <h1> is the most important (biggest), <h6> is the least.

```
<h1>This is a very big heading</h1>
<h2>This is a large heading</h2>
<h3>This is a medium heading</h3>
```

Study-Hub uses these for section titles in notes and inside modals.

Paragraphs: <p>

For blocks of text:

```
<p>Your personal knowledge system</p>
```

**Divisions: **

 is the most generic tag — it's just a box. It has no meaning by itself, but it's used everywhere to group things and apply layout/styling.

```
<div class="auth-panel">
  <!-- everything in the login panel -->
</div>
```

In Study-Hub, virtually every visual section is a (or <section>) with a specific class.

**Spans: **

Like but for *inline* content (doesn't create a new line). Used for styling part of a sentence:

```
<span class="brand-study">Study</span><span class="brand-hub">Hub</span>
```

This lets "Study" and "Hub" be styled differently within the same word.

Buttons: <button>

Clickable elements:

```
<button type="submit" class="btn btn-primary btn-full" id="login-btn">
  <span class="btn-text">Sign In</span>
  <span class="btn-loader hidden"> </span>
</button>
```

Notice this button contains two s. One shows "Sign In" normally. The other shows a spinning icon while loading — and is hidden by default (class="hidden"). JavaScript toggles them. This is a common pattern.

type="submit" — when clicked inside a form, this triggers the form's submit event.

Anchors (links): `<a>`

```
<a href="#" id="show-register">Create one</a>
```

`href="#"` means "go nowhere" — the `#` prevents the page from reloading. The actual behaviour (showing the register form) is handled by JavaScript.

Forms: `<form>`

Groups input fields that belong together. When submitted, the browser collects all the input values.

```
<form id="login-form" class="auth-form" autocomplete="off">
  <!-- inputs and buttons go here -->
</form>
```

- `id="login-form"` — JavaScript uses this to listen for the submit event
- `autocomplete="off"` — tells the browser not to auto-fill saved passwords/emails (we handle that ourselves)

Inputs: `<input>`

The fundamental way users enter data:

```
<input type="email" id="login-email" class="form-input" placeholder="you@example.com" required />
<input type="password" id="login-password" class="form-input" placeholder="••••••••" required />
<input type="text" id="reg-name" class="form-input" placeholder="Your name" required />
```

The `type` attribute changes what kind of input it is: - `type="email"` — validates email format, shows email keyboard on mobile - `type="password"` — hides the typed characters as dots - `type="text"` — plain text input

Labels: `<label>`

A text label for an input. Very important for accessibility:

```
<label class="form-label">Email</label>
<input type="email" ... />
```

When a label is properly linked to an input, clicking the label focuses the input. You can link them with `for` and `id` attributes or by nesting the input inside the label.

Headers and Navigation: `<header>`, `<nav>`

Semantic tags — they mean something specific:

```
<header class="topnav">
  <!-- top navigation bar content -->
</header>
```

`<header>` marks the header area of a page or section. It works just like a `<div>` but tells the browser (and screen readers) "this is a header."

The `hidden` class

Study-Hub uses a CSS class called `hidden` that sets `display: none`. Elements with this class are invisible and take up no space:

```
<div id="auth-screen" class="auth-screen hidden">
```

The auth screen starts hidden. When the user is not logged in, JavaScript removes the `hidden` class to show it. When logged in, the app shell is shown and auth screen is hidden. This toggling pattern is fundamental to how the whole app works.

3.6 The `data-*` Attributes in Study-Hub

Study-Hub uses custom `data-*` attributes extensively to store information directly on HTML elements:

```
<button class="toggle-password" data-target="login-password"> </button>
```

This button has `data-target="login-password"`. When JavaScript handles the click, it reads `btn.dataset.target` (which equals `"login-password"`), then finds the input with that ID and toggles whether the password is visible. This is more elegant than hardcoding IDs in JavaScript.

```
<button class="pill-option" data-style="glass">Glass</button>
```

The theme switcher uses `data-style`. When clicked, JavaScript reads `btn.dataset.style` to know which theme to apply.

3.7 Study-Hub's HTML Structure at a High Level

The entire application lives in one HTML file: `frontend/index.html`. This is called a **Single-Page Application (SPA)** — instead of navigating between different HTML files, JavaScript dynamically shows and hides different sections.

The top-level structure looks like this:

```
<body>
    #auth-screen          (login/register - shown when not logged in)
        .auth-bg          (animated background blobs)
        .auth-panel       (the card with login/register forms)
            #login-form   (the login form)
            #register-form (the register form)

    #app                  (main application - shown when logged in)
        header.topnav     (top navigation bar)
            .topnav-left  (hamburger + logo)
            .topnav-center (search bar)
            .topnav-right (theme switcher + user avatar)

        .app-body         (sidebar + content area)
            aside.sidebar (left navigation panel)
                #notes-nav
                #pdfs-nav
                #chat-nav

            main.content  (where views render)
                #notes-view
                #pdfs-view
                #chat-view

    #toast-container      (floating notification messages)
```

Each section is shown or hidden by adding/removing the `hidden` class. Only one view (`notes`, `pdfs`, or `chat`) is visible at a time.

3.8 Comments in HTML

```
<!-- This is a comment - the browser ignores it -->
```

Use comments to explain your code. Study-Hub uses them extensively to mark sections:

```
<!--   AUTH SCREEN                          -->
```

3.9 Key Takeaways

- HTML describes *what* is on a page, not how it looks
- Tags come in pairs: opening `<tag>` and closing `</tag>`
- Attributes give extra information to tags (`id`, `class`, `type`, `href`, etc.)
- `id` identifies a single unique element; `class` groups multiple elements
- Study-Hub is a Single-Page Application — one HTML file, sections shown/hidden by JavaScript
- `data-*` attributes store custom information on elements for JavaScript to read
- The `hidden` class (`display: none`) is the primary mechanism for showing/hiding things

In the next section, we'll learn CSS — how all of this is made to look beautiful.

Section 04 — CSS Foundations

How the web becomes beautiful — styling, layout, themes, animations

---

4.1 What Is CSS?

CSS stands for Cascading Style Sheets. It's a language that describes *how HTML elements should look*.

If HTML is the skeleton of a webpage, CSS is the skin, clothes, colours, and positioning. Without CSS, every website would look like a plain document — black text on a white background with default browser fonts.

The "Cascading" part is important: styles can cascade (flow down) from parent elements to child elements, and later rules can override earlier ones. The order and specificity of CSS rules matters.

4.2 How CSS Works — The Basic Syntax

CSS is a series of **rules**. Each rule has: 1. A **selector** — which HTML elements to style 2. A **declaration block** — inside `{}`, a list of property/value pairs

```
selector {
  property: value;
  another-property: another-value;
}
```

For example:

```
.btn {
  background-color: #6c63ff;
  color: white;
  padding: 12px 24px;
  border-radius: 8px;
}
```

This styles every element with the class `btn`: purple background, white text, some padding, rounded corners.

4.3 Selectors — How CSS Finds Elements

Element selector

Styles every element of that type:

```
h1 { font-size: 32px; }        /* all h1 elements */
button { cursor: pointer; }    /* all buttons */
```

Class selector (`.`)

Styles every element with that class:

```
.form-input { border: 1px solid #333; }    /* elements with class="form-input" */
.hidden { display: none; }                  /* elements with class="hidden" */
```

ID selector (`#`)

Styles one specific element (IDs must be unique):

```
#login-form { display: flex; flex-direction: column; }
```

Attribute selector

Styles elements with a specific attribute:

```
[data-theme="dark"] { background: #0f0f0f; }
```

This is exactly how Study-Hub's theme system works — CSS reads the `data-theme` attribute on `<html>` and applies different colors.

Descendant selector (space between selectors)

Styles elements that are inside other elements:

```
.auth-panel .form-input { width: 100%; }
/* form inputs that are inside an element with class="auth-panel" */
```

Pseudo-classes (`:`)

Styles elements in special states:

```
.btn:hover { opacity: 0.85; }           /* when mouse hovers */
.btn:active { transform: scale(0.98); } /* when being clicked */
input:focus { border-color: var(--accent); } /* when selected/focused */
```

4.4 The Box Model — How Space Works

Every HTML element is a box. The **CSS Box Model** describes the layers of that box:

```
MARGIN                  ← space outside the element

 BORDER                 ← the visible border

  PADDING               ← space between border and content

  CONTENT               ← the actual text/image
```

In Study-Hub's CSS:

```
--sp-4: 16px;   /* a spacing token for 16 pixels */

.form-group {
  margin-bottom: var(--sp-4);  /* 16px space below each form group */
}

.btn {
  padding: var(--sp-3) var(--sp-6);  /* 12px top/bottom, 24px left/right */
}
```

4.5 CSS Variables (Custom Properties)

Study-Hub uses CSS variables extensively. They look like this:

```
:root {
  --accent: #f0ede8;
  --bg: #0f0f0f;
  --sp-4: 16px;
}
```

The `:root` selector means "the very top of the document" — it's the global scope for CSS variables.

To use them:

```
.some-element {
  color: var(--accent);
  background: var(--bg);
```

```
  margin: var(--sp-4);
}
```

`var(--name)` reads the value of that variable.

Why use variables instead of hardcoding colors?

Because Study-Hub has four themes: Minimal Dark, Minimal Light, Glass Dark, Glass Light. Each theme needs different colors. Instead of writing four separate versions of every CSS rule, you write it once using variables, and only change the variable values per theme:

```
/* Minimal Dark theme */
[data-theme="minimal-dark"] {
  --bg: #0f0f0f;
  --text: #f0ede8;
  --accent: #f0ede8;
}

/* Minimal Light theme */
[data-theme="minimal-light"] {
  --bg: #f8f5f0;
  --text: #1a1714;
  --accent: #1a1714;
}
```

Every component that uses `var(--bg)` automatically gets the right color for the active theme. This is elegant, maintainable design.

Study-Hub's spacing tokens

Rather than scattering `16px` and `24px` everywhere, Study-Hub defines a consistent spacing system:

```
--sp-1:  4px;
--sp-2:  8px;
--sp-3:  12px;
--sp-4:  16px;
--sp-5:  20px;
--sp-6:  24px;
--sp-8:  32px;
--sp-10: 40px;
--sp-12: 48px;
--sp-16: 64px;
```

This makes the design consistent — spacing is always multiples of 4px.

Border radius tokens

```
--r-sm:   6px;
--r-md:   10px;
--r-lg:   16px;
--r-xl:   24px;
--r-full: 9999px;  /* makes anything a perfect pill/circle */
```

Shadow tokens

```
--shadow-xs: 0 1px 3px rgba(0,0,0,.12);
--shadow-sm: 0 2px 8px rgba(0,0,0,.16);
--shadow-md: 0 4px 20px rgba(0,0,0,.22);
```

Shadow syntax: `offset-x offset-y blur-radius color`

4.6 Flexbox — The Layout System

Flexbox is how Study-Hub (and most modern websites) arrange elements in rows or columns.

When you set `display: flex` on an element, its direct children become **flex items** that can be arranged, aligned, and distributed automatically.

```
.topnav {
  display: flex;
  align-items: center;       /* center items vertically */
  justify-content: space-between;  /* spread items across the width */
}
```

The top navigation bar has three sections (left, center, right). With `justify-content: space-between`, they spread out automatically.

Flex direction

```
.auth-form {
  display: flex;
  flex-direction: column;  /* stack items vertically */
  gap: var(--sp-4);        /* 16px space between each item */
}
```

`gap` is a modern CSS property that adds space between flex/grid items — much cleaner than using margins.

Common flex properties

Property	What it does	Common values
`flex-direction`	Which direction items go	`row` (default), `column`
`justify-content`	Spacing along the main axis	`flex-start`, `center`, `flex-end`, `space-between`, `space-around`
`align-items`	Alignment on the cross axis	`stretch` (default), `center`, `flex-start`, `flex-end`
`gap`	Space between items	any size value
`flex-wrap`	Whether items wrap to new lines	`nowrap` (default), `wrap`
`flex: 1`	Item grows to fill available space	(on the flex item itself)

Study-Hub's sidebar/content layout

```
.app-body {
  display: flex;
```

```
  height: calc(100vh - var(--topnav-h));  /* full height minus nav bar */
}

.sidebar {
  width: var(--sidebar-w);   /* 240px fixed width */
  flex-shrink: 0;            /* don't shrink */
}

.content {
  flex: 1;                   /* fill all remaining space */
  overflow: hidden;
}
```

`calc()` is a CSS function that does math — here it calculates the height as 100% of the viewport height minus the navigation bar height.

`100vh` means "100% of the viewport height" (vh = viewport height).

4.7 CSS Grid

While Flexbox handles one-dimensional layouts (rows OR columns), **CSS Grid** handles two-dimensional layouts (rows AND columns simultaneously).

Study-Hub uses Grid for the notes card layout:

```
.notes-grid {
  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(280px, 1fr));
  gap: var(--sp-4);
}
```

This is a powerful one-liner: - `repeat(auto-fill, ...)` — create as many columns as fit - `minmax(280px, 1fr)` — each column is at least 280px wide, and grows to fill remaining space equally (`1fr` = one fraction of available space)

The result: a responsive grid that automatically adjusts the number of columns based on screen width. No JavaScript needed.

4.8 Positioning

CSS has several positioning modes:

`position: relative`

The element stays in its normal flow position but you can offset it with `top`, `left`, etc. Also creates a **positioning context** for children with `absolute`.

`position: absolute`

The element is removed from normal flow and positioned relative to its nearest `relative` ancestor:

```
.btn-loader {
  position: absolute;
  top: 50%;
  left: 50%;
  transform: translate(-50%, -50%);  /* center it */
}
```

`position: fixed`

Positioned relative to the viewport — it stays in place when you scroll:

```
.topnav {
  position: fixed;
  top: 0;
  left: 0;
  right: 0;
  z-index: 100;  /* stays above other elements */
}

#toast-container {
  position: fixed;
  bottom: var(--sp-6);
  right: var(--sp-6);
  z-index: 1000;
}
```

`z-index` controls stacking order — higher numbers appear in front.

4.9 Transitions and Animations

Transitions — smooth property changes

```
.btn {
  transition: opacity var(--t-fast), transform var(--t-fast);
}

.btn:hover {
  opacity: 0.85;
  transform: translateY(-1px);  /* slight upward movement */
}
```

When the user hovers, instead of instantly changing, CSS smoothly animates from the original to the hover state over the `--t-fast` duration (`160ms`).

Study-Hub's transition tokens:

```
--t-fast:   160ms cubic-bezier(.4, 0, .2, 1);
--t-base:   300ms cubic-bezier(.4, 0, .2, 1);
--t-slow:   460ms cubic-bezier(.4, 0, .2, 1);
--t-spring: 500ms cubic-bezier(.34, 1.2, .64, 1);
```

`cubic-bezier` controls the animation curve (acceleration/deceleration). The spring one overshoots slightly, creating a bouncy feel.

Keyframe Animations

For more complex animations, CSS uses `@keyframes`:

```
@keyframes fadeInUp {
  from {
    opacity: 0;
    transform: translateY(20px);
  }
  to {
    opacity: 1;
    transform: translateY(0);
  }
}

.note-card {
  animation: fadeInUp var(--t-base) ease forwards;
}
```

This makes note cards fade in and slide up slightly when they appear.

Study-Hub also uses animations for the background blobs on the auth screen:

```
@keyframes blobFloat {
  0%, 100% { transform: translate(0, 0) scale(1); }
  33%       { transform: translate(30px, -40px) scale(1.05); }
  66%       { transform: translate(-20px, 20px) scale(0.95); }
}

.auth-blob {
  animation: blobFloat 8s ease-in-out infinite;
}
```

This creates a gentle, floating, organic movement on the coloured blobs in the background.

4.10 Glassmorphism

Study-Hub's glass theme uses a popular modern design style called **glassmorphism** — elements look like frosted glass.

The key CSS property is `backdrop-filter`:

```
[data-theme^="glass"] .sidebar {
  background: rgba(255, 255, 255, 0.05);
  backdrop-filter: blur(20px) saturate(180%);
  -webkit-backdrop-filter: blur(20px) saturate(180%);
  border: 1px solid rgba(255, 255, 255, 0.1);
}
```

- `background: rgba(255,255,255,0.05)` — almost transparent white
- `backdrop-filter: blur(20px)` — blurs everything behind the element
- `border: 1px solid rgba(255,255,255,0.1)` — a subtle white edge
- `-webkit-backdrop-filter` — the same property for Safari (webkit prefix required)

The selector `[data-theme^="glass"]` means "any element with a `data-theme` attribute that starts with (`^=`) the word glass."

4.11 The CSS Reset

Study-Hub's first CSS file is `reset.css`. This removes the default browser styles — different browsers have different defaults, which cause inconsistencies. A reset gives you a clean slate:

```
*, *::before, *::after {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}
```

- `box-sizing: border-box` — this is crucial. By default, when you set `width: 200px` on an element, that's the content width. With `border-box`, it includes padding and border. This makes sizing far more predictable.
- `margin: 0; padding: 0` — removes all default spacing

4.12 Responsive Design

Study-Hub adapts to different screen sizes using **media queries**:

```
@media (max-width: 768px) {
  .sidebar {
    transform: translateX(-100%);  /* slide sidebar off screen on mobile */
  }

  .notes-grid {
    grid-template-columns: 1fr;  /* single column on mobile */
  }
}
```

`@media (max-width: 768px)` means "apply these rules only when the screen is 768px wide or less." This is how websites become "responsive" — they adapt their layout for phones, tablets, and desktops.

4.13 The `hidden` Class

This single class is used constantly throughout Study-Hub:

```
.hidden {
  display: none !important;
}
```

`display: none` completely removes the element from the visual layout — it's invisible and takes up no space.

`!important` overrides any other `display` value that might be set. This ensures that when JavaScript adds `hidden`, nothing else can accidentally make it visible.

4.14 Study-Hub's CSS File Architecture

The CSS is split into 11 files, loaded in order:

File	Purpose
`variables.css`	All CSS custom properties (colors, spacing, shadows, fonts) for all four themes
`reset.css`	Remove browser defaults
`base.css`	Typography, scrollbars, base element styles
`components.css`	Reusable UI components: buttons, badges, modals
`layout.css`	The overall page structure: topnav, sidebar, content area
`notes.css`	Notes-specific styles: cards, editor, folders
`pdfs.css`	PDF viewer, upload, library styles
`chat.css`	Chat interface: messages, sessions sidebar
`auth.css`	Login/register screen
`animations.css`	Keyframe animations, entrances, toast animations
`glassmorphic.css`	Glass theme overrides

This separation makes the codebase organized — you always know where to look for a specific style.

4.15 Key Takeaways

- CSS is rules: selector + property/value pairs
- Classes (`.`) style groups of elements; IDs (`#`) style one specific element
- CSS variables (`--name`) enable theming and consistency
- Flexbox arranges items in rows or columns
- Grid arranges items in two-dimensional grids
- Transitions and animations make the UI feel alive
- Media queries make layouts responsive
- Study-Hub switches themes by changing the `data-theme` attribute on `<html>`, which triggers different CSS variable values

Next: JavaScript — the engine that makes everything interactive.

Section 05 — JavaScript Foundations

The language that makes the web interactive — from variables to async/await

---

5.1 What Is JavaScript?

JavaScript is the only programming language that browsers understand natively. Where HTML describes *what* is on a page and CSS describes *how it looks*, JavaScript describes *what happens* — responding to clicks, fetching data, updating the screen, making decisions.

JavaScript is also used on the server (via Node.js) to handle requests, talk to the database, and serve responses. Study-Hub uses JavaScript for everything.

5.2 Where JavaScript Runs

In the browser: JavaScript is downloaded with the HTML page and runs inside the user's browser. It can manipulate the page, respond to user actions, and send requests to the server.

On the server (Node.js): The same JavaScript language, but running on a server. It receives HTTP requests, processes them, and sends back responses.

5.3 Variables — Storing Information

A variable is a named container for a value. You create one like this:

```
let message = "Hello, world!";
const maxFileSize = 50;
var oldStyle = true;  // don't use this
```

`let` — changeable variable

```
let count = 0;
count = count + 1;  // now count is 1
count = 5;          // now count is 5
```

Use `let` when the value will change.

`const` — unchangeable reference

```
const PORT = 3000;
PORT = 4000;  // ERROR - cannot reassign a const
```

Use `const` when the value should not change. This is the default in Study-Hub — most variables are `const`.

Note: `const` with objects/arrays doesn't mean the contents can't change — it means you can't reassign the variable itself:

```
const user = { name: "Alice" };
user.name = "Bob";     // ALLOWED - changing a property
user = { name: "Charlie" };  // ERROR - reassigning the variable
```

Data types

```
// String - text
const name = "Alice";
const greeting = `Hello, ${name}!`;  // template literal - embeds variables

// Number - integers and decimals
const age = 25;
const price = 19.99;

// Boolean - true or false
const isLoggedIn = true;
const hasError = false;

// Null - intentionally empty
const selectedNote = null;
```

```
// Undefined - not yet assigned
let currentUser;  // undefined until assigned

// Array - ordered list
const colors = ['#6c63ff', '#ff6584', '#43b97f'];
const tags = [];  // empty array

// Object - collection of named properties
const user = {
  id: 'abc-123',
  name: 'Alice',
  email: 'alice@example.com',
  avatarColor: '#6c63ff'
};
```

5.4 Functions — Reusable Instructions

A function is a named block of code that does something. You define it once and call it many times.

```
// Function declaration
function greet(name) {
  return `Hello, ${name}!`;
}

// Arrow function (modern, more concise)
const greet = (name) => {
  return `Hello, ${name}!`;
};

// Even shorter arrow function (for simple one-liners)
const greet = (name) => `Hello, ${name}!`;

// Calling a function
const message = greet("Alice");
console.log(message);  // "Hello, Alice!"
```

Parameters vs Arguments

- **Parameters** are the variable names listed in the function definition (`name` above)
- **Arguments** are the actual values passed when calling the function (`"Alice"` above)

Functions in Study-Hub

Study-Hub's helpers use this pattern constantly:

```
// From helpers.js
const formatDate = (dateStr) => {
  if (!dateStr) return '';
  const date = new Date(dateStr);
  const now = new Date();
  const diffMs = now - date;
```

```
  const diffMins = Math.floor(diffMs / 1000 / 60);

  if (diffMins < 60) return `${diffMins}m ago`;
  // ... etc
  return date.toLocaleDateString('en-US', { month: 'short', day: 'numeric' });
};
```

This function takes a date string, calculates how long ago it was, and returns a human-friendly string like "5m ago" or "yesterday."

5.5 Control Flow — Making Decisions

if/else

```
const age = 20;

if (age >= 18) {
  console.log("Adult");
} else if (age >= 13) {
  console.log("Teenager");
} else {
  console.log("Child");
}
```

Ternary operator — compact if/else

```
const label = age >= 18 ? "Adult" : "Minor";
// means: if age >= 18 then "Adult", else "Minor"
```

Study-Hub uses this everywhere:

```
// From server.js
const PORT = process.env.PORT || 3000;
// means: use process.env.PORT if it exists, otherwise use 3000
```

`||` is the OR operator. `process.env.PORT || 3000` means "use PORT if it's truthy, otherwise use 3000."

Early returns — guard clauses

Study-Hub uses early returns to handle errors before doing real work:

```
const login = async (req, res, next) => {
  const { email, password } = req.body;

  if (!email || !password) {
    return res.status(400).json({ error: 'Email and password are required' });
    // function stops here if missing fields
  }

  // only reaches here if both fields are present
  // ... actual login logic
};
```

5.6 Arrays — Working with Lists

Arrays store ordered collections of values. They are fundamental to Study-Hub — notes, PDFs, chat sessions, messages are all arrays.

```
const notes = ['note1', 'note2', 'note3'];

// Access by index (zero-based)
console.log(notes[0]);  // 'note1'
console.log(notes[2]);  // 'note3'

// Length
console.log(notes.length);  // 3

// Add to end
notes.push('note4');

// Remove from end
const last = notes.pop();

// Does it include something?
notes.includes('note1');  // true
```

Array methods — the most important ones

```
const numbers = [1, 2, 3, 4, 5];

// .map() - transform each item, returns new array
const doubled = numbers.map(n => n * 2);  // [2, 4, 6, 8, 10]

// .filter() - keep only items that pass a test
const evens = numbers.filter(n => n % 2 === 0);  // [2, 4]

// .find() - get first item that passes a test
const first_big = numbers.find(n => n > 3);  // 4

// .forEach() - do something with each item (no return value)
numbers.forEach(n => console.log(n));

// .join() - combine array into a string
['a', 'b', 'c'].join(', ');  // "a, b, c"

// .some() - does any item pass the test?
numbers.some(n => n > 4);  // true

// .every() - do all items pass the test?
numbers.every(n => n > 0);  // true
```

Study-Hub uses these constantly. For example, building SQL placeholders:

```
const noteIds = ['id1', 'id2', 'id3'];
const placeholders = noteIds.map((_, i) => `$${i + 2}`).join(',');
// placeholders = "$2,$3,$4"
```

_ is a convention meaning "I don't care about this argument" (the value), `i` is the index.

5.7 Objects — Working with Named Data

Objects store named properties. JSON responses from the backend are objects.

```
const user = {
  id: 'abc-123',
  name: 'Alice',
  email: 'alice@example.com'
};

// Access properties
console.log(user.name);          // 'Alice' (dot notation)
console.log(user['email']);      // 'alice@example.com' (bracket notation)

// Modify
user.name = 'Bob';

// Add new property
user.avatarColor = '#6c63ff';

// Check if property exists
'name' in user;          // true
user.hasOwnProperty('phone');  // false
```

Destructuring — unpacking objects

```
const { name, email } = user;
// same as:
// const name = user.name;
// const email = user.email;

// With renaming
const { avatar_color: avatarColor } = user;

// In function parameters
const login = async ({ email, password }) => {
  // can use email and password directly
};
```

Study-Hub uses destructuring constantly in controllers:

```
const { content, context_note_ids, context_pdf_ids } = req.body;
```

Spread operator ...

```
const base = { method: 'GET', timeout: 5000 };
const withAuth = { ...base, headers: { Authorization: 'Bearer token' } };
// { method: 'GET', timeout: 5000, headers: { ... } }
```

`...base` copies all properties of `base` into the new object. This is used in Study-Hub's API utility to merge headers:

```
const headers = {
  'Content-Type': 'application/json',
  ...(token ? { 'Authorization': `Bearer ${token}` } : {}),
};
```

5.8 The DOM — JavaScript Talking to the Page

DOM stands for Document Object Model. It's the browser's internal representation of the HTML page as a tree of objects that JavaScript can read and modify.

Finding elements

```
// By ID - returns one element or null
const loginForm = document.getElementById('login-form');

// By CSS selector - returns first match
const btn = document.querySelector('.btn-primary');

// By CSS selector - returns all matches
const allCards = document.querySelectorAll('.note-card');
// returns a NodeList (array-like)
```

Study-Hub's helpers module shortens these:

```
const qs  = (sel, ctx = document) => ctx.querySelector(sel);
const qsa = (sel, ctx = document) => [...ctx.querySelectorAll(sel)];

// Usage:
const loginBtn = qs('#login-btn');
const allInputs = qsa('.form-input');
```

Showing and hiding elements

```
// Hide element
element.classList.add('hidden');

// Show element
element.classList.remove('hidden');

// Toggle
element.classList.toggle('hidden');

// Check if hidden
element.classList.contains('hidden');
```

Changing content

```
// Change text
element.textContent = 'New text';

// Change HTML (use carefully - risk of XSS if using user input)
element.innerHTML = '<strong>Bold text</strong>';
```

Changing attributes

```
// Read
document.documentElement.dataset.theme;  // reads data-theme on <html>
```

```
// Write
document.documentElement.dataset.theme = 'glass-dark';
// This changes the theme instantly! CSS reacts immediately.
```

Creating new elements

```
const card = document.createElement('div');
card.className = 'note-card';
card.innerHTML = `<h3>${note.title}</h3><p>${note.content}</p>`;
document.getElementById('notes-grid').appendChild(card);
```

5.9 Events — Responding to User Actions

An **event** is something that happens: a click, a key press, a form submission, the page loading.

You **listen** for events with `addEventListener`:

```
const button = document.getElementById('login-btn');

button.addEventListener('click', () => {
  console.log('Button was clicked!');
});
```

The second argument is a **callback function** — a function that runs when the event happens.

Form submission event

```
const form = document.getElementById('login-form');

form.addEventListener('submit', (event) => {
  event.preventDefault();  // CRITICAL - prevents the browser's default form submit (which would reload

  const email = document.getElementById('login-email').value;
  const password = document.getElementById('login-password').value;

  // now handle the login
});
```

`event.preventDefault()` stops the browser's default behaviour. Without it, submitting a form causes a full page reload — not what we want in a SPA.

Common events

Event	When it fires
`click`	Element is clicked
`submit`	Form is submitted
`input`	Input value changes
`change`	Input loses focus after value changed
`keydown`	A key is pressed
`focus`	Element gains focus
`blur`	Element loses focus

Custom events

Study-Hub uses custom events to communicate between modules:

```
// Dispatching (in api.js when session expires)
window.dispatchEvent(new CustomEvent('auth:expired'));

// Listening (in app.js)
window.addEventListener('auth:expired', () => {
  // log the user out
});
```

This is like a radio signal — one module broadcasts, another listens, and they don't need to know about each other directly.

5.10 Async JavaScript — Promises and async/await

This is the most important advanced concept for Study-Hub. Almost everything involves waiting: waiting for the database, waiting for the AI, waiting for file uploads.

The problem: JavaScript is single-threaded

JavaScript runs one thing at a time. If you waited for a database query the "normal" way, everything would freeze until it finished. The solution is **asynchronous** code — starting an operation and saying "call me back when it's done."

Promises

A **Promise** is an object representing a future value. It can be: - **Pending** — still waiting - **Resolved** — completed successfully (has a value) - **Rejected** — failed (has an error)

```
const promise = fetch('https://api.example.com/data');

promise
  .then(response => response.json())   // when resolved: parse JSON
  .then(data => console.log(data))     // when that resolves: use data
  .catch(error => console.error(error)); // if anything fails: handle error
```

async/await — the modern way

`async/await` is cleaner syntax for working with Promises. It makes async code read like synchronous code.

```
// An async function always returns a Promise
const fetchData = async () => {
  try {
    const response = await fetch('https://api.example.com/data');
    // await pauses here until the fetch completes

    const data = await response.json();
    // await pauses here until JSON parsing completes

    return data;
  } catch (error) {
```

```
    console.error('Something went wrong:', error);
  }
};
```

`await` can only be used inside an `async` function.

How Study-Hub uses async/await everywhere

Every backend route handler is async:

```
// From authController.js
const login = async (req, res, next) => {
  try {
    const { email, password } = req.body;

    // Wait for database query
    const result = await query(
      'SELECT * FROM users WHERE email = $1',
      [email.toLowerCase()]
    );

    if (result.rows.length === 0) {
      return res.status(401).json({ error: 'Invalid credentials' });
    }

    const user = result.rows[0];

    // Wait for password comparison (bcrypt is slow on purpose for security)
    const isValid = await bcrypt.compare(password, user.password_hash);

    if (!isValid) {
      return res.status(401).json({ error: 'Invalid credentials' });
    }

    const token = generateToken(user.id);
    res.json({ success: true, token, user });
  } catch (err) {
    next(err);  // pass error to Express error handler
  }
};
```

Every frontend API call is also async:

```
// From auth.js module
const handleLogin = async (email, password) => {
  try {
    const data = await API.auth.login({ email, password });
    Storage.setToken(data.token);
    Storage.setUser(data.user);
    App.init();
  } catch (err) {
    showError('login-error', err.message);
  }
};
```

5.11 The Module Pattern (IIFEs)

Study-Hub's frontend JavaScript doesn't use modern ES modules (no `import`/`export`). Instead it uses a pattern called **IIFE** (Immediately Invoked Function Expression) to create isolated modules.

```
const Helpers = (() => {
  // Private variables and functions (only accessible inside this function)
  const privateHelper = () => { /* ... */ };

  // Public API - what we expose
  const formatDate = (dateStr) => { /* ... */ };
  const toast = { /* ... */ };

  // Return only what should be public
  return {
    formatDate,
    toast,
    // ... other public functions
  };
})();  // The () at the end immediately calls the function
```

The result: `Helpers` is an object with only the functions you chose to expose. Everything else is private. This prevents naming conflicts between files.

All of Study-Hub's frontend files (`app.js`, `auth.js`, `notes.js`, `pdfs.js`, `chat.js`, `api.js`, `storage.js`, `helpers.js`) use this pattern and are loaded in order via `<script>` tags in `index.html`.

5.12 localStorage — Persisting Data in the Browser

`localStorage` stores key-value pairs that persist even after the browser closes:

```
// Store
localStorage.setItem('studyhub_token', 'eyJhbGc...');

// Read
const token = localStorage.getItem('studyhub_token');

// Delete
localStorage.removeItem('studyhub_token');
```

Study-Hub wraps this in a `Storage` module with safer, typed methods:

```
const Storage = (() => {
  const PREFIX = 'studyhub_';

  const set = (key, value) => {
    try {
      localStorage.setItem(PREFIX + key, JSON.stringify(value));
    } catch (e) {
      console.warn('Storage.set failed:', e);
    }
  };

  const get = (key, fallback = null) => {
```

```
    try {
      const raw = localStorage.getItem(PREFIX + key);
      return raw !== null ? JSON.parse(raw) : fallback;
    } catch (e) {
      return fallback;
    }
  };

  // ... etc
})();
```

The `PREFIX` (`studyhub_`) prevents conflicts with other websites' localStorage keys. `JSON.stringify` and `JSON.parse` allow storing objects and arrays (localStorage only stores strings natively).

5.13 JSON — The Language of APIs

JSON (JavaScript Object Notation) is the format used to send data between frontend and backend. It looks just like a JavaScript object, but as a string:

```
{
  "success": true,
  "user": {
    "id": "ac2c0621-89c1-4925-9c4c-40d5ace82e9a",
    "name": "Alice",
    "email": "alice@example.com",
    "avatarColor": "#6c63ff"
  },
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
}
```

- `JSON.stringify(obj)` — converts a JavaScript object to a JSON string (for sending)
- `JSON.parse(str)` — converts a JSON string to a JavaScript object (for receiving)

`response.json()` in fetch calls does the parsing for you.

5.14 Error Handling

JavaScript has a `try/catch` mechanism for handling errors gracefully:

```
try {
  // code that might throw an error
  const data = await API.notes.getAll();
  renderNotes(data.notes);
} catch (err) {
  // if anything in the try block throws, execution jumps here
  Helpers.toast.error(err.message);
  console.error('Failed to load notes:', err);
}
```

Without `try/catch`, an unhandled error would crash the current execution and potentially leave the UI in a broken state.

5.15 Debounce — Throttling Rapid Events

When users type in a search box, you don't want to send a server request for every single keystroke. **Debouncing** waits until the user stops typing for a moment before firing:

```
const debounce = (fn, delay = 300) => {
  let timer;
  return (...args) => {
    clearTimeout(timer);          // cancel the previous timer
    timer = setTimeout(() => fn(...args), delay);  // start a new one
  };
};

// Usage
const handleSearch = debounce((query) => {
  API.notes.getAll({ search: query }).then(renderNotes);
}, 400);

searchInput.addEventListener('input', (e) => handleSearch(e.target.value));
```

If the user types "study" quickly, only one request fires after they stop — not 5 (one per letter).

5.16 Key Takeaways

- Variables: `const` for fixed values, `let` for changeable ones
- Functions can be stored in variables (arrow functions)
- Destructuring unpacks objects/arrays cleanly
- The DOM is the page — JavaScript can read and modify any element
- Events are how JavaScript listens to user actions
- `async/await` makes asynchronous code (waiting for data) readable
- The IIFE module pattern creates isolated, non-conflicting code files
- JSON is the language of web APIs
- `try/catch` prevents errors from crashing the app

In the next section, we walk through the entire project folder structure before diving into the backend and frontend code in detail.

Section 06 — Project Structure

Why every file and folder exists, and where everything lives

6.1 The Full Folder Tree

Here is every important file and folder in Study-Hub, and what it does:

```
study-hub/

  backend/                          ← Server-side code (runs on the server)
     config/
        database.js                ← PostgreSQL connection and query helper
        schema.sql                 ← Database table definitions
```

```
    controllers/
        authController.js         ← Register, login, profile
        notesController.js        ← Create, read, update, delete notes
        pdfController.js          ← Upload, view, delete PDFs
        chatController.js         ← Chat sessions and AI messages
    middleware/
        auth.js                   ← JWT verification (is this user logged in?)
        errorHandler.js           ← Catches unhandled errors
        upload.js                 ← Multer configuration for file uploads
    routes/
        index.js                  ← All API endpoints defined here
    services/
        llmService.js             ← Talks to the OpenRouter AI API
        pdfService.js             ← Extracts text from PDF files
    server.js                     ← The main entry point - starts everything

frontend/                         ← Client-side code (runs in the browser)
    index.html                    ← The one HTML file for the entire app
    css/
        variables.css             ← Color/spacing/font tokens for all themes
        reset.css                 ← Remove browser default styles
        base.css                  ← Base typography and element styles
        components.css            ← Buttons, modals, badges, form elements
        layout.css                ← Page structure: nav, sidebar, content
        notes.css                 ← Notes view, cards, editor
        pdfs.css                  ← PDF library and viewer
        chat.css                  ← Chat interface
        auth.css                  ← Login/register screen
        animations.css            ← Keyframe animations
        glassmorphic.css          ← Glass theme overrides
    js/
        app.js                    ← App entry point: routing, theme, sidebar
        utils/
            helpers.js            ← Utility functions (markdown, dates, toast)
            api.js                ← HTTP request wrapper, all API calls
            storage.js            ← localStorage wrapper
        modules/
            auth.js               ← Login/register UI logic
            notes.js              ← Notes UI: list, editor, folders
            pdfs.js               ← PDF UI: upload, library, viewer
            chat.js               ← Chat UI: sessions, messages, context

scripts/
    setup.js                      ← One-time setup wizard
    db-init.js                    ← Create database and apply schema
    db-reset.js                   ← Drop and recreate all tables (dev only)

uploads/
    pdfs/
        {user-id}/                ← Uploaded PDFs organized by user
            {uuid}.pdf

.env                              ← Secret configuration (NOT in Git)
.env.example                      ← Template showing what .env should contain
```

```
.gitignore                    ← Files Git should ignore
package.json                  ← Project description + dependencies list
package-lock.json             ← Exact versions of every installed package
render.yaml                   ← Deployment configuration for Render.com
README.md                     ← Project documentation
```

6.2 Why This Structure?

Separation of concerns

The guiding principle behind this structure is **separation of concerns** — each file has one clear responsibility. This makes the code: - Easier to find ("where does PDF upload logic live?" → `pdfController.js`) - Easier to change ("I need to change how login works" → `authController.js`) - Easier to understand (reading one file doesn't require understanding everything)

Backend folder design

`config/` — Things that configure the application: database connection, schema. These never change at runtime.

`controllers/` — The actual business logic. Each controller handles one feature area. When a request comes in for `/api/notes`, the notes controller handles it. Controllers know nothing about each other.

`middleware/` — Code that runs *between* the request arriving and the controller handling it. Think of it as a pipeline:

```
Request → auth middleware → controller → response
```

The auth middleware checks "is this user logged in?" before any protected route runs. If not logged in, it stops the request and sends an error. The controller never even runs.

`routes/` — A single file mapping URL paths to controller functions. It's the "directory" of the API.

`services/` — Complex operations that don't fit neatly into a controller. The AI service and PDF parsing service are here because they're standalone operations that might be called from multiple controllers.

Frontend folder design

`css/` — Split into logical files by purpose. You always know where to find a style.

`js/utils/` — Low-level utilities that everything else depends on: - `helpers.js` — pure utility functions (no state) - `api.js` — the one place where all HTTP calls are defined - `storage.js` — the one place where localStorage is touched

`js/modules/` — Feature modules, one per major feature. Each module manages all the UI logic for its feature.

`js/app.js` — The orchestrator. It initializes everything, handles routing between views, and wires up global behaviour (theme, sidebar, search).

6.3 Load Order Matters

The JavaScript files are loaded in this order in `index.html`:

```
<!-- Utils first - everything else depends on these -->
<script src="/js/utils/storage.js"></script>
<script src="/js/utils/helpers.js"></script>
<script src="/js/utils/api.js"></script>

<!-- Feature modules - depend on utils -->
<script src="/js/modules/auth.js"></script>
<script src="/js/modules/notes.js"></script>
<script src="/js/modules/pdfs.js"></script>
<script src="/js/modules/chat.js"></script>

<!-- App entry point - depends on everything above -->
<script src="/js/app.js"></script>
```

This is important. If `app.js` loaded first, it would try to use `API.notes.getAll()` before `api.js` had created the `API` object — you'd get a "API is not defined" error.

6.4 The `node_modules` Folder

When you run `npm install`, npm creates a `node_modules` folder containing all the installed packages — and all *their* dependencies, and all *those* dependencies. This folder can easily contain thousands of files.

You never touch this folder manually. It's regenerated by `npm install` from `package.json`. That's why it's in `.gitignore` — there's no point tracking it in Git.

6.5 The `package.json` File

```
{
  "name": "study-hub",
  "version": "1.0.0",
  "description": "Personal knowledge + AI-powered study system",
  "main": "backend/server.js",
  "scripts": {
    "start": "node backend/server.js",
    "dev": "nodemon backend/server.js",
    "setup": "node scripts/setup.js",
    "db:init": "node scripts/db-init.js",
    "db:reset": "node scripts/db-reset.js"
  },
  "dependencies": { ... },
  "devDependencies": { "nodemon": "^3.1.0" }
}
```

`"main"` — the entry point when someone runs `node .`

`"scripts"` — shortcuts you run with `npm run`. `npm run dev` runs `nodemon backend/server.js`. `npm run db:init` runs the database initialization script.

`"dependencies"` — packages required in production

`"devDependencies"` — packages only needed during development (nodemon restarts the server on changes — not needed in production)

6.6 Environment Variables and the `.env` File

Your `.env` file is a list of key-value pairs that configure the application:

```
NODE_ENV=development
PORT=3000
DB_PASSWORD=mysecretpassword
JWT_SECRET=a_long_random_string
GEMINI_API_KEY=sk-or-v1-...
```

In the code, they're accessed via `process.env`:

```
const PORT = process.env.PORT || 3000;
const dbPassword = process.env.DB_PASSWORD;
```

`process` is a Node.js global object. `process.env` is an object containing all environment variables.

Why environment variables instead of hardcoding?

1. **Security** — API keys and passwords should never be in code that's shared or committed to Git
2. **Flexibility** — On your laptop, `DB_PASSWORD` is your local Postgres password. On the production server, it's a different password. Same code, different config.
3. **Multiple environments** — Development, staging, production — each can have different settings

`dotenv` (the npm package) is what reads the `.env` file and populates `process.env`:

```
require('dotenv').config();  // This is the first line of server.js
```

After that call, everything in `.env` becomes available via `process.env`.

6.7 The `uploads/` Folder

When a user uploads a PDF, Multer saves it here:

```
uploads/
  pdfs/
      ac2c0621-89c1-4925-9c4c-40d5ace82e9a/   ← user's ID
          832e87e7-25d9-45d8-b8ee-45220ecacf9b.pdf
          d1cf1873-8897-4da9-a965-c2c83eee8f36.pdf
```

Each user gets their own subfolder (named with their UUID). Each PDF gets a random UUID filename (prevents collisions and hides the original filename from the URL).

This folder is in `.gitignore` — uploaded files don't belong in Git (they're large, personal, and separate from code).

6.8 The `render.yaml` File

This file tells Render.com (our deployment service) how to run the app:

```yaml
services:
  - type: web
    name: study-hub
    runtime: node
    buildCommand: npm install
    startCommand: node backend/server.js
    envVars:
      - key: NODE_ENV
        value: production
      - key: DATABASE_URL
        fromDatabase:
          name: study-hub-db
          property: connectionString
      - key: JWT_SECRET
        generateValue: true

databases:
  - name: study-hub-db
    plan: free
```

This tells Render: install packages, then start the server. Use a free PostgreSQL database. Auto-generate a JWT secret. This makes deployment a one-click operation.

Now that you understand where everything is and why, let's build it — starting with the backend server.

Section 07 — Backend: Node.js & Express

What a server is, how Express works, and a line-by-line walkthrough of server.js

---

7.1 What Is a Server?

A **server** is just a computer program that listens for incoming requests and sends back responses. It runs continuously, waiting.

When your browser visits `http://localhost:3000`, it sends a request to port 3000 on your computer. The Study-Hub server (which is listening on port 3000) receives that request, decides what to send back, and sends a response.

Port — a number that identifies a specific program on a computer. Think of a building (the computer) with many doors (ports). Each program listens on its own door. Port 3000 is common for development servers. Port 80 is HTTP, 443 is HTTPS.

7.2 What Is Express?

Express is a framework — a pre-built set of tools — that makes writing Node.js servers much easier.

Without Express, you'd write low-level code like:

```
const http = require('http');
const server = http.createServer((req, res) => {
  if (req.url === '/api/notes' && req.method === 'GET') {
    // parse query string, set headers, handle errors...
    res.writeHead(200, { 'Content-Type': 'application/json' });
    res.end(JSON.stringify({ notes: [] }));
  }
  // ... hundreds more if/else blocks
});
server.listen(3000);
```

With Express:

```
const express = require('express');
const app = express();

app.get('/api/notes', (req, res) => {
  res.json({ notes: [] });
});

app.listen(3000);
```

Express handles routing, parsing, error handling, and much more — so you write business logic instead of infrastructure code.

7.3 server.js — The Entry Point, Line by Line

This is the first file that runs when you start the app. Let's go through it completely.

Loading dependencies

```
require('dotenv').config();
```

This is the very first line. It reads your `.env` file and populates `process.env` with all your environment variables. **Must be first** — everything else needs those variables.

```
const express = require('express');
const path = require('path');
const cors = require('cors');
const helmet = require('helmet');
const morgan = require('morgan');
const rateLimit = require('express-rate-limit');
```

`require()` is Node.js's way of importing modules. These are all packages from `node_modules`.

- `express` — the web framework
- `path` — a Node.js built-in module for working with file paths
- `cors` — handles Cross-Origin Resource Sharing
- `helmet` — adds security headers
- `morgan` — logs every request
- `express-rate-limit` — limits request frequency

```
const { testConnection } = require('./config/database');
const routes = require('./routes/index');
const { errorHandler, notFound } = require('./middleware/errorHandler');
```

These are our own files. `require('./config/database')` imports from `backend/config/database.js`. The `./` means "in the current folder" — relative path.

`{ testConnection }` uses destructuring — we only import the specific function we need from that module.

Creating the app

```
const app = express();
const PORT = process.env.PORT || 3000;
```

`express()` creates the Express application object. Everything in the server is built on this `app` object.

`PORT` reads from the environment. On your machine, `process.env.PORT` is probably undefined, so it falls back to `3000`. On Render.com, they set `PORT` to whatever port their infrastructure uses.

Security headers with Helmet

```
app.use(helmet({
  contentSecurityPolicy: {
    directives: {
      defaultSrc: ["'self'"],
      scriptSrc: ["'self'", "'unsafe-inline'", "https://cdnjs.cloudflare.com"],
      styleSrc: ["'self'", "'unsafe-inline'", "https://fonts.googleapis.com"],
      fontSrc: ["'self'", "https://fonts.gstatic.com"],
      imgSrc: ["'self'", "data:", "blob:"],
      connectSrc: ["'self'"],
      frameSrc: ["'self'", "blob:"],
      workerSrc: ["'self'", "blob:"],
      objectSrc: ["'self'", "blob:"],
    }
  }
}));
```

`app.use()` registers middleware — code that runs for every request before it reaches a route handler.

Helmet adds security headers to every HTTP response. The most important one it sets is the **Content Security Policy (CSP)**.

CSP tells the browser: "Only load resources from these approved places." This prevents XSS attacks (Cross-Site Scripting) — where an attacker injects malicious JavaScript into your page.

Breaking down the CSP directives: - `defaultSrc: ["'self'"]` — by default, only allow resources from our own server - `scriptSrc: ["'self'", "'unsafe-inline'", "https://cdnjs.cloudflare.com"]` — allow scripts from ourselves, inline scripts, and cdnjs (a CDN we use for libraries) - `styleSrc: ["'self'", "'unsafe-inline'", "https://fonts.googleapis.com"]` — allow CSS from ourselves, inline styles, and Google Fonts - `fontSrc: ["'self'", "https://fonts.gstatic.com"]` — Google Fonts are served from gstatic.com - `imgSrc: ["'self'", "data:", "blob:"]` — allow images from ourselves, base64 data URLs, and blob URLs (for the PDF viewer) - `frameSrc: ["'self'", "blob:"]` — allow iframes from ourselves and blobs (PDF viewer uses an iframe)

CORS

```
const allowedOrigins = process.env.ALLOWED_ORIGINS
  ? process.env.ALLOWED_ORIGINS.split(',').map(o => o.trim())
```

```
  : ['http://localhost:3000'];

app.use(cors({
  origin: process.env.NODE_ENV === 'development' ? true : allowedOrigins,
  credentials: true,
  methods: ['GET', 'POST', 'PUT', 'PATCH', 'DELETE', 'OPTIONS'],
  allowedHeaders: ['Content-Type', 'Authorization']
}));
```

CORS (Cross-Origin Resource Sharing) is a browser security feature. By default, browsers prevent JavaScript on one website from making requests to another website. This is called the **Same-Origin Policy**.

For example, JavaScript running on `https://evil-site.com` can't make requests to `https://your-bank.com`. This is good security.

But it also means that if you hosted your frontend at `frontend.com` and your backend at `api.com`, the browser would block the requests. CORS is the mechanism for servers to say "actually, it's OK for `frontend.com` to talk to me."

In development, we use `origin: true` (allow everything) because it's local. In production, we restrict to specific allowed origins.

- `credentials: true` — allow cookies and Authorization headers
- `methods` — which HTTP methods to allow
- `allowedHeaders` — which request headers to allow

Rate limiting

```
const generalLimiter = rateLimit({
  windowMs: 60 * 1000,    // 1 minute window
  max: 300,               // max 300 requests per minute
  message: { success: false, error: 'Too many requests, please try again later' }
});

const authLimiter = rateLimit({
  windowMs: 15 * 60 * 1000,  // 15 minute window
  max: 10,                    // max 10 requests per 15 minutes
  message: { success: false, error: 'Too many login attempts, please try again later' }
});

app.use('/api/auth/login', authLimiter);
app.use('/api/auth/register', authLimiter);

app.use('/api/', (req, res, next) => {
  if (req.path.startsWith('/auth/')) return next();
  return generalLimiter(req, res, next);
});
```

Rate limiting prevents abuse. Without it: - A bot could try millions of password combinations (brute force attack) - Someone could spam your API with thousands of requests per second (DoS attack)

Two limiters: 1. **Auth limiter** — very strict: only 10 login/register attempts per 15 minutes. This makes brute-force attacks impractical (10 guesses / 15 min = 1,000 guesses per day — for a modern password, that's nothing). 2. **General limiter** — lenient: 300 requests per minute for normal use

`app.use('/api/auth/login', authLimiter)` means "apply `authLimiter` to all requests to `/api/auth/login`."

Body parsing

```
app.use(express.json({ limit: '10mb' }));
app.use(express.urlencoded({ extended: true, limit: '10mb' }));
```

These tell Express how to parse incoming request bodies: - `express.json()` — parse `Content-Type: application/json` bodies. Without this, `req.body` would be undefined. - `express.urlencoded()` — parse HTML form data (rarely needed in a JSON API but good to have) - `limit: '10mb'` — maximum body size (prevents memory exhaustion attacks)

Request logging

```
if (process.env.NODE_ENV !== 'test') {
  app.use(morgan(process.env.NODE_ENV === 'development' ? 'dev' : 'combined'));
}
```

Morgan logs every HTTP request to the console. In development mode (`'dev'`), it's colourful and concise. In production (`'combined'`), it's the standard Apache log format with full details.

This is invaluable during development. Every request you make appears in the terminal, showing you the method, URL, status code, and response time.

Serving the frontend

```
app.use(express.static(path.join(__dirname, '../frontend'), {
  maxAge: process.env.NODE_ENV === 'production' ? '1d' : 0
}));
```

This tells Express to serve all files in the `frontend/` folder as static files. When a browser requests `/css/variables.css`, Express finds `frontend/css/variables.css` and sends it.

`__dirname` is a Node.js variable containing the directory of the current file. `path.join(__dirname, '../frontend')` goes up one folder from `backend/` and into `frontend/`.

`maxAge: '1d'` in production means browsers can cache files for 1 day without re-downloading them. In development, `0` means no caching (so changes are always visible immediately).

API routes

```
app.use('/api', routes);
```

All requests starting with `/api` are handed to the `routes` module. The routes module then further directs to specific controllers.

SPA fallback

```
app.get('*', (req, res) => {
  if (req.path.startsWith('/api/')) {
    return res.status(404).json({ success: false, error: 'API route not found' });
  }
  res.sendFile(path.join(__dirname, '../frontend/index.html'));
});
```

This is the **Single-Page Application fallback**. The * means "catch everything that hasn't been handled yet."

If the user visits `http://localhost:3000/notes` directly (or refreshes the page while on the notes view), the browser sends a GET request for `/notes`. Since there's no route for `/notes` on the server, Express would normally return a 404. But we don't want that — the frontend should handle routing.

So instead, for any unknown path that's not an API route, we send back `index.html`. The browser loads the SPA, which reads the URL and shows the right view.

Error handling

```
app.use(notFound);
app.use(errorHandler);
```

These two middleware functions are the last ones registered. Any error that wasn't caught by a route handler will reach `errorHandler`. The `notFound` handler catches any request that reached this point without a response.

Starting the server

```
const start = async () => {
  console.log('\n Starting Study-Hub...\n');

  const dbOk = await testConnection();
  if (!dbOk) {
    console.error('\nERROR: Cannot start without database. Check your .env configuration.\n');
    process.exit(1);
  }

  try {
    const fs = require('fs');
    const { pool } = require('./config/database');
    const schema = fs.readFileSync(path.join(__dirname, './config/schema.sql'), 'utf-8');
    await pool.query(schema);
    console.log(' Schema applied successfully');
  } catch (err) {
    console.error(' Schema apply failed:', err.message);
    process.exit(1);
  }

  app.listen(PORT, () => {
    console.log(`\nSUCCESS: Study-Hub running at http://localhost:${PORT}`);
  });
};

start().catch(err => {
  console.error('Fatal startup error:', err);
  process.exit(1);
});
```

The `start` function: 1. Tests the database connection — if it fails, stop. There's no point running without a database. 2. Reads and applies the SQL schema — the `CREATE TABLE IF NOT EXISTS` statements. Because of `IF NOT EXISTS`, this is safe to re-run — it won't overwrite existing tables. 3. Starts the HTTP server on the specified PORT

`process.exit(1)` stops the Node.js process with exit code 1 (convention: 0 = success, non-zero = error).

7.4 Middleware — The Request Pipeline

The most important concept in Express is **middleware**. It's a function with this signature:

```
(req, res, next) => {
  // do something
  next(); // pass to the next middleware/route
}
```

- `req` — the incoming request (URL, headers, body, etc.)
- `res` — the outgoing response (you call `res.json()`, `res.send()`, etc. to respond)
- `next` — a function to call when this middleware is done. If you don't call `next()` (and don't respond), the request gets stuck.

Middleware runs in order. Study-Hub's middleware pipeline for a protected API request looks like:

```
Request: GET /api/notes
      ↓
Helmet (adds security headers)
      ↓
CORS (sets CORS headers)
      ↓
Rate limiter (checks if too many requests)
      ↓
JSON body parser (parses request body)
      ↓
Morgan (logs the request)
      ↓
Auth middleware (verifies JWT token)
      ↓
Route handler (notes controller gets data, sends response)
      ↓
Response sent
```

If any middleware calls `res.json()` or `res.send()` without calling `next()`, the chain stops there — the request is handled.

7.5 How Express Routes Work

```
// From routes/index.js
router.get('/notes', authenticate, notesController.getNotes);
```

This says: for GET requests to `/notes`: 1. First run `authenticate` middleware 2. If that calls `next()` (user is authenticated), run `notesController.getNotes`

The full URL is `/api/notes` because the router is mounted with `app.use('/api', routes)`.

Multiple middleware can be listed before the final handler:

```
router.post('/pdfs/upload', authenticate, upload.single('pdf'), pdfController.uploadPdf);
```

Three steps: authenticate → handle file upload → run controller.

7.6 Request and Response Objects

Every route handler receives `req` and `res`:

The `req` object (incoming request)

```
req.body          // parsed request body (JSON or form data)
req.params        // URL parameters: /notes/:id → req.params.id
req.query         // Query string: /notes?search=foo → req.query.search
req.headers       // HTTP headers
req.user          // Added by auth middleware - the logged-in user
req.file          // Added by Multer - the uploaded file
```

The `res` object (outgoing response)

```
res.json({ success: true, data: [] })   // send JSON
res.status(201).json({ ... })           // set status code then send
res.status(404).json({ error: 'Not found' })
res.sendFile('/path/to/file.pdf')       // send a file
```

7.7 Error Handling Middleware

```
// From middleware/errorHandler.js (conceptual)
const errorHandler = (err, req, res, next) => {
  console.error(err.stack);

  const status = err.status || err.statusCode || 500;
  const message = err.message || 'Internal Server Error';

  res.status(status).json({
    success: false,
    error: message
  });
};
```

Error handling middleware has **four** parameters (`err, req, res, next`). Express knows it's an error handler because of the four-parameter signature.

When a controller calls `next(err)`, Express skips all regular middleware and jumps to the error handler.

Now you understand the server. In the next section, we go deep into the database.

Section 08 — Database & PostgreSQL

What a database is, how SQL works, and Study-Hub's complete data design

---

8.1 What Is a Database?

A database is a program specifically designed to **store, organize, and retrieve data reliably and efficiently**.

Think of it as a collection of spreadsheets (called **tables**), where: - Each table stores one type of thing (users, notes, PDFs) - Each row is one record (one user, one note, one PDF) - Each column is one piece of information about that thing (name, email, created date)

But unlike spreadsheets, a database can: - Handle millions of rows and stay fast - Run complex searches instantly - Handle thousands of users simultaneously - Never lose data, even if the power goes out mid-write - Link tables to each other (a note belongs to a user)

8.2 SQL — The Language of Databases

SQL (Structured Query Language) is the language you use to talk to a database. It's been around since the 1970s and is used by virtually every serious application.

SQL statements:

```sql
-- Create a table
CREATE TABLE notes (
  id UUID PRIMARY KEY,
  title VARCHAR(500),
  content TEXT
);

-- Insert a row
INSERT INTO notes (id, title, content) VALUES ('abc-123', 'My Note', 'Content here');

-- Read rows
SELECT * FROM notes WHERE title LIKE '%study%';

-- Update a row
UPDATE notes SET title = 'Updated Title' WHERE id = 'abc-123';

-- Delete a row
DELETE FROM notes WHERE id = 'abc-123';
```

-- is a SQL comment (ignored by the database).

8.3 database.js — The Connection Module

```node
const { Pool } = require('pg');
```

`pg` is the PostgreSQL client for Node.js. `Pool` is a **connection pool** — instead of creating a new database connection for every request (slow), it maintains a pool of reusable connections (fast).

```node
const pool = new Pool(
  process.env.DATABASE_URL
    ? {
        connectionString: process.env.DATABASE_URL,
        ssl: isProduction ? { rejectUnauthorized: false } : false,
```

```
        max: 20,
        idleTimeoutMillis: 30000,
        connectionTimeoutMillis: 5000,
      }
    : {
        host: process.env.DB_HOST || 'localhost',
        port: parseInt(process.env.DB_PORT) || 5432,
        database: process.env.DB_NAME || 'studyhub',
        user: process.env.DB_USER || 'postgres',
        password: process.env.DB_PASSWORD,
        max: 20,
        idleTimeoutMillis: 30000,
        connectionTimeoutMillis: 2000,
      }
);
```

Two connection modes: 1. `DATABASE_URL` — a single URL like `postgres://user:pass@host/dbname`. Render.com provides this automatically. 2. **Individual settings** — host, port, user, password separately. Used for local development.

Pool settings: - `max: 20` — keep up to 20 connections open simultaneously - `idleTimeoutMillis: 30000` — close connections that have been idle for 30 seconds - `connectionTimeoutMillis: 2000` — give up trying to connect after 2 seconds

```
const query = async (text, params) => {
  const start = Date.now();
  try {
    const res = await pool.query(text, params);
    const duration = Date.now() - start;
    if (process.env.NODE_ENV === 'development') {
      console.log('Query executed', { text: text.slice(0, 80), duration, rows: res.rowCount });
    }
    return res;
  } catch (err) {
    console.error('Database query error:', err.message);
    throw err;
  }
};
```

This is the `query` helper used everywhere. It: 1. Records the start time 2. Runs the SQL query with parameters 3. Logs the query in development (helpful for debugging) 4. Throws any errors (the calling code will catch them)

Parameterized queries — the `params` argument is critical for security:

```
// GOOD - parameterized (safe)
await query('SELECT * FROM users WHERE email = $1', [email]);

// BAD - string interpolation (vulnerable to SQL injection)
await query(`SELECT * FROM users WHERE email = '${email}'`);  // NEVER DO THIS
```

SQL Injection is an attack where a malicious user enters SQL code as their input:

```
email input: admin@site.com' OR '1'='1
Injected SQL: SELECT * FROM users WHERE email = 'admin@site.com' OR '1'='1'
Result: returns ALL users (because '1'='1' is always true)
```

With parameterized queries, the `$1` is treated as pure data — the database never interprets it

as SQL code.

8.4 The Schema — Study-Hub's Data Design

`schema.sql` defines all the tables. Let's go through each one.

The UUID Extension

```sql
CREATE EXTENSION IF NOT EXISTS "uuid-ossp";
```

This enables the UUID generation function in PostgreSQL. UUID (Universally Unique Identifier) looks like `ac2c0621-89c1-4925-9c4c-40d5ace82e9a` — a random, practically-impossible-to-guess, globally unique ID.

Why UUIDs instead of sequential numbers (1, 2, 3...)? - Sequential IDs reveal information: user #1003 tells attackers there are ~1000 users - Sequential IDs are guessable: if you can access record 42, try 43 - UUIDs work across distributed systems without conflicts

The Users Table

```sql
CREATE TABLE IF NOT EXISTS users (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  email VARCHAR(255) UNIQUE NOT NULL,
  password_hash VARCHAR(255) NOT NULL,
  name VARCHAR(100) NOT NULL,
  avatar_color VARCHAR(7) DEFAULT '#6c63ff',
  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
  updated_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);
```

Column by column:

`id UUID PRIMARY KEY DEFAULT uuid_generate_v4()` - `UUID` — the data type (a 128-bit unique identifier) - `PRIMARY KEY` — this column uniquely identifies each row; automatically indexed - `DEFAULT uuid_generate_v4()` — if no ID is provided on insert, generate one automatically

`email VARCHAR(255) UNIQUE NOT NULL` - `VARCHAR(255)` — a string up to 255 characters - `UNIQUE` — no two rows can have the same email - `NOT NULL` — this column must have a value; can't be empty

`password_hash VARCHAR(255) NOT NULL` - Stores the *hashed* version of the password, never the real one - A bcrypt hash always looks like `$2a$12$...` — about 60 characters

`name VARCHAR(100) NOT NULL` - User's display name

`avatar_color VARCHAR(7) DEFAULT '#6c63ff'` - A hex color code (always 7 characters: `#` + 6 hex digits) - Used to color the user's avatar circle - Has a default value

`created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()` - `TIMESTAMP WITH TIME ZONE` — stores date and time including timezone (always stored as UTC internally) - `DEFAULT NOW()` — automatically set to the current time when the row is inserted

`updated_at` — same as `created_at` but updated by a trigger whenever the row changes

The Note Folders Table

```sql
CREATE TABLE IF NOT EXISTS note_folders (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  name VARCHAR(100) NOT NULL,
  icon VARCHAR(10) DEFAULT ' ',
  color VARCHAR(7) DEFAULT '#6c63ff',
  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
  updated_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);
```

`user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE`

This is a **Foreign Key** — the most important concept in relational databases.

`REFERENCES users(id)` means: "the value in `user_id` must exist as an `id` in the `users` table." This is called **referential integrity** — you can't create a folder for a user that doesn't exist.

`ON DELETE CASCADE` means: "if the referenced user is deleted, automatically delete this folder too." If user Alice is deleted, all her folders are automatically deleted as well. Without this, you'd have orphaned folders pointing to a non-existent user.

`icon VARCHAR(10) DEFAULT ' '`

Emoji are stored as Unicode characters in the database. `VARCHAR(10)` allows for 10 characters (emoji can be multiple bytes, but PostgreSQL's VARCHAR counts characters, not bytes).

The Notes Table

```sql
CREATE TABLE IF NOT EXISTS notes (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  folder_id UUID REFERENCES note_folders(id) ON DELETE SET NULL,
  title VARCHAR(500) DEFAULT '',
  content TEXT DEFAULT '',
  color VARCHAR(7) DEFAULT '#1a1a1a',
  is_pinned BOOLEAN DEFAULT FALSE,
  is_archived BOOLEAN DEFAULT FALSE,
  tags TEXT[] DEFAULT '{}',
  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
  updated_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);
```

`folder_id UUID REFERENCES note_folders(id) ON DELETE SET NULL`

Notice `ON DELETE SET NULL` here (not CASCADE). If a folder is deleted, notes in that folder have their `folder_id` set to NULL (they become "uncategorized") rather than being deleted. This is the right behaviour — deleting a folder shouldn't delete all the notes inside it.

`content TEXT DEFAULT ''`

`TEXT` is unlimited-length text. Unlike `VARCHAR(n)`, there's no maximum length. Use `TEXT` for anything that could be long.

`is_pinned BOOLEAN DEFAULT FALSE`

`BOOLEAN` is true or false. `DEFAULT FALSE` means new notes start unpinned.

`tags TEXT[] DEFAULT '{}'`

`TEXT[]` is a **PostgreSQL array** of text values. `'{}'` is the syntax for an empty array in SQL. This lets us store multiple tags without a separate tags table.

Full-Text Search Indexes

```
CREATE INDEX IF NOT EXISTS notes_fts_idx ON notes
  USING gin(to_tsvector('english', COALESCE(title, '') || ' ' || COALESCE(content, '')));
```

This creates a **full-text search index** — without it, searching through notes would require scanning every row, which is slow.

Breaking it down: - `CREATE INDEX` — creates a database index (a pre-computed structure that speeds up lookups) - `USING gin` — GIN (Generalized Inverted Index) is the index type designed for text search - `to_tsvector('english', ...)` — converts text to a "text search vector" — a processed representation of words, with language-specific stemming (so "running" matches "run") - `COALESCE(title, '') || ' ' || COALESCE(content, '')` — combines title and content into one searchable string. `COALESCE` returns the first non-null value — this prevents errors if title or content is NULL.

When you search notes, the query uses this index:

```
WHERE to_tsvector('english', COALESCE(title,'') || ' ' || COALESCE(content,''))
      @@ plainto_tsquery('english', $1)
```

`@@` is the full-text match operator. `plainto_tsquery` converts a search term like "machine learning" into a query that matches "machine" AND "learning".

Other Indexes

```
CREATE INDEX IF NOT EXISTS notes_user_idx ON notes(user_id);
CREATE INDEX IF NOT EXISTS notes_folder_idx ON notes(folder_id);
CREATE INDEX IF NOT EXISTS notes_pinned_idx ON notes(user_id, is_pinned);
```

Why these?

Every query for notes includes `WHERE user_id = $1`. Without an index on `user_id`, the database scans every note in the table to find ones belonging to this user. With the index, it goes directly to the right records.

A database index is like the index at the back of a book — instead of reading every page to find "PostgreSQL," you look it up in the index and go directly to the right page.

The PDFs and PDF Folders Tables

Similar structure to notes. Key differences:

```
CREATE TABLE IF NOT EXISTS pdfs (
  filename VARCHAR(255) NOT NULL,         -- the UUID filename on disk
  original_name VARCHAR(255) NOT NULL,    -- the original filename (e.g., "lecture-3.pdf")
  file_size BIGINT NOT NULL,              -- size in bytes
  page_count INTEGER DEFAULT 0,          -- number of pages
  extracted_text TEXT DEFAULT '',         -- the full text extracted from the PDF
  file_path VARCHAR(500) NOT NULL,        -- where the file is on disk
  ...
);
```

`BIGINT` is for large integers (up to 9 quintillion) — `file_size` in bytes can exceed 2 billion for large files (the limit of regular `INTEGER`).

`extracted_text` stores the entire text content of the PDF. This is what the AI reads when you ask it about a PDF.

The Chat Tables

```
CREATE TABLE IF NOT EXISTS chat_sessions (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  title VARCHAR(300) DEFAULT 'New Chat',
  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
  updated_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

CREATE TABLE IF NOT EXISTS chat_messages (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  session_id UUID NOT NULL REFERENCES chat_sessions(id) ON DELETE CASCADE,
  user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  role VARCHAR(20) NOT NULL CHECK (role IN ('user', 'assistant', 'system')),
  content TEXT NOT NULL,
  context_type VARCHAR(50),
  context_ref_id UUID,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);
```

Chat sessions group related messages together (like a conversation thread). A user can have many sessions.

Chat messages belong to a session. The `role` column is constrained:

```
CHECK (role IN ('user', 'assistant', 'system'))
```

`CHECK` is a **constraint** — the database itself enforces that only these values are allowed. If you try to insert `role = 'robot'`, the database throws an error. This is data integrity enforced at the database level, not just in application code.

`context_type` and `context_ref_id` track what material was attached to a message (which note or PDF the user was asking about).

The Auto-Update Trigger

```
CREATE OR REPLACE FUNCTION update_updated_at_column()
RETURNS TRIGGER AS $$
BEGIN
  NEW.updated_at = NOW();
  RETURN NEW;
END;
$$ language 'plpgsql';
```

This creates a **database function** written in PL/pgSQL (PostgreSQL's procedural language). It's called a **trigger function** — it runs automatically when something happens.

`NEW` refers to the row being updated. `NEW.updated_at = NOW()` sets the updated_at column to the current time.

Then we attach this function to each table:

```
DO $$
DECLARE
```

```
  tbl TEXT;
BEGIN
  FOREACH tbl IN ARRAY ARRAY['users','note_folders','notes','pdf_folders','pdfs','chat_sessions'] LOOP
    EXECUTE format('
      DROP TRIGGER IF EXISTS update_%I_updated_at ON %I;
      CREATE TRIGGER update_%I_updated_at
        BEFORE UPDATE ON %I
        FOR EACH ROW EXECUTE FUNCTION update_updated_at_column();
    ', tbl, tbl, tbl, tbl);
  END LOOP;
END;
$$;
```

This is a `DO` block — anonymous PL/pgSQL code that runs once. It loops through each table name and creates a trigger:

- `BEFORE UPDATE ON table` — fire before any UPDATE statement
- `FOR EACH ROW` — fire for every row being updated (not just once per statement)

The result: whenever you update a note, the `updated_at` column is automatically set to NOW(). You never have to remember to update it in your code.

8.5 Relationships Between Tables

Here's a visual of how the tables relate:

```
users

    note_folders (user has many folders)

          notes (folder has many notes)

    notes (user has many notes directly)

    pdf_folders

          pdfs

    pdfs

    chat_sessions (user has many chat sessions)

           chat_messages (session has many messages)
```

When you query notes for a user, you JOIN with the folders table to also get the folder name:

```
SELECT n.*, f.name AS folder_name, f.color AS folder_color
FROM notes n
LEFT JOIN note_folders f ON f.id = n.folder_id
WHERE n.user_id = $1
```

`LEFT JOIN` includes all notes even if they don't have a folder (folder_id is NULL). A regular `JOIN` (or `INNER JOIN`) would exclude notes without a folder.

8.6 Key Takeaways

- A database stores data permanently in structured tables
- SQL is the language for reading and writing data
- Parameterized queries (`$1`, `$2`) prevent SQL injection
- Foreign keys (`REFERENCES`) enforce relationships between tables
- `ON DELETE CASCADE` auto-deletes children when the parent is deleted
- `ON DELETE SET NULL` nullifies references when the parent is deleted
- Indexes dramatically speed up queries on frequently-searched columns
- Full-text search indexes enable efficient keyword search
- Triggers automatically maintain `updated_at` timestamps
- `CHECK` constraints enforce valid values at the database level

Next: how Study-Hub handles login and keeps users authenticated.

Section 09 — Authentication

Passwords, hashing, JWT tokens — how login really works

9.1 The Problem: Proving Identity

Authentication answers one question: **"Is this person really who they claim to be?"**

On the web, this is harder than it sounds. HTTP is **stateless** — each request is completely independent. The server doesn't remember who sent the last request. So for every protected request, we need a way to prove identity without making the user log in again.

The solution Study-Hub uses: **JWT tokens**. But first, let's understand how passwords are handled.

9.2 Password Hashing with bcrypt

The golden rule: Never store passwords in plain text. Ever.

If your database is ever stolen (it happens to big companies regularly), plain text passwords expose every user's account. Since people reuse passwords, this means their email, bank, and social media accounts are also compromised.

Instead, we store a **hash** of the password.

What is a hash?

A hash function takes any input and produces a fixed-length output (called the hash or digest) with these properties: - **Deterministic**: same input always produces same output - **One-way**: you can't reverse it to get the input back - **Avalanche effect**: a tiny change in input produces a completely different hash

`"password123"` → hash → `$2a$12$LQv3c1yqBWVHxkd0LHAkCOYz6TtxMQJqhN8/lewKyNiLXCJzjklJG`

You cannot reverse that hash to get `"password123"` back. It's mathematically infeasible.

Why bcrypt specifically?

bcrypt is designed to be slow. This is intentional security.

Regular hash functions (like SHA-256) are extremely fast — billions per second on modern hardware. This makes brute-force attacks feasible: an attacker who steals your database can try billions of password guesses per second.

bcrypt has a **work factor** (cost parameter). In Study-Hub: `bcrypt.hash(password, 12)`. The `12` means `2^12 = 4096` iterations. This makes each hash take about 250ms — slow enough to make brute-force attacks impractical (4000 guesses per second instead of billions), but fast enough that legitimate logins feel instant.

Salting — preventing rainbow table attacks

A **rainbow table** is a pre-computed lookup table: "here's what `"password123"` hashes to for every hash function." An attacker can look up your hash and instantly find the password.

bcrypt automatically adds a random **salt** — a random value mixed into the password before hashing:

```
salt = random("x8Pk...")
hash("password123" + salt) → "$2a$12$x8Pk...hashed_output"
```

The salt is stored in the hash itself (the `$2a$12$x8Pk...` prefix). When verifying, bcrypt extracts the salt and rehashes the input for comparison.

Because each user's salt is different, even if two users have the same password, their hashes are completely different. Rainbow tables are useless.

The code

```
// In authController.js - Registration
const passwordHash = await bcrypt.hash(password, 12);
// 12 = work factor. Takes ~250ms. Store this hash, not the password.

await query(
  'INSERT INTO users (name, email, password_hash, ...) VALUES ($1, $2, $3, ...)',
  [name, email, passwordHash, ...]
);

// In authController.js - Login
const result = await query(
  'SELECT id, name, email, password_hash FROM users WHERE email = $1',
  [email.toLowerCase()]
);

const user = result.rows[0];
const isValid = await bcrypt.compare(password, user.password_hash);
// bcrypt re-hashes the input and compares - returns true/false
// This also takes ~250ms (same work factor)

if (!isValid) {
  return res.status(401).json({ error: 'Invalid email or password' });
}
```

Note that we return the same error for “email not found” and “wrong password.” If we returned different messages, an attacker could enumerate valid emails (“ah, that email exists but wrong password”).

9.3 JWT — Staying Logged In

Once the user proves their identity (correct password), we need to give them something to prove it for future requests. That “something” is a **JWT** (JSON Web Token).

What is a JWT?

A JWT is a string that looks like this:

`eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VySWQiOiJhYzJjMDYyMS04OWMxLTQ5MjUtOWM0Yy00MGQ1YWNlODJlOWEiLC`

It’s three base64-encoded sections separated by dots:

1. **Header** — algorithm used (`HS256`) and token type (`JWT`)
2. **Payload** — the actual data (userId, issue time, expiry time)
3. **Signature** — a cryptographic signature proving the token is genuine

Decoded payload:

```
{
  "userId": "ac2c0621-89c1-4925-9c4c-40d5ace82e9a",
  "iat": 1700000000,    // issued at (Unix timestamp)
  "exp": 1700604800     // expires at (7 days later)
}
```

Why can we trust the token?

The signature is created using a **secret key** that only the server knows:

```
signature = HMAC_SHA256(header + "." + payload, JWT_SECRET)
```

When the server receives a token, it re-computes the signature using its secret and compares. If they match, the token is genuine — it was created by the server, not forged by an attacker.

An attacker could decode the payload (it’s just base64 — not encrypted) and see `"userId": "abc"`. But if they try to change it to their own userId and send it, the signature won’t match the new payload, and the server rejects it.

JWT is not encrypted — the payload is readable. Never put sensitive data (passwords, credit card numbers) in a JWT payload. Only put IDs and non-sensitive identifiers.

Creating a token

```
// From middleware/auth.js
const jwt = require('jsonwebtoken');

const generateToken = (userId) => {
  return jwt.sign(
    { userId },                              // payload
    process.env.JWT_SECRET,                  // secret key (from .env)
    { expiresIn: process.env.JWT_EXPIRES_IN || '7d' }  // expire after 7 days
```

```
  );
};
```

`jwt.sign()` creates the token. The server sends this token to the client after successful login.

Verifying a token

```
// From middleware/auth.js - the authenticate middleware
const authenticate = async (req, res, next) => {
  try {
    let token = null;

    // Accept token from Authorization header
    const authHeader = req.headers.authorization;
    if (authHeader && authHeader.startsWith('Bearer ')) {
      token = authHeader.split(' ')[1];
      // "Bearer eyJhbGc..." → splits on space → takes [1] = the token part
    } else if (req.query.token) {
      // Also accept from URL query parameter (for PDF stream/download URLs)
      token = req.query.token;
    }

    if (!token) {
      return res.status(401).json({
        success: false,
        error: 'Authentication required. Please provide a valid Bearer token.'
      });
    }

    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    // jwt.verify throws if:
    // - signature doesn't match (token was tampered with)
    // - token has expired
    // Returns the decoded payload: { userId: 'abc-123', iat: ..., exp: ... }

    // Verify user still exists in DB (account might have been deleted)
    const result = await query(
      'SELECT id, email, name, avatar_color FROM users WHERE id = $1',
      [decoded.userId]
    );

    if (result.rows.length === 0) {
      return res.status(401).json({ success: false, error: 'User no longer exists' });
    }

    req.user = result.rows[0];
    // Attach user to request so controllers can use it
    next();
  } catch (err) {
    if (err.name === 'TokenExpiredError') {
      return res.status(401).json({ success: false, error: 'Token expired. Please log in again.' });
    }
    if (err.name === 'JsonWebTokenError') {
      return res.status(401).json({ success: false, error: 'Invalid token' });
```

```
    }
    res.status(500).json({ success: false, error: 'Internal server error' });
  }
};
```

After `authenticate` runs and calls `next()`, every subsequent handler has access to `req.user` — the full user object from the database. Controllers can use `req.user.id` to scope queries to the logged-in user.

Why do we look up the user in the database every request?

The JWT contains `userId`. We could just trust that and use it directly. Why go to the database?

Because users can be deleted or have their accounts suspended. The JWT might still be valid (not expired), but the user no longer exists. Checking the database ensures we're only serving data to users who currently exist.

9.4 The Full Auth Flow

Registration:

```
1. User fills form → POST /api/auth/register { name, email, password }
2. Server validates inputs (all required, email format, password length)
3. Server checks if email already registered
4. Server hashes the password with bcrypt (work factor 12)
5. Server inserts new user row into database
6. Server generates JWT token for this new user
7. Server creates default folders (Personal, Study, Work) for the user
8. Server responds: { success: true, token: "eyJ...", user: { id, name, email } }
9. Frontend stores token in localStorage
10. Frontend shows the main app
```

Login:

```
1. User fills form → POST /api/auth/login { email, password }
2. Server queries database for user with that email
3. If not found → return 401 "Invalid email or password" (don't reveal which is wrong)
4. Server uses bcrypt.compare(password, stored_hash) → true/false
5. If false → return 401 "Invalid email or password"
6. Server generates JWT token
7. Server responds: { success: true, token: "eyJ...", user: { ... } }
8. Frontend stores token, shows app
```

Subsequent requests:

```
1. Frontend adds header: Authorization: Bearer eyJ...
2. authenticate middleware reads token from header
3. jwt.verify() checks signature and expiry
4. Database lookup confirms user exists
5. req.user = user object
6. next() → controller runs
7. Controller uses req.user.id for all database queries
```

Logout:

```
1. Frontend deletes token from localStorage
2. No server call needed - the JWT is just forgotten
3. If user visits a protected page, they have no token → redirected to login
```

Note: JWTs can't be truly "invalidated" before they expire (unlike sessions stored server-side). Study-Hub uses 7-day expiry — the token works for 7 days regardless. For most apps this is acceptable. For high-security needs, you'd maintain a server-side revocation list.

9.5 How the Frontend Handles Auth

After login/register success:

```
// From frontend/js/modules/auth.js
Storage.setToken(data.token);   // save to localStorage
Storage.setUser(data.user);     // save user info
App.init();                     // show the main app
```

On every API request:

```
// From frontend/js/utils/api.js
const token = Storage.getToken();

const headers = {
  'Content-Type': 'application/json',
  ...(token ? { 'Authorization': `Bearer ${token}` } : {}),
};
```

If a token exists, attach it as the `Authorization` header.

When the token expires:

```
// From api.js
if (res.status === 401) {
  Storage.removeToken();
  Storage.removeUser();
  window.dispatchEvent(new CustomEvent('auth:expired'));
  throw new Error('Session expired. Please sign in again.');
}
```

A custom event `'auth:expired'` is dispatched. App.js listens for it:

```
// From app.js
window.addEventListener('auth:expired', () => {
  Storage.clear();
  showAuthScreen();
});
```

The user is automatically logged out and shown the login screen.

9.6 Input Validation

Study-Hub validates user input before doing anything else:

```
// From authController.js
const { name, email, password } = req.body;

if (!name || !email || !password) {
  return res.status(400).json({ success: false, error: 'Name, email and password are required' });
}

if (password.length < 8) {
  return res.status(400).json({ success: false, error: 'Password must be at least 8 characters' });
}

const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
if (!emailRegex.test(email)) {
  return res.status(400).json({ success: false, error: 'Invalid email address' });
}
```

`/^[^\s@]+@[^\s@]+\.[^\s@]+$/` is a **regular expression** for basic email validation: - `^` — start of string - `[^\s@]+` — one or more characters that aren't spaces or `@` - `@` — the literal `@` - `[^\s@]+` — more non-space non-`@` characters (domain) - `\.` — a literal dot (`.` alone means "any character" in regex, `\.` means literal) - `[^\s@]+` — the TLD (`.com`, `.in`, etc.) - `$` — end of string

Status code `400 Bad Request` means "you sent something wrong." Status code `401 Unauthorized` means "you're not logged in." Status code `409 Conflict` means "this already exists" (used for duplicate email).

9.7 Key Takeaways

- Never store plain text passwords — always use bcrypt with a work factor of 10+
- `bcrypt.hash()` creates the hash; `bcrypt.compare()` verifies input against hash
- JWT tokens prove identity for subsequent requests without re-entering passwords
- JWTs are signed but not encrypted — don't store secrets in them
- The `authenticate` middleware runs before any protected route
- `req.user` is populated by `authenticate` so controllers know who's making the request
- Frontend stores the token in localStorage and sends it in every request header
- When a 401 is received, the frontend clears the token and shows the login screen

Next: how all the API routes are defined and what they do.

Section 10 — API Design

What is an API? REST principles, HTTP methods, and every route in Study-Hub

---

10.1 What Is an API?

API stands for Application Programming Interface. In the context of web development, an API is a set of rules for how software components communicate over HTTP.

Think of it as a waiter at a restaurant: - You (the frontend) don't go into the kitchen yourself - You tell the waiter (the API) what you want - The waiter goes to the kitchen (the database/services) and brings back your order - You don't need to know how the kitchen works

The API defines: - What you can ask for (which URLs/endpoints exist) - How to ask for it (which HTTP methods to use) - What format to send data in (JSON) - What format the response comes back in (JSON) - What happens if something goes wrong (error codes and messages)

10.2 REST — The Design Style

Study-Hub's API is **RESTful** (Representational State Transfer). REST is not a protocol — it's a set of design principles for building intuitive, consistent APIs.

The core idea: **URLs represent resources (things), and HTTP methods represent actions on those things.**

Resources and URLs

A resource is a thing your API manages. Study-Hub's resources: - Users - Notes - Note Folders - PDFs - PDF Folders - Chat Sessions - Chat Messages

URLs identify resources:

```
/api/notes                → the collection of all notes
/api/notes/abc-123        → one specific note (by ID)
/api/notes/folders        → the collection of note folders
/api/chat/sessions        → the collection of chat sessions
/api/chat/sessions/xyz/messages  → messages in a specific session
```

HTTP Methods as Actions

Method	Action	SQL equivalent
`GET`	Read/retrieve	SELECT
`POST`	Create new	INSERT
`PUT`	Replace/update	UPDATE
`PATCH`	Partial update	UPDATE (partial)
`DELETE`	Remove	DELETE

So you can intuitively guess what an endpoint does: - `GET /api/notes` → get all notes - `POST /api/notes` → create a new note - `PUT /api/notes/abc-123` → update note abc-123 - `DELETE /api/notes/abc-123` → delete note abc-123

This predictability is what makes REST intuitive.

10.3 Study-Hub's Complete API Reference

Auth Routes

`POST /api/auth/register` Create a new user account.

Request body:

```
{
  "name": "Alice",
  "email": "alice@example.com",
  "password": "mypassword123"
}
```

Success response (201 Created):

```
{
  "success": true,
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "user": {
    "id": "ac2c0621-89c1-4925-9c4c-40d5ace82e9a",
    "name": "Alice",
    "email": "alice@example.com",
    "avatarColor": "#43b97f"
  }
}
```

Possible errors: - `400` — Missing fields, password too short, invalid email format - `409` — Email already registered

---

`POST /api/auth/login` Log in with existing credentials.

Request body:

```
{ "email": "alice@example.com", "password": "mypassword123" }
```

Success response (200):

```
{ "success": true, "token": "eyJ...", "user": { ... } }
```

Possible errors: - `400` — Missing fields - `401` — Invalid email or password

---

`GET /api/auth/profile` Get the current user's profile. Requires auth.

Success response:

```
{
  "success": true,
  "user": { "id": "...", "name": "Alice", "email": "alice@example.com", "avatarColor": "#43b97f" }
}
```

`PUT /api/auth/profile` Update name or avatar color. Requires auth.

Request body:

```
{ "name": "Alice Smith", "avatarColor": "#ff6584" }
```

Notes Routes

All notes routes require authentication (`Authorization: Bearer <token>` header).

`GET /api/notes` Get all notes for the logged-in user.

Query parameters (all optional): - `folder` — filter by folder ID, or `"uncategorized"` for notes without a folder - `search` — full-text search query - `archived` — `"true"` to show archived notes (default: `"false"`) - `pinned` — `"true"` to show only pinned notes

Example: `GET /api/notes?search=machine+learning&folder=abc-123`

Success response:

```
{
  "success": true,
  "notes": [
    {
      "id": "xyz-789",
      "title": "Machine Learning Notes",
      "content": "# Chapter 1\n\nSupervised learning is...",
      "color": "#1a1a1a",
      "is_pinned": false,
      "is_archived": false,
      "tags": ["ml", "study"],
      "folder_id": "abc-123",
      "folder_name": "Study",
      "folder_color": "#43b97f",
      "created_at": "2026-04-20T14:30:00Z",
      "updated_at": "2026-04-21T09:15:00Z"
    }
  ]
}
```

`GET /api/notes/:id` Get one specific note by ID.

URL parameter: `:id` is replaced with the note's UUID in actual requests. Example: `GET /api/notes/xyz-789`

`POST /api/notes` Create a new note.

Request body:

```
{
  "title": "New Note",
  "content": "# My Note\n\nContent here...",
  "color": "#1a2035",
  "folder_id": "abc-123",
  "tags": ["study", "important"]
}
```

All fields are optional — a completely empty note is valid.

Success response (201):

```
{ "success": true, "note": { ...note object... } }
```

`PUT /api/notes/:id` Update a note. Only send the fields you want to change.

Request body (partial update — all fields optional):

```
{
  "title": "Updated Title",
  "is_pinned": true
}
```

`DELETE /api/notes/:id` Delete a note permanently.

Success response:

```
{ "success": true, "message": "Note deleted" }
```

`GET /api/notes/folders` Get all note folders with note counts.

Success response:

```
{
  "success": true,
  "folders": [
    { "id": "abc-123", "name": "Study", "icon": " ", "color": "#43b97f", "note_count": 5 }
  ]
}
```

`POST /api/notes/folders` Create a folder.

```
{ "name": "Research", "icon": " ", "color": "#6c63ff" }
```

`PUT /api/notes/folders/:id` Update a folder's name, icon, or color.

`DELETE /api/notes/folders/:id` Delete a folder. Notes inside become uncategorized (not deleted).

PDF Routes

`GET /api/pdfs` List all PDFs. Supports `?folder=id` filter.

`GET /api/pdfs/:id` Get one PDF's metadata.

`POST /api/pdfs/upload` Upload a PDF file. This is a **multipart form upload** (not JSON).

The frontend sends a `FormData` object:

```
const formData = new FormData();
formData.append('pdf', file);          // the PDF file
formData.append('folder_id', folderId);  // optional folder
formData.append('original_name', file.name);
```

The server receives it, saves the file to disk with a UUID filename, extracts text with `pdf-parse`, and stores metadata in the database.

Success response:

```
{
  "success": true,
  "pdf": {
    "id": "...",
    "filename": "832e87e7-25d9-45d8-b8ee-45220ecacf9b.pdf",
    "original_name": "lecture-3.pdf",
    "file_size": 1065095,
    "page_count": 42,
    "created_at": "..."
  }
}
```

`GET /api/pdfs/:id/stream` Stream the PDF file directly to the browser (for viewing in-browser).

This endpoint accepts the token as a URL query parameter (`?token=eyJ...`) because browser PDF viewers and `<iframe>` elements can't send custom headers:

```
// From api.js
streamUrl: (id) => `/api/pdfs/${id}/stream?token=${Storage.getToken()}`
```

`GET /api/pdfs/:id/download` Download the PDF file (forces browser "Save As" dialog).

Chat Routes

`GET /api/chat/sessions` List all chat sessions with their last message and message count.

Success response:

```
{
  "success": true,
  "sessions": [
    {
      "id": "session-uuid",
      "title": "Questions about Chapter 3",
      "last_message": "Thanks, that's very helpful!",
      "message_count": 12,
      "updated_at": "2026-04-23T20:00:00Z"
    }
  ]
}
```

`POST /api/chat/sessions` Create a new chat session.

```
{ "title": "New Chat" }
```

`PUT /api/chat/sessions/:id` Rename a session.

`DELETE /api/chat/sessions/:id` Delete a session and all its messages.

`GET /api/chat/sessions/:id/messages` Get all messages in a session.

Success response:

```
{
  "success": true,
  "messages": [
    { "id": "...", "role": "user", "content": "Summarize the PDF", "created_at": "..." },
    { "id": "...", "role": "assistant", "content": "Sure! The PDF covers...", "created_at": "..." }
  ]
}
```

`POST /api/chat/sessions/:sessionId/messages` Send a message and get an AI response. This is the most complex endpoint.

Request body:

```
{
  "content": "What are the key points of this PDF?",
  "context_note_ids": [],
  "context_pdf_ids": ["pdf-uuid-1", "pdf-uuid-2"]
}
```

`context_note_ids` and `context_pdf_ids` are arrays of IDs. The AI will be given the content of these notes/PDFs as context.

Success response:

```
{
  "success": true,
  "userMessage": { "id": "...", "role": "user", "content": "What are..." },
  "assistantMessage": { "id": "...", "role": "assistant", "content": "The key points are..." },
  "savedNote": null    // or the saved note object if AI created one
}
```

What happens server-side: 1. Save the user's message to the database 2. Fetch the content of any attached notes/PDFs 3. Fetch the last 20 messages in this session (conversation history) 4. Build a system prompt with all the context 5. Call OpenRouter API with the message history 6. Check if the response contains `[[CREATE_NOTE]]` tags — if so, save the note 7. Save the AI response to the database 8. Auto-update the session title if this is the first message 9. Return both messages and any saved note

---

`GET /api/health` Health check — returns 200 if the server is running.

```
{ "success": true, "status": "ok", "timestamp": "2026-04-24T10:30:00Z" }
```

Used by monitoring services to check if the server is alive.

10.4 The Standard Response Format

Study-Hub always responds with JSON in a consistent format:

Success:

```
{
  "success": true,
  "data": { ... }    // or "notes", "user", "session", etc.
}
```

Error:

```
{
  "success": false,
  "error": "Human-readable error message"
}
```

Why consistent format?

The frontend always knows where to look. It checks `data.success` first. If true, process `data.notes` (or whatever). If false, show `data.error` to the user. No guessing.

10.5 HTTP Status Codes Used in Study-Hub

Code	Meaning	When used
`200 OK`	Success	Successful GET, PUT, DELETE
`201 Created`	Successfully created	Successful POST (new resource created)
`400 Bad Request`	Invalid input	Missing fields, bad format
`401 Unauthorized`	Not logged in	Missing or invalid JWT
`403 Forbidden`	Logged in but not allowed	(would be used for admin features)
`404 Not Found`	Resource doesn't exist	Note/PDF/session not found
`409 Conflict`	Resource already exists	Duplicate email
`429 Too Many Requests`	Rate limit exceeded	Too many login attempts
`500 Internal Server Error`	Server crashed	Unexpected error

10.6 Ownership Checks — Security Detail

Every query that reads or modifies a resource includes an ownership check:

```
-- Getting a note
SELECT * FROM notes WHERE id = $1 AND user_id = $2
--                             ↑           ↑
--                          note id    logged-in user's id
```

If user Alice (id: `alice-uuid`) tries to access user Bob's note (user_id: `bob-uuid`), the query returns 0 rows — the note isn't found. The controller returns a 404.

Alice can never access Bob's data, because: - The JWT identifies Alice to the server (`req.user.id = "alice-uuid"`) - Every query includes `AND user_id = $2` where `$2 = req.user.id` - No matter what ID Alice sends in the URL, she can only get records where `user_id = "alice-uuid"`

This is **authorization** — not just "are you logged in" but "are you allowed to do this specific thing."

10.7 Key Takeaways

- An API is a set of rules for how programs communicate over HTTP
- REST uses URLs to identify resources and HTTP methods to define actions
- `GET` reads, `POST` creates, `PUT` updates, `DELETE` removes
- Consistent response format (`{ success, data/error }`) makes frontend code simpler
- HTTP status codes communicate what happened (200 OK, 400 bad input, 401 not logged in, etc.)
- Every database query includes an ownership check — users can only access their own data
- The chat endpoint is the most complex — it saves messages, gathers context, calls the AI, detects note creation, and updates session title

Next: the frontend architecture — how the single-page app is organized.

Section 11 — Frontend Architecture

How the Single-Page Application is organized, and the utility layer that powers it

---

11.1 What Is a Single-Page Application (SPA)?

A traditional website has multiple HTML pages. Clicking "Notes" sends a request for `notes.html`, clicking "PDFs" sends a request for `pdfs.html`. Every navigation causes a page reload — you see a white flash, the server gets hit, everything re-renders.

A **Single-Page Application** is different. There's one HTML file (`index.html`). JavaScript handles all navigation by dynamically showing and hiding sections, fetching data in the background, and updating only what needs to change — no page reloads.

Benefits: - Faster navigation (no round-trip to server) - Smoother experience (no flash/reload) - State persists between views (sidebar stays consistent, theme persists) - Can work offline (with the right setup)

Trade-offs: - More complex JavaScript - All code must load on first visit (initial load might be slower) - Requires special handling for the browser Back button

Study-Hub is a SPA. The entire app is `frontend/index.html` — a single file containing the auth screen and all three views (Notes, PDFs, Chat).

11.2 The Module System

Since Study-Hub doesn't use a build tool (no webpack, no React), it uses a classic pattern for organizing code into modules: **IIFEs** (Immediately Invoked Function Expressions).

Each file exports one global object:

```
// storage.js
const Storage = (() => {
  // private internals
  const PREFIX = 'studyhub_';

  const set = (key, value) => { ... };
  const get = (key, fallback) => { ... };

  // public API
  return { set, get, ... };
})();
```

After the file loads, `Storage` is available globally. Any subsequent file can call `Storage.getToken()`.

The load order in `index.html` matters:

```
<script src="/js/utils/storage.js"></script>   <!-- Storage is now global -->
<script src="/js/utils/helpers.js"></script>   <!-- Helpers is now global -->
<script src="/js/utils/api.js"></script>       <!-- API is now global, uses Storage -->
<script src="/js/modules/auth.js"></script>    <!-- Auth uses API, Storage, Helpers -->
<script src="/js/modules/notes.js"></script>
<script src="/js/modules/pdfs.js"></script>
<script src="/js/modules/chat.js"></script>
<script src="/js/app.js"></script>             <!-- App uses everything -->
```

11.3 storage.js — The localStorage Wrapper

Why wrap localStorage?

1. `localStorage` only stores strings. Wrapping it with `JSON.stringify`/`JSON.parse` lets you store objects.
2. A prefix (`studyhub_`) prevents conflicts with other apps or browser extensions.
3. `try/catch` handles the case where localStorage is disabled (private browsing in some browsers).
4. Named shortcuts make the intent clear (`Storage.getToken()` vs `localStorage.getItem('studyhub_token')`).

```
const Storage = (() => {
  const PREFIX = 'studyhub_';

  const set = (key, value) => {
    try {
      localStorage.setItem(PREFIX + key, JSON.stringify(value));
      return true;
    } catch (e) {
      console.warn('Storage.set failed:', e);
      return false;
    }
  };

  const get = (key, fallback = null) => {
    try {
      const raw = localStorage.getItem(PREFIX + key);
      // If key doesn't exist, raw is null → return fallback
```

```
      return raw !== null ? JSON.parse(raw) : fallback;
    } catch (e) {
      return fallback;  // JSON.parse might fail on corrupted data
    }
  };

  const remove = (key) => {
    try { localStorage.removeItem(PREFIX + key); return true; }
    catch (e) { return false; }
  };

  const clear = () => {
    // Only clear our own keys (not other apps')
    const keys = Object.keys(localStorage).filter(k => k.startsWith(PREFIX));
    keys.forEach(k => localStorage.removeItem(k));
  };

  // Named shortcuts
  const setToken = (token) => set('token', token);
  const getToken = () => get('token', null);
  const removeToken = () => remove('token');

  const setUser = (user) => set('user', user);
  const getUser = () => get('user', null);
  const removeUser = () => remove('user');

  const setTheme = (theme) => set('theme', theme);
  const getTheme = () => get('theme', 'dark');

  const setActiveView = (view) => set('activeView', view);
  const getActiveView = () => get('activeView', 'notes');  // default to notes

  const setActiveChatSession = (id) => set('activeChatSession', id);
  const getActiveChatSession = () => get('activeChatSession', null);

  return {
    set, get, remove, clear,
    setToken, getToken, removeToken,
    setUser, getUser, removeUser,
    setTheme, getTheme,
    setActiveView, getActiveView,
    setActiveChatSession, getActiveChatSession,
  };
})();
```

11.4 helpers.js — Utility Functions

This module contains pure utility functions that do one thing well and have no side effects.

renderMarkdown — The Custom Markdown Renderer

Study-Hub uses Markdown for notes and AI responses. Markdown is a simple text format:

```
# Heading
**bold text**
*italic text*
- list item
`inline code`
```

The function converts this to HTML. It's a lightweight custom implementation (no external library):

```
const renderMarkdown = (text) => {
  if (!text) return '';

  // Step 1: Extract code blocks (protect them from other processing)
  const codeBlocks = [];
  let html = text.replace(/```(\w*)\n?([\s\S]*?)```/gm, (_, lang, code) => {
    const idx = codeBlocks.length;
    const escaped = code.trim()
      .replace(/&/g, '&amp;').replace(/</g, '&lt;').replace(/>/g, '&gt;');
    codeBlocks.push(`<pre><code class="lang-${lang}">${escaped}</code></pre>`);
    return `\x00CODE${idx}\x00`;  // placeholder
  });

  // ... process headings, bold, italic, lists, links ...

  // Step 2: Restore code blocks (substitute placeholders back)
  html = html.replace(/\x00CODE(\d+)\x00/g, (_, i) => codeBlocks[i]);

  return `<div class="md-content">${html}</div>`;
};
```

The trickiest part: code blocks must be processed first (and their content saved aside) because `#`, `*`, `_` inside code blocks should NOT be interpreted as markdown — they're just characters.

`\x00` is a null character — used as a placeholder delimiter that would never appear in normal text.

The regex `/```(\w*)\n?([\s\S]*?)```/gm`: - ````` — literal backticks - `(\w*)` — optional language identifier (captured as group 1) - `\n?` — optional newline - `([\s\S]*?)` — content, lazily matched (captured as group 2). `[\s\S]` matches any character including newlines (unlike `.` which doesn't match newlines by default). `*?` is lazy — matches as little as possible. - `gm` flags: `g` = global (find all matches), `m` = multiline (`^` and `$` match line starts/ends)

formatDate — Human-Friendly Dates

```
const formatDate = (dateStr) => {
  if (!dateStr) return '';
  const date = new Date(dateStr);
  const now = new Date();
  const diffMs = now - date;        // milliseconds difference
  const diffSecs = Math.floor(diffMs / 1000);
  const diffMins = Math.floor(diffSecs / 60);
  const diffHrs = Math.floor(diffMins / 60);
  const diffDays = Math.floor(diffHrs / 24);

  if (diffSecs < 60)  return 'just now';
  if (diffMins < 60)  return `${diffMins}m ago`;
```

```
  if (diffHrs < 24)   return `${diffHrs}h ago`;
  if (diffDays === 1) return 'yesterday';
  if (diffDays < 7)   return `${diffDays}d ago`;

  return date.toLocaleDateString('en-US', { month: 'short', day: 'numeric' });
};
```

`new Date(dateStr)` parses an ISO 8601 string like `"2026-04-24T10:30:00Z"` into a JavaScript Date object. Subtracting two Date objects gives milliseconds between them.

formatFileSize — Human-Friendly File Sizes

```
const formatFileSize = (bytes) => {
  if (!bytes) return '0 B';
  const k = 1024;
  const sizes = ['B', 'KB', 'MB', 'GB'];
  const i = Math.floor(Math.log(bytes) / Math.log(k));
  // log(bytes) / log(1024) = log base 1024 of bytes
  // floor of that = which unit (0=B, 1=KB, 2=MB, 3=GB)
  return `${parseFloat((bytes / Math.pow(k, i)).toFixed(1))} ${sizes[i]}`;
};
// 1065095 bytes → "1.0 MB"
// 39475 bytes → "38.5 KB"
```

Toast — Pop-up Notifications

```
const toast = (() => {
  const container = () => document.getElementById('toast-container');

  const show = (message, type = 'info', duration = 3500) => {
    const icons = { success: ' ', error: ' ', info: ' ' };
    const el = document.createElement('div');
    el.className = `toast toast-${type}`;
    el.innerHTML = `
      <span class="toast-icon">${icons[type]}</span>
      <span class="toast-msg">${message}</span>
    `;

    container()?.appendChild(el);

    const remove = () => {
      el.classList.add('removing');  // triggers CSS exit animation
      el.addEventListener('animationend', () => el.remove(), { once: true });
      setTimeout(() => el.remove(), 500);  // fallback in case animationend doesn't fire
    };

    setTimeout(remove, duration);  // auto-remove after duration
    el.addEventListener('click', remove);  // click to dismiss
  };

  return {
    success: (msg, dur) => show(msg, 'success', dur),
    error:   (msg, dur) => show(msg, 'error',   dur || 5000),
    info:    (msg, dur) => show(msg, 'info',     dur),
```

```
  };
})();
```

`container()?. appendChild(el)` uses **optional chaining** (`?.`). If `container()` returns null (element not found), the `?.` prevents a crash and just does nothing.

`{ once: true }` in `addEventListener` means the listener automatically removes itself after firing once.

Errors get a longer default duration (`5000ms`) because users need more time to read error messages.

debounce

```
const debounce = (fn, delay = 300) => {
  let timer;
  return (...args) => {
    clearTimeout(timer);                          // cancel previous
    timer = setTimeout(() => fn(...args), delay); // schedule new
  };
};
```

How this works:

Imagine the user types "study" in 400ms: - `s` typed → timer set for 300ms - `t` typed (100ms later) → old timer cancelled, new timer set for 300ms - `u` typed (100ms later) → old timer cancelled, new timer set - `d` typed → same - `y` typed → same - 300ms pass with no more typing → `fn("study")` fires

Only one API call made, not five.

11.5 api.js — The HTTP Client

This module wraps `fetch()` (the browser's built-in HTTP client) and adds: - Automatic JWT token injection - Consistent error handling - Session expiry detection - File upload support with progress

The core `request` function

```
const request = async (method, path, body = null, options = {}) => {
  const token = Storage.getToken();

  const headers = {
    'Content-Type': 'application/json',
    ...(token ? { 'Authorization': `Bearer ${token}` } : {}),
    ...(options.headers || {}),
  };

  const config = {
    method,
    headers,
    ...(body && method !== 'GET' ? { body: JSON.stringify(body) } : {}),
  };

  try {
```

```
    const res = await fetch(BASE + path, config);

    if (res.status === 401) {
      Storage.removeToken();
      Storage.removeUser();
      window.dispatchEvent(new CustomEvent('auth:expired'));
      throw new Error('Session expired. Please sign in again.');
    }

    const data = await res.json();

    if (!res.ok) {
      // res.ok is true for 200-299. For 400, 404, 500 etc, throw the error message
      throw new Error(data.error || data.message || `Request failed (${res.status})`);
    }

    return data;  // the parsed JSON response
  } catch (err) {
    if (err.name === 'TypeError' && err.message.includes('fetch')) {
      throw new Error('Network error. Please check your connection.');
    }
    throw err;  // re-throw so calling code can catch it
  }
};
```

`...(token ? { 'Authorization': ... } : {})` — conditionally spread: if token exists, add the Authorization header; if not, spread an empty object (adds nothing).

`body && method !== 'GET' ? { body: JSON.stringify(body) } : {}` — only add a body for non-GET requests. GET requests can't have a body (HTTP spec).

The upload function (XMLHttpRequest for progress tracking)

```
const upload = async (path, formData, onProgress) => {
  const token = Storage.getToken();

  return new Promise((resolve, reject) => {
    const xhr = new XMLHttpRequest();

    xhr.open('POST', BASE + path);
    if (token) xhr.setRequestHeader('Authorization', `Bearer ${token}`);

    xhr.upload.onprogress = (e) => {
      if (e.lengthComputable && onProgress) {
        onProgress(Math.round((e.loaded / e.total) * 100));
        // called with 0, 23, 47, 75, 100 as the upload progresses
      }
    };

    xhr.onload = () => {
      try {
        const data = JSON.parse(xhr.responseText);
        if (xhr.status >= 200 && xhr.status < 300) {
          resolve(data);
        } else {
```

```
          reject(new Error(data.error || 'Upload failed'));
        }
      } catch {
        reject(new Error('Invalid response'));
      }
    };

    xhr.onerror = () => reject(new Error('Network error during upload'));
    xhr.send(formData);
  });
};
```

Why use `XMLHttpRequest` instead of `fetch` for uploads?

`fetch` doesn't support progress events for uploads. `XMLHttpRequest` (the older API) does. The `xhr.upload.onprogress` callback fires repeatedly as the upload progresses, allowing you to show a progress bar.

The function wraps XHR in a `Promise` so it can be `await`ed just like fetch.

Domain methods

```
const auth = {
  login:    (body) => post('/auth/login',    body),
  register: (body) => post('/auth/register', body),
  profile:  ()     => get('/auth/profile'),
  update:   (body) => put('/auth/profile',   body),
};

const notes = {
  getFolders:    ()            => get('/notes/folders'),
  createFolder:  (body)        => post('/notes/folders', body),
  updateFolder:  (id, body)    => put(`/notes/folders/${id}`, body),
  deleteFolder:  (id)          => del(`/notes/folders/${id}`),
  getAll:        (params = {}) => {
    const q = new URLSearchParams(params).toString();
    return get('/notes' + (q ? '?' + q : ''));
  },
  // ...
};
```

`new URLSearchParams({ search: 'study', folder: 'abc' }).toString()` → `"search=study&folder=abc"`

This builds the query string for GET requests with filters.

11.6 app.js — The Orchestrator

`app.js` is the central controller. It: 1. Decides whether to show the auth screen or the app 2. Handles view routing (notes/pdfs/chat) 3. Manages the theme system 4. Controls the sidebar 5. Handles global search 6. Initializes all feature modules

Theme System

```
const applyTheme = (style, mode) => {
  const attr = style + '-' + mode;  // e.g., 'glass-dark' or 'minimal-light'
  document.documentElement.dataset.theme = attr;
  // This changes data-theme on <html>, which CSS immediately reacts to

  Storage.set('themeStyle', style);
  Storage.set('themeMode', mode);
};

const toggleMode = () => {
  const style = Storage.get('themeStyle', 'minimal');
  const mode = Storage.get('themeMode', 'dark');
  applyTheme(style, mode === 'dark' ? 'light' : 'dark');
};
```

The theme is stored in localStorage so it persists across sessions.

View Routing

```
const navigateTo = (view) => {
  state.activeView = view;
  Storage.setActiveView(view);

  // Hide all views
  ['notes', 'pdfs', 'chat'].forEach(v => {
    Helpers.hide(el(`${v}-view`));
  });

  // Show selected view
  Helpers.show(el(`${view}-view`));

  // Update sidebar active state
  document.querySelectorAll('.nav-item').forEach(item => {
    item.classList.toggle('active', item.dataset.view === view);
  });

  // Initialize the module for this view
  if (view === 'notes') Notes.init();
  if (view === 'pdfs') Pdfs.init();
  if (view === 'chat') Chat.init();
};
```

Each view is a `<div>` in the HTML. The router shows one and hides the others. Each module's `init()` function is called when you navigate to that view, loading fresh data.

Initialization

```
const init = async () => {
  if (state.initialized) return;

  const token = Storage.getToken();
  if (!token) {
    showAuthScreen();
```

```
    return;
  }

  try {
    // Verify token is still valid
    const data = await API.auth.profile();
    state.user = data.user;
    showApp();
    navigateTo(Storage.getActiveView());
    state.initialized = true;
  } catch (err) {
    // Token invalid or expired
    Storage.clear();
    showAuthScreen();
  }
};
```

On page load, check localStorage for a token. If found, verify it with the server. If valid, show the app. If invalid (expired, tampered), clear storage and show login.

11.7 The Data Flow Pattern

Every feature module follows the same pattern:

```
User action (click, type, submit)
    ↓
Event listener fires
    ↓
Validate input (if needed)
    ↓
Show loading state (disable button, show spinner)
    ↓
await API.something.doSomething(data)
    ↓
Update UI with response (re-render list, show toast)
    ↓
Hide loading state
    ↓
catch (err) → show error toast
```

Example from notes:

```
const handleCreateNote = async () => {
  Helpers.setLoading(createBtn, true);  // disable button, show spinner

  try {
    const data = await API.notes.create({ title: '', content: '' });
    state.notes.unshift(data.note);  // add to front of local array
    renderNotes();                    // re-render the grid
    openNote(data.note);             // open the editor for the new note
    Helpers.toast.success('Note created');
  } catch (err) {
    Helpers.toast.error(err.message);
  } finally {
```

```
    Helpers.setLoading(createBtn, false);  // always restore button
  }
};
```

`finally` runs regardless of success or failure — always restores the button state.

11.8 Key Takeaways

- A SPA has one HTML file; JavaScript shows/hides sections and fetches data
- The IIFE pattern (`const X = (() => { ... })()`) creates isolated modules
- Files must be loaded in dependency order in index.html
- `Storage` wraps localStorage with prefix, JSON serialization, and error handling
- `Helpers` provides pure utility functions: markdown rendering, date formatting, toast notifications, debounce
- `API` wraps `fetch()` with token injection, error handling, and a clean interface for every endpoint
- `App` orchestrates everything: auth check, routing, theme, sidebar, initialization
- Every feature module follows the same action → API call → UI update pattern

The next three sections go through each feature module in detail.

Section 12 — The Notes Feature

Building the notes system — folders, cards, editor, search, pin, archive

---

12.1 What the Notes Feature Does

The notes feature is the core of Study-Hub. Users can: - Create notes with a rich Markdown editor - Organize notes into folders (Personal, Study, Work, etc.) - Pin important notes (they float to the top) - Archive old notes (hide them without deleting) - Search notes by content (full-text search) - Color-code notes - Tag notes for organization - Delete notes

---

12.2 The Backend: notesController.js

getFolders

```
const getFolders = async (req, res, next) => {
  try {
    const result = await query(
      `SELECT f.*, COUNT(n.id)::int AS note_count
       FROM note_folders f
       LEFT JOIN notes n ON n.folder_id = f.id AND n.is_archived = false
       WHERE f.user_id = $1
       GROUP BY f.id
       ORDER BY f.created_at ASC`,
      [req.user.id]
    );
    res.json({ success: true, folders: result.rows });
```

```
  } catch (err) { next(err); }
};
```

This query does several things at once:

`SELECT f.*` — all columns from note_folders

`COUNT(n.id)::int AS note_count` — count how many notes are in each folder. `::int` casts the result from bigint (what COUNT returns) to integer. `AS note_count` names this calculated column.

`LEFT JOIN notes n ON n.folder_id = f.id AND n.is_archived = false` — join each folder with its notes, but only non-archived ones. We count active notes, not archived ones.

`GROUP BY f.id` — required when using aggregate functions (COUNT). Groups all notes per folder so COUNT works per folder, not globally.

`ORDER BY f.created_at ASC` — folders appear in the order they were created (oldest first).

getNotes — with filtering and search

```
const getNotes = async (req, res, next) => {
  try {
    const { folder, search, archived = 'false', pinned } = req.query;
    const values = [req.user.id];
    let idx = 2;  // parameter index (starts at 2 because $1 is already user.id)
    const conditions = ['n.user_id = $1'];

    conditions.push(`n.is_archived = ${archived === 'true'}`);
    // archived === 'true' compares the string (query params are always strings)
    // Produces either: n.is_archived = true OR n.is_archived = false

    if (folder === 'uncategorized') {
      conditions.push('n.folder_id IS NULL');
    } else if (folder) {
      conditions.push(`n.folder_id = $${idx++}`);
      values.push(folder);
    }

    if (pinned === 'true') {
      conditions.push('n.is_pinned = true');
    }

    if (search) {
      conditions.push(
        `to_tsvector('english', COALESCE(n.title,'') || ' ' || COALESCE(n.content,''))
         @@ plainto_tsquery('english', $${idx++})`
      );
      values.push(search);
    }

    const sql = `
      SELECT n.*, f.name AS folder_name, f.color AS folder_color, f.icon AS folder_icon
      FROM notes n
      LEFT JOIN note_folders f ON f.id = n.folder_id
      WHERE ${conditions.join(' AND ')}
      ORDER BY n.is_pinned DESC, n.updated_at DESC
```

```
    `;

    const result = await query(sql, values);
    res.json({ success: true, notes: result.rows });
  } catch (err) { next(err); }
};
```

This is **dynamic SQL building** — constructing the query based on which filters are active. The approach:

1. Start with a base condition array: `['n.user_id = $1']`
2. Push more conditions as filters are found
3. Build values array in parallel
4. Use `conditions.join(' AND ')` to combine into a WHERE clause

`ORDER BY n.is_pinned DESC, n.updated_at DESC` — pinned notes first (`DESC` = descending, so `true` > `false`), then by most recently updated.

createNote

```
const createNote = async (req, res, next) => {
  try {
    const { title = '', content = '', color = '#1a1a1a', folder_id = null, tags = [] } = req.body;
    // Destructuring with defaults - if any field is not provided, use the default

    const result = await query(
      `INSERT INTO notes (user_id, title, content, color, folder_id, tags)
       VALUES ($1,$2,$3,$4,$5,$6) RETURNING *`,
      [req.user.id, title, content, color, folder_id || null, tags]
    );
    res.status(201).json({ success: true, note: result.rows[0] });
  } catch (err) { next(err); }
};
```

`RETURNING *` — after the INSERT, PostgreSQL returns the complete inserted row (including the auto-generated `id`, `created_at`, etc.). This is PostgreSQL-specific and very useful.

`folder_id || null` — if `folder_id` is an empty string (no folder selected), convert to `null` so the database stores a proper NULL, not an empty string.

updateNote — Dynamic Updates

```
const updateNote = async (req, res, next) => {
  try {
    const { id } = req.params;
    const { title, content, color, folder_id, tags, is_pinned, is_archived } = req.body;
    const updates = [];
    const values = [];
    let idx = 1;

    if (title !== undefined)     { updates.push(`title = $${idx++}`);      values.push(title); }
    if (content !== undefined)   { updates.push(`content = $${idx++}`);    values.push(content); }
    if (color !== undefined)     { updates.push(`color = $${idx++}`);      values.push(color); }
    if (folder_id !== undefined) { updates.push(`folder_id = $${idx++}`);  values.push(folder_id || null
    if (tags !== undefined)      { updates.push(`tags = $${idx++}`);       values.push(tags); }
    if (is_pinned !== undefined) { updates.push(`is_pinned = $${idx++}`);  values.push(is_pinned); }
```

```
    if (is_archived !== undefined){ updates.push(`is_archived = $${idx++}`); values.push(is_archived); 

    if (!updates.length) {
      return res.status(400).json({ success: false, error: 'Nothing to update' });
    }

    values.push(id, req.user.id);
    const result = await query(
      `UPDATE notes SET ${updates.join(', ')} WHERE id = $${idx++} AND user_id = $${idx} RETURNING *`,
      values
    );
    if (!result.rows.length) {
      return res.status(404).json({ success: false, error: 'Note not found' });
    }
    res.json({ success: true, note: result.rows[0] });
  } catch (err) { next(err); }
};
```

Why check `!== undefined` instead of truthiness? Because you might legitimately send `title: ''` (an empty string) to clear the title. An empty string is falsy, so `if (title)` would skip it. `if (title !== undefined)` correctly includes it.

The dynamic SQL builds:

```
UPDATE notes SET title = $1, is_pinned = $2 WHERE id = $3 AND user_id = $4
```

Only the provided fields are updated. If you send `{ is_pinned: true }`, only `is_pinned` changes — the title stays as-is.

12.3 The Frontend: notes.js

The Notes module manages the entire notes UI. Here's the conceptual flow:

State

Every module maintains local state — a snapshot of what's currently loaded:

```
let state = {
  notes: [],           // all loaded notes
  folders: [],         // all note folders
  activeFolder: null, // currently selected folder ID (null = all)
  activeNote: null,   // currently open note object
  searchQuery: '',
  saveTimer: null,    // for auto-save debouncing
};
```

init() — Load Data

When you navigate to the Notes view, `Notes.init()` is called:

```
const init = async () => {
  await Promise.all([loadFolders(), loadNotes()]);
  // Promise.all runs both in parallel - faster than sequential
};
```

`Promise.all([p1, p2])` starts both promises at the same time and waits for both to complete. If they ran sequentially, it would take twice as long.

loadNotes()

```
const loadNotes = async () => {
  try {
    const params = {};
    if (state.activeFolder) params.folder = state.activeFolder;
    if (state.searchQuery) params.search = state.searchQuery;

    const data = await API.notes.getAll(params);
    state.notes = data.notes;
    renderNotes();
  } catch (err) {
    Helpers.toast.error('Failed to load notes');
  }
};
```

renderNotes() — Building the UI

```
const renderNotes = () => {
  const grid = document.getElementById('notes-grid');

  if (state.notes.length === 0) {
    grid.innerHTML = `
      <div class="empty-state">
        <div class="empty-icon"> </div>
        <p>No notes yet</p>
        <button class="btn btn-primary" onclick="Notes.createNote()">Create your first note</button>
      </div>
    `;
    return;
  }

  grid.innerHTML = state.notes.map(note => `
    <div class="note-card ${note.is_pinned ? 'pinned' : ''}"
         style="background: ${note.color}"
         data-id="${note.id}"
         onclick="Notes.openNote('${note.id}')">

      <div class="note-card-header">
        ${note.is_pinned ? '<span class="pin-icon"> </span>' : ''}
        <span class="note-date">${Helpers.formatDate(note.updated_at)}</span>
      </div>

      <h3 class="note-title">${Helpers.escHtml(note.title) || 'Untitled'}</h3>

      <p class="note-preview">
        ${Helpers.truncate(Helpers.escHtml(stripMarkdown(note.content)), 120)}
      </p>

      ${note.folder_name ? `
        <div class="note-folder" style="color: ${note.folder_color}">
```

```
          ${note.folder_icon} ${Helpers.escHtml(note.folder_name)}
        </div>
      ` : ''}

      ${note.tags?.length ? `
        <div class="note-tags">
          ${note.tags.map(tag => `<span class="tag">${Helpers.escHtml(tag)}</span>`).join('')}
        </div>
      ` : ''}
    </div>
  `).join('');
};
```

Key points:

`Helpers.escHtml(note.title)` — this is critical for security. If a note title is `<script>alert('hacked')</script>`, `escHtml` converts `<` to `<` and `>` to `>`, making it display as text rather than executing as HTML. This prevents **Cross-Site Scripting (XSS)**.

```
const escHtml = (str) => {
  const div = document.createElement('div');
  div.textContent = str;   // textContent never interprets HTML
  return div.innerHTML;    // innerHTML returns escaped version
};
```

`Helpers.truncate(text, 120)` — cuts long content to 120 characters with an ellipsis (…) for the card preview.

`note.tags?.length` — optional chaining: if `note.tags` is null/undefined, don't throw an error, just return undefined (falsy).

Template literals with `${}` embed JavaScript expressions directly in HTML strings — modern and clean.

Auto-save — The Editor

```
const openNote = async (noteId) => {
  const note = state.notes.find(n => n.id === noteId)
               || await API.notes.getOne(noteId).then(d => d.note);

  state.activeNote = note;

  // Populate editor
  titleInput.value = note.title;
  contentTextarea.value = note.content;

  // Show editor panel
  Helpers.show(editorPanel);
};

// Auto-save on any change
const debouncedSave = Helpers.debounce(async () => {
  if (!state.activeNote) return;

  try {
    const updated = await API.notes.update(state.activeNote.id, {
      title: titleInput.value,
```

```
      content: contentTextarea.value,
    });
    state.activeNote = updated.note;

    // Update the card in the grid too
    const idx = state.notes.findIndex(n => n.id === updated.note.id);
    if (idx !== -1) state.notes[idx] = updated.note;
    renderNotes();
  } catch (err) {
    Helpers.toast.error('Failed to save note');
  }
}, 1000);  // save 1 second after user stops typing

titleInput.addEventListener('input', debouncedSave);
contentTextarea.addEventListener('input', debouncedSave);
```

Auto-save means users never have to click "Save." Every time they stop typing for 1 second, the note is saved automatically. The debounce ensures we don't spam the server with a request for every keystroke.

Folder Sidebar

```
const renderFolders = () => {
  const nav = document.getElementById('notes-nav');

  nav.innerHTML = `
    <div class="nav-section-title">Folders</div>

    <div class="nav-item ${!state.activeFolder ? 'active' : ''}"
         onclick="Notes.filterByFolder(null)">
      <span> </span> All Notes
      <span class="nav-count">${state.notes.length}</span>
    </div>

    ${state.folders.map(folder => `
      <div class="nav-item ${state.activeFolder === folder.id ? 'active' : ''}"
           onclick="Notes.filterByFolder('${folder.id}')">
        <span>${folder.icon}</span>
        ${Helpers.escHtml(folder.name)}
        <span class="nav-count">${folder.note_count}</span>
      </div>
    `).join('')}

    <button class="nav-add-btn" onclick="Notes.showCreateFolderModal()">
      + New Folder
    </button>
  `;
};

const filterByFolder = async (folderId) => {
  state.activeFolder = folderId;
  renderFolders();  // update active state
  await loadNotes();  // reload notes for this folder
};
```

Clicking a folder in the sidebar: 1. Updates `state.activeFolder` 2. Re-renders the sidebar (to show which folder is active) 3. Reloads notes with the folder filter

---

12.4 Pinning and Archiving

```
const togglePin = async (noteId) => {
  const note = state.notes.find(n => n.id === noteId);
  if (!note) return;

  try {
    const updated = await API.notes.update(noteId, { is_pinned: !note.is_pinned });
    const idx = state.notes.findIndex(n => n.id === noteId);
    state.notes[idx] = updated.note;
    renderNotes();
    Helpers.toast.success(updated.note.is_pinned ? 'Note pinned' : 'Note unpinned');
  } catch (err) {
    Helpers.toast.error(err.message);
  }
};

const archiveNote = async (noteId) => {
  try {
    await API.notes.update(noteId, { is_archived: true });
    // Remove from local state immediately (no need to reload from server)
    state.notes = state.notes.filter(n => n.id !== noteId);
    renderNotes();

    if (state.activeNote?.id === noteId) {
      state.activeNote = null;
      Helpers.hide(editorPanel);
    }

    Helpers.toast.success('Note archived');
  } catch (err) {
    Helpers.toast.error(err.message);
  }
};
```

Optimistic UI update: after a successful API call, update the local state directly (`state.notes[idx] = updated.note`) and re-render — faster than fetching everything from the server again.

12.5 Full-Text Search

```
const searchInput = document.getElementById('global-search');

searchInput.addEventListener('input', Helpers.debounce((e) => {
  state.searchQuery = e.target.value.trim();
  loadNotes();  // reload with search filter
}, 400));
```

The search query is sent to the server, which uses PostgreSQL's full-text search. The server returns only matching notes. This is better than client-side filtering (which requires loading all notes) — it works even with thousands of notes.

12.6 The Complete Data Flow for Creating a Note

```
1. User clicks "New Note" button
2. handleCreateNote() runs
3. API.notes.create({ title: '', content: '' }) called
4. POST /api/notes with { title: '', content: '' }
5. Server: INSERT INTO notes (user_id, title, content, ...) VALUES (...) RETURNING *
6. Server returns { success: true, note: { id: 'new-uuid', title: '', ... } }
7. state.notes.unshift(note) - add to front of local array
8. renderNotes() - re-render the grid (new note appears first)
9. openNote(note.id) - open the editor for the new note
10. User types title "Machine Learning" → debouncedSave queued
11. User types content → debouncedSave queued
12. 1 second after last keystroke: PUT /api/notes/new-uuid { title: 'Machine Learning', content: '...' }
13. Server updates the note, returns updated note
14. state.notes updated, grid re-rendered (shows new title on card)
```

12.7 Key Takeaways

- Notes are stored in PostgreSQL with full-text search indexes for fast searching
- Dynamic SQL building allows flexible filtering (by folder, search, pinned, archived)
- `RETURNING *` in INSERT/UPDATE gives back the complete row so the client has it
- The frontend maintains local state and re-renders the grid on changes
- Auto-save uses debouncing — saves 1 second after the user stops typing
- Optimistic UI updates (update local state immediately after success) feel faster
- `escHtml()` is used on any user-provided content to prevent XSS attacks
- `Promise.all()` loads folders and notes simultaneously for faster initialization

Section 13 — The PDF Feature

Upload, text extraction, in-browser viewing, and the file storage system

---

13.1 What the PDF Feature Does

Users can: - Upload PDF files (up to 50MB by default) - View PDFs directly in the browser (no download needed) - Download PDFs - Organize PDFs into folders - Search PDFs by name - Use PDFs as context for AI chat (the AI reads the extracted text)

13.2 File Upload Architecture

When a user uploads a file, several things need to happen: 1. The file travels from the browser to the server (multipart HTTP upload) 2. The server saves the file to disk 3. The server extracts

text from the PDF 4. The server stores file metadata in the database 5. The server responds with the new PDF record

Why not store files in the database?

Storing file *metadata* (name, size, path) in the database is fine. But storing the actual file bytes in the database is usually a bad idea: - Databases are optimized for structured data, not binary blobs - Large binary data makes the database slow and massive - Serving files from a database is much slower than from disk - Backups become huge

Instead, files live on disk in `uploads/pdfs/<user-id>/<uuid>.pdf` and the database stores just the path.

13.3 middleware/upload.js — Multer Configuration

Multer is the middleware that handles file uploads. It intercepts requests with `Content-Type: multipart/form-data` and parses the file from the request.

```
const multer = require('multer');
const path = require('path');
const fs = require('fs');

const uploadDir = process.env.UPLOAD_DIR || './uploads/pdfs';

// Ensure upload directory exists when the server starts
if (!fs.existsSync(uploadDir)) {
  fs.mkdirSync(uploadDir, { recursive: true });
}
```

`fs` is Node.js's built-in file system module. `fs.existsSync()` checks if a path exists. `fs.mkdirSync(path, { recursive: true })` creates the folder (and any missing parent folders).

```
const storage = multer.diskStorage({
  destination: (req, file, cb) => {
    // Each user gets their own subfolder
    const userDir = path.join(uploadDir, req.user.id);
    if (!fs.existsSync(userDir)) {
      fs.mkdirSync(userDir, { recursive: true });
    }
    cb(null, userDir);  // cb(error, destination_path)
  },

  filename: (req, file, cb) => {
    const { v4: uuidv4 } = require('uuid');
    const ext = path.extname(file.originalname);  // e.g., '.pdf'
    cb(null, `${uuidv4()}${ext}`);  // random UUID + extension
    // e.g., "832e87e7-25d9-45d8-b8ee-45220ecacf9b.pdf"
  }
});
```

`multer.diskStorage()` configures where and how files are saved: - `destination` — runs for each upload, creates user folder, tells Multer where to save the file - `filename` — generates the on-disk filename (UUID prevents collisions and hides original names)

`cb` is a **callback function** — Multer calls it with `(error, result)`. `cb(null, value)` means no error, here's the value.

```
const fileFilter = (req, file, cb) => {
  if (file.mimetype === 'application/pdf') {
    cb(null, true);   // accept the file
  } else {
    cb(new Error('Only PDF files are allowed'), false);  // reject
  }
};
```

MIME type (Multipurpose Internet Mail Extensions) is a standardized identifier for file types. PDFs are always `application/pdf`. This filter rejects anything that isn't a PDF.

Note: MIME type can be spoofed — a malicious user could rename a `.exe` to `.pdf`. Production systems would also inspect the file's magic bytes (the first few bytes that identify the file type). For Study-Hub's use case, MIME check is sufficient.

```
const maxSizeMB = parseInt(process.env.MAX_FILE_SIZE_MB) || 50;

const upload = multer({
  storage,
  fileFilter,
  limits: {
    fileSize: maxSizeMB * 1024 * 1024  // convert MB to bytes
  }
});

module.exports = upload;
```

`limits.fileSize` prevents memory exhaustion from huge uploads. `50 * 1024 * 1024 = 52,428,800 bytes = 50MB`.

In the route:

```
router.post('/pdfs/upload', authenticate, upload.single('pdf'), pdfController.uploadPdf);
```

`upload.single('pdf')` tells Multer to expect one file with the field name `'pdf'`. After Multer processes it, `req.file` is available to the controller:

```
req.file = {
  fieldname: 'pdf',
  originalname: 'lecture-3.pdf',
  encoding: '7bit',
  mimetype: 'application/pdf',
  destination: './uploads/pdfs/user-uuid',
  filename: '832e87e7-25d9-45d8-b8ee-45220ecacf9b.pdf',
  path: './uploads/pdfs/user-uuid/832e87e7....pdf',
  size: 1065095    // bytes
}
```

13.4 PDF Text Extraction — pdfService.js

```
// services/pdfService.js
const pdfParse = require('pdf-parse');
const fs = require('fs');
```

```
const extractText = async (filePath) => {
  try {
    const dataBuffer = fs.readFileSync(filePath);
    const data = await pdfParse(dataBuffer);
    return {
      text: data.text,
      pageCount: data.numpages
    };
  } catch (err) {
    console.error('PDF text extraction failed:', err.message);
    return { text: '', pageCount: 0 };
  }
};

module.exports = { extractText };
```

`pdf-parse` is a Node.js library that reads PDF binary data and extracts the text content. It reads the PDF's internal text objects.

Limitation: Only PDFs with embedded text work. Scanned PDFs (photos of pages) have no embedded text — they're just images. For those, you'd need OCR (Optical Character Recognition) — a more complex system not included in Study-Hub.

`fs.readFileSync(filePath)` reads the entire file into memory as a `Buffer` (binary data). This is synchronous (blocking) — for large files in production, you'd use the async version. For Study-Hub's scale, this is fine.

13.5 pdfController.js — The Upload Handler

```
const uploadPdf = async (req, res, next) => {
  try {
    if (!req.file) {
      return res.status(400).json({ success: false, error: 'No file uploaded' });
    }

    const { folder_id } = req.body;

    // Extract text from the uploaded PDF
    const { text, pageCount } = await pdfService.extractText(req.file.path);

    // Store metadata in database
    const result = await query(
      `INSERT INTO pdfs (user_id, folder_id, filename, original_name, file_size, page_count, extracted_t
       VALUES ($1, $2, $3, $4, $5, $6, $7, $8) RETURNING *`,
      [
        req.user.id,
        folder_id || null,
        req.file.filename,        // UUID filename on disk
        req.file.originalname,    // original user-provided name
        req.file.size,
        pageCount,
        text,                     // extracted text (can be very long)
        req.file.path
      ]
```

```
    );

    res.status(201).json({ success: true, pdf: result.rows[0] });
  } catch (err) {
    // If anything fails, clean up the uploaded file
    if (req.file?.path) {
      try { fs.unlinkSync(req.file.path); } catch {}  // delete the orphaned file
    }
    next(err);
  }
};
```

Error cleanup: if the database insert fails after the file was already saved to disk, we delete the orphaned file. Otherwise you'd have files on disk with no database record — "ghost files."

`fs.unlinkSync(path)` deletes a file synchronously. The empty `catch` (`catch {}`) means "if the delete also fails, just ignore it" — the error reporting is more important than the file cleanup.

Streaming PDFs to the Browser

```
const streamPdf = async (req, res, next) => {
  try {
    const result = await query(
      'SELECT file_path, original_name, file_size FROM pdfs WHERE id = $1 AND user_id = $2',
      [req.params.id, req.user.id]
    );

    if (!result.rows.length) {
      return res.status(404).json({ success: false, error: 'PDF not found' });
    }

    const pdf = result.rows[0];

    if (!fs.existsSync(pdf.file_path)) {
      return res.status(404).json({ success: false, error: 'File not found on disk' });
    }

    // Set headers for PDF display (not download)
    res.setHeader('Content-Type', 'application/pdf');
    res.setHeader('Content-Disposition', `inline; filename="${pdf.original_name}"`);
    res.setHeader('Content-Length', pdf.file_size);

    // Stream the file (don't load entire file into memory first)
    const stream = fs.createReadStream(pdf.file_path);
    stream.pipe(res);
    // pipe connects the read stream to the response write stream
    // Data flows from disk → response without buffering it all in memory
  } catch (err) { next(err); }
};
```

`Content-Disposition: inline` tells the browser to display the file rather than download it. `inline` vs `attachment` (which triggers the Save dialog).

Streaming is crucial for large files. Instead of reading the entire 50MB PDF into memory before sending, `fs.createReadStream()` reads chunks from disk and `pipe()` sends each chunk to the response as it's read. Memory usage stays low regardless of file size.

13.6 Frontend: pdfs.js

Upload with Progress Bar

```
const uploadPdf = async (file, folderId) => {
  showUploadProgress(0);

  const formData = new FormData();
  formData.append('pdf', file);
  if (folderId) formData.append('folder_id', folderId);

  try {
    const data = await API.pdfs.uploadFile(formData, (percent) => {
      showUploadProgress(percent);  // callback called with 0...100
    });

    state.pdfs.unshift(data.pdf);
    renderPdfs();
    Helpers.toast.success(`"${file.name}" uploaded successfully`);
  } catch (err) {
    Helpers.toast.error(err.message);
  } finally {
    hideUploadProgress();
  }
};

const showUploadProgress = (percent) => {
  const bar = document.getElementById('upload-progress-bar');
  const label = document.getElementById('upload-progress-label');
  bar.style.width = `${percent}%`;
  label.textContent = `Uploading... ${percent}%`;
};
```

`FormData` is the browser's class for building multipart form data. The `append(name, value)` method adds fields. When `fetch` or XHR receives a `FormData`, it automatically sets `Content-Type: multipart/form-data` with the correct boundary.

Drag-and-Drop Upload

```
const initDragDrop = () => {
  const dropZone = document.getElementById('pdf-drop-zone');

  dropZone.addEventListener('dragover', (e) => {
    e.preventDefault();  // necessary to allow drop
    dropZone.classList.add('drag-over');
  });

  dropZone.addEventListener('dragleave', () => {
    dropZone.classList.remove('drag-over');
  });

  dropZone.addEventListener('drop', (e) => {
```

```
    e.preventDefault();
    dropZone.classList.remove('drag-over');

    const files = e.dataTransfer.files;
    const pdf = Array.from(files).find(f => f.type === 'application/pdf');

    if (pdf) {
      uploadPdf(pdf, state.activeFolder);
    } else {
      Helpers.toast.error('Please drop a PDF file');
    }
  });
};
```

`e.preventDefault()` on `dragover` is required — without it, dropping a file would navigate the browser to that file (default browser behaviour for file drops).

`e.dataTransfer.files` is a `FileList` containing the dropped files. `Array.from()` converts it to a regular array so you can use `.find()`.

In-Browser PDF Viewer

```
const openPdf = (pdf) => {
  state.activePdf = pdf;

  const streamUrl = API.pdfs.streamUrl(pdf.id);
  // Returns: /api/pdfs/{id}/stream?token=eyJ...

  const viewer = document.getElementById('pdf-viewer');
  viewer.innerHTML = `
    <iframe
      src="${streamUrl}"
      class="pdf-iframe"
      title="${Helpers.escHtml(pdf.original_name)}"
    ></iframe>
  `;

  Helpers.show(document.getElementById('pdf-view-panel'));
};
```

An `<iframe>` is an embedded browser window. The browser knows how to display PDFs natively in an iframe.

The token in the URL problem: iframes and browser PDF viewers can't send custom HTTP headers (like `Authorization: Bearer <token>`). That's why the stream endpoint accepts `?token=` as a URL query parameter:

```
// From middleware/auth.js
} else if (req.query.token) {
  token = req.query.token;
}
```

And the URL is built with the token:

```
streamUrl: (id) => `/api/pdfs/${id}/stream?token=${Storage.getToken()}`
```

This is a minor security trade-off — the token appears in server logs and browser history. For this application it's acceptable. Production systems might use signed temporary URLs that

expire.

13.7 File Storage Security

Several security measures protect uploaded files:

1. **User ID in path**: `uploads/pdfs/{user-id}/{file.pdf}`. Even if you know the filename, the path requires the user ID to find it.
2. **Ownership check in the stream endpoint**: `WHERE id = $1 AND user_id = $2`. Without the right user ID in the JWT, you can't access any file.
3. **UUID filenames**: The on-disk filename is a random UUID, not the original name. You can't guess filenames.
4. **Static serving disabled**: The `uploads/` folder is NOT served as static files. You can't access `http://server.com/uploads/pdfs/user/file.pdf` directly — all access goes through the authenticated API endpoint.

13.8 Deleting PDFs

```
const deletePdf = async (req, res, next) => {
  try {
    const result = await query(
      'DELETE FROM pdfs WHERE id = $1 AND user_id = $2 RETURNING file_path',
      [req.params.id, req.user.id]
    );

    if (!result.rows.length) {
      return res.status(404).json({ success: false, error: 'PDF not found' });
    }

    // Delete the physical file
    const filePath = result.rows[0].file_path;
    try {
      if (fs.existsSync(filePath)) {
        fs.unlinkSync(filePath);
      }
    } catch (fileErr) {
      console.error('Failed to delete file:', fileErr.message);
      // Don't fail the request - DB record is deleted, file cleanup is best-effort
    }

    res.json({ success: true, message: 'PDF deleted' });
  } catch (err) { next(err); }
};
```

`RETURNING file_path` — we get the file path back from the DELETE so we know what to remove from disk.

The file deletion is in a separate try/catch. If the database delete succeeded but the file delete failed (permission error, file already gone), we still return success — the important thing is the database record is gone.

13.9 Key Takeaways

- Files are stored on disk; only metadata goes in the database
- Multer handles multipart uploads, file filtering, and filename generation
- UUID filenames prevent guessing and collisions
- PDF text is extracted on upload and stored in the database for AI access
- Large files are streamed to the browser (not buffered in memory)
- `<iframe>` renders PDFs natively — no additional libraries needed
- Token is passed as URL query parameter for iframe/download access
- All file access goes through authenticated API endpoints — no direct file serving
- Deleting a record also deletes the physical file (best-effort cleanup)

Next: the AI chat feature — the most complex part of the application.

Section 14 — The AI Chat Feature

Sessions, context injection, the LLM service, streaming, and auto-note creation

---

14.1 What the Chat Feature Does

The AI chat is Study-Hub's crown jewel. Users can: - Have multi-turn conversations with an AI assistant - Attach notes and PDFs as context so the AI can read them - Ask the AI to summarize, explain, or quiz them on their material - Have the AI automatically create and save notes to their library - Maintain multiple independent chat sessions (like different chat threads)

The AI is not a generic chatbot — it has access to the user's personal knowledge library.

14.2 How AI APIs Work

Study-Hub uses **OpenRouter**, a service that provides access to multiple AI language models. The interface is the **OpenAI-compatible chat completions API** — the same format used by OpenAI, Anthropic, and most modern AI services.

You send a list of messages (the conversation history) and receive the AI's next reply.

The request format:

```
{
  "model": "meta-llama/llama-3.3-70b-instruct:free",
  "messages": [
    { "role": "system", "content": "You are a study assistant..." },
    { "role": "user",   "content": "What is machine learning?" },
    { "role": "assistant", "content": "Machine learning is..." },
    { "role": "user",   "content": "Can you give me an example?" }
  ],
  "temperature": 0.7,
  "max_tokens": 2048
}
```

- `system` — instructions that define the AI's personality and capabilities. The user doesn't write this — you (the developer) do.
- `user` — messages from the human
- `assistant` — the AI's previous replies (needed so the AI has memory of the conversation)
- `temperature` — how creative/random vs. focused/deterministic the AI is (0.0 = very focused, 1.0 = very creative). 0.7 is a good balance for a study assistant.
- `max_tokens` — maximum length of the AI's response

The response contains the AI's reply in `choices[0].message.content`.

14.3 llmService.js — The Complete AI Service

The Model Fallback Chain

```
const MODELS = [
  'openrouter/free',
  'meta-llama/llama-3.3-70b-instruct:free',
  'google/gemma-3-12b-it:free',
];
```

Study-Hub tries multiple models in order. If the first fails (rate limit, model unavailable), it tries the second, then the third.

- `openrouter/free` — OpenRouter's auto-router picks whatever free model is available right now. Best starting point.
- `meta-llama/llama-3.3-70b-instruct:free` — Meta's Llama 3.3 70B (a large, capable model)
- `google/gemma-3-12b-it:free` — Google's Gemma 12B (smaller but still capable)

All three are free tiers. No credit card required.

The System Prompt

```
const SYSTEM_PROMPT = `You are Study-Hub AI, a smart study assistant integrated into a personal knowled

You have access to the user's notes and uploaded PDF documents. You can:
1. Answer questions based on stored notes and PDFs
2. Summarize notes or PDF content
3. Generate structured study notes from PDFs
4. Help organize and understand study materials
5. Answer general knowledge questions

When the user asks you to CREATE or SAVE notes, generate the note content between special tags:
[[CREATE_NOTE]]
# Note Title

Note content in markdown format...
[[/CREATE_NOTE]]

The system will automatically detect this and save the note to their library.

Always be concise, helpful, and academic in tone. Format responses with markdown when appropriate.
Use headings, bullet points, and code blocks where they aid clarity.

If you reference a PDF or note, mention it by name for transparency.`;
```

The `[[CREATE_NOTE]]` / `[[/CREATE_NOTE]]` tags are a **custom protocol** invented for Study-Hub. When the user asks "create a note about photosynthesis," the AI includes its response inside these tags. The server detects the tags, extracts the content, saves it as a real note, and strips the tags from the displayed response.

This is prompt engineering — carefully crafting the system instruction to make the AI behave in specific, predictable ways.

Calling the OpenRouter API

```
const callOpenRouter = async (apiKey, model, messages) => {
  const response = await fetch(OPENROUTER_API_URL, {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json',
      'Authorization': `Bearer ${apiKey}`,
      'HTTP-Referer': 'http://localhost:3000',   // OpenRouter requires this
      'X-Title': 'Study-Hub',                    // identifies your app
    },
    body: JSON.stringify({
      model,
      messages,
      temperature: 0.7,
      max_tokens: 2048,
    }),
    timeout: 30000,   // give up after 30 seconds
  });

  if (!response.ok) {
    const errorBody = await response.text();
    let errorMsg;
    try {
      errorMsg = JSON.parse(errorBody).error?.message || `HTTP ${response.status}`;
    } catch {
      errorMsg = `HTTP ${response.status}: ${errorBody.slice(0, 200)}`;
    }
    throw new Error(errorMsg);
  }

  const data = await response.json();
  if (!data.choices || data.choices.length === 0) {
    throw new Error('No response generated');
  }
  return data.choices[0].message?.content || '';
};
```

`HTTP-Referer` and `X-Title` are headers OpenRouter requires to identify your application. This helps them moderate usage.

Error handling is defensive: try to parse the error body as JSON (structured error from OpenRouter), fall back to raw text if parsing fails.

Building the Context String

```
const chat = async (history, context, user) => {
  const apiKey = process.env.OPENROUTER_API_KEY;
  if (!apiKey || apiKey === 'your_openrouter_api_key_here') {
    throw new Error('OpenRouter API key not configured...');
  }

  let contextStr = '';

  if (context.notes && context.notes.length > 0) {
    if (context.notes.length === 1) {
      // Single note: show it fully
      contextStr += `\n\n--- ATTACHED NOTE: "${context.notes[0].title || 'Untitled'}" ---\n`;
      contextStr += context.notes[0].content;
      contextStr += '\n--- END OF NOTE ---\n';
    } else {
      // Multiple notes: truncate each to 3000 chars to stay within token limits
      contextStr += "\n\n--- USER'S ATTACHED NOTES ---\n";
      context.notes.forEach(n => {
        contextStr += `\n**${n.title || 'Untitled'}:**\n${n.content.slice(0, 3000)}\n---\n`;
      });
    }
  }

  if (context.pdfs && context.pdfs.length > 0) {
    if (context.pdfs.length === 1) {
      const p = context.pdfs[0];
      contextStr += `\n\n--- ATTACHED PDF: "${p.original_name}" (${p.page_count} pages) ---\n`;
      // Limit to 15000 chars - a large PDF could be millions of characters
      contextStr += p.extracted_text ? p.extracted_text.slice(0, 15000) : '[No text extracted]';
      contextStr += '\n--- END OF PDF ---\n';
    } else {
      // Multiple PDFs: 5000 chars each
      contextStr += "\n\n--- USER'S ATTACHED PDFs ---\n";
      context.pdfs.forEach(p => {
        contextStr += `\n**${p.original_name}** (${p.page_count} pages):\n`;
        if (p.extracted_text) contextStr += p.extracted_text.slice(0, 5000) + '\n---\n';
      });
    }
  }

  // Append context to system prompt
  const systemWithContext = SYSTEM_PROMPT + (contextStr ? `\n\nCONTEXT FROM USER'S LIBRARY:${contextStr
```

Token limits are the key constraint. AI models have a maximum context window (how much text they can consider at once). A large PDF could contain hundreds of thousands of characters — far more than any model can handle. So we truncate: - Single note: full content (notes are usually small) - Multiple notes: 3,000 chars each - Single PDF: 15,000 chars - Multiple PDFs: 5,000 chars each

The truncation is crude (hard cutoff at a character count) rather than intelligent (summarizing to fit). For a more sophisticated system, you'd use RAG (Retrieval-Augmented Generation) — storing text in vector databases and retrieving only the most relevant chunks.

Building the Message Array

```
  const messages = [{ role: 'system', content: systemWithContext }];

  for (const msg of history) {
    if (msg.role === 'system') continue;  // don't include old system messages
    messages.push({ role: msg.role, content: msg.content });
  }
```

The messages array starts with the system prompt (which includes the context). Then the conversation history is appended (user and assistant messages in order). The last message in `history` is the user's new question.

The Fallback Loop

```
  let lastError;
  for (const model of MODELS) {
    try {
      console.log(`[LLM] Trying OpenRouter model: ${model}`);
      const text = await callOpenRouter(apiKey, model, messages);
      console.log(`[LLM] Success with ${model}`);
      return text;
    } catch (err) {
      console.warn(`[LLM] Failed with ${model}: ${err.message}`);
      lastError = err;
    }
  }
  throw new Error(`All models failed. Last error: ${lastError?.message}`);
};
```

If `openrouter/free` fails, try `llama-3.3-70b`, then `gemma-3-12b`. If all three fail, throw an error with the last failure's message.

14.4 chatController.js — The Send Message Handler

This is the most complex controller in the app. It orchestrates everything.

Parsing the Request

```
const sendMessage = async (req, res, next) => {
  try {
    const { sessionId } = req.params;
    const {
      content,
      context_note_ids,
      context_pdf_ids,
      // legacy fallback fields (backwards compat)
      contextType,
      contextRefId,
    } = req.body;

    if (!content || !content.trim()) {
      return res.status(400).json({ success: false, error: 'Message content is required' });
```

```
}

// Normalise to arrays
const noteIds = Array.isArray(context_note_ids) ? context_note_ids
  : (context_note_ids ? [context_note_ids] : []);
const pdfIds  = Array.isArray(context_pdf_ids)  ? context_pdf_ids
  : (context_pdf_ids  ? [context_pdf_ids]  : []);

// Legacy single-item support
if (noteIds.length === 0 && contextType === 'note' && contextRefId) noteIds.push(contextRefId);
if (pdfIds.length  === 0 && contextType === 'pdf'  && contextRefId) pdfIds.push(contextRefId);
```

The frontend sends `context_note_ids` as an array. The normalization handles edge cases: - If sent as a single string instead of array, wrap it in an array - If old `contextType`/`contextRefId` fields are used (backward compatibility), convert them

Saving the User Message

```
// Verify session ownership
const session = await query(
  'SELECT * FROM chat_sessions WHERE id=$1 AND user_id=$2',
  [sessionId, req.user.id]
);
if (!session.rows.length) {
  return res.status(404).json({ success: false, error: 'Session not found' });
}

// Save user message
const savedContextType  = noteIds.length ? 'note' : (pdfIds.length ? 'pdf' : null);
const savedContextRefId = noteIds[0] || pdfIds[0] || null;

const userMsg = await query(
  `INSERT INTO chat_messages (session_id, user_id, role, content, context_type, context_ref_id)
   VALUES ($1,$2,'user',$3,$4,$5) RETURNING *`,
  [sessionId, req.user.id, content.trim(), savedContextType, savedContextRefId]
);
```

The user's message is saved to the database before we even call the AI. This ensures it's recorded even if the AI call fails.

`context_type` and `context_ref_id` store a reference to what was attached (for display purposes later — "this message had a PDF attached").

Building Context

```
const context = await buildContext(req.user.id, content, noteIds, pdfIds);
```

The `buildContext` function is the context assembly engine:

```
async function buildContext(userId, userMessage, noteIds = [], pdfIds = []) {
  const context = { notes: [], pdfs: [] };
  const msg = userMessage.toLowerCase();

  //   Explicitly attached items
  if (noteIds.length > 0) {
    const placeholders = noteIds.map((_, i) => `$${i + 2}`).join(',');
```

```
    // If noteIds = ['id1','id2'], generates: "$2,$3"
    // Full query: WHERE user_id=$1 AND id IN ($2,$3)
    const result = await query(
      `SELECT title, content FROM notes WHERE user_id=$1 AND id IN (${placeholders})`,
      [userId, ...noteIds]
    );
    context.notes = result.rows;
  }

  if (pdfIds.length > 0) {
    const placeholders = pdfIds.map((_, i) => `$${i + 2}`).join(',');
    const result = await query(
      `SELECT original_name, extracted_text, page_count FROM pdfs WHERE user_id=$1 AND id IN (${placeho
      [userId, ...pdfIds]
    );
    context.pdfs = result.rows;
  }

  //   Keyword-based fallback
  // Only runs if NOTHING was explicitly attached
  const hasAttached = noteIds.length > 0 || pdfIds.length > 0;
  if (!hasAttached) {
    const wantsNotes = msg.includes('note') || msg.includes('notes');
    const wantsPdf   = msg.includes('pdf') || msg.includes('document') || msg.includes('summarize');

    if (wantsNotes) {
      const notes = await query(
        'SELECT title, content FROM notes WHERE user_id=$1 ORDER BY updated_at DESC LIMIT 5',
        [userId]
      );
      context.notes = notes.rows;
    }

    if (wantsPdf) {
      const pdfs = await query(
        'SELECT id, original_name, page_count, extracted_text FROM pdfs WHERE user_id=$1 ORDER BY creat
        [userId]
      );
      context.pdfs = pdfs.rows;
    }
  }

  return context;
}
```

Two modes: 1. **Explicit attachment** — user attached specific notes/PDFs → fetch exactly those 2. **Keyword fallback** — nothing attached, but message mentions "note" or "pdf" → auto-fetch recent items

The keyword fallback means "tell me about my notes" works even without explicitly attaching anything.

`IN ($2,$3)` with dynamic placeholders is the safe way to query multiple IDs. `...noteIds` spreads the array as individual arguments.

Fetching Conversation History

```
const history = await query(
  `SELECT role, content FROM chat_messages
   WHERE session_id=$1 ORDER BY created_at ASC LIMIT 20`,
  [sessionId]
);
```

The last 20 messages are sent to the AI as conversation history. This gives the AI memory of recent context. More messages = better memory but more tokens consumed (and higher cost).

Calling the AI

```
let aiResponse;
try {
  aiResponse = await llmService.chat(history.rows, context, req.user);
} catch (llmErr) {
  console.error('LLM error:', llmErr.message);
  aiResponse = `I encountered an error connecting to the AI service: ${llmErr.message}. Please checl
}
```

If the AI fails, we don't crash — we return a helpful error message as the AI's reply. The conversation still works; the user just sees an error in the chat bubble.

Auto-Note Creation Detection

```
let savedNote = null;
if (aiResponse.includes('[[CREATE_NOTE]]')) {
  const noteMatch = aiResponse.match(/\[\[CREATE_NOTE\]\]([\s\S]*?)\[\[\/CREATE_NOTE\]\]/);
  if (noteMatch) {
    const noteContent = noteMatch[1].trim();

    // Extract title from the first markdown heading
    const titleMatch = noteContent.match(/^#\s+(.+)/m);
    const noteTitle = titleMatch ? titleMatch[1] : 'AI Generated Note';

    try {
      const noteResult = await query(
        'INSERT INTO notes (user_id, title, content, color) VALUES ($1,$2,$3,$4) RETURNING *',
        [req.user.id, noteTitle, noteContent, '#1a2035']
      );
      savedNote = noteResult.rows[0];

      // Strip the CREATE_NOTE tags from the displayed response
      aiResponse = aiResponse
        .replace(/\[\[CREATE_NOTE\]\][\s\S]*?\[\[\/CREATE_NOTE\]\]/, '')
        .trim();
      aiResponse += '\n\n  **Note saved to your Notes library!**';
    } catch (e) {
      console.error('Failed to save AI note:', e.message);
    }
  }
}
```

The regex `/\[\[CREATE_NOTE\]\]([\s\S]*?)\[\[/CREATE_NOTE\]\]/`: - `\[\[CREATE_NOTE\]\]` — literal `[[CREATE_NOTE]]` (brackets are special in regex, so they're escaped with `\`) - `([\s\S]*?)` — capture everything between the tags, including newlines (`[\s\S]` = any character, `*?` = lazy/minimal match) - `\[\[/CREATE_NOTE\]\]` — literal `[[/CREATE_NOTE]]`

If found: 1. Extract the note content (everything between the tags) 2. Extract the title from the first `# Heading` in the markdown 3. Insert it as a new note in the database with the AI's dark blue color (`#1a2035`) 4. Strip the tags from the response so they don't show in the chat UI 5. Append a success message

Auto-titling Sessions

```
    if (session.rows[0].title === 'New Chat') {
      const autoTitle = content.trim().slice(0, 60) + (content.length > 60 ? '...' : '');
      await query('UPDATE chat_sessions SET title=$1 WHERE id=$2', [autoTitle, sessionId]);
    }
```

If the session is still named "New Chat" (the default), auto-title it with the first message (truncated to 60 characters). This makes the session list readable.

14.5 Frontend: chat.js — The Chat Interface

State

```
let state = {
  sessions:       [],
  activeSession:  null,
  messages:       [],
  isStreaming:    false,   // prevent sending while waiting for AI
  context: {
    noteIds: [],   // IDs of attached notes
    pdfIds:  [],   // IDs of attached PDFs
    notes:   [],   // full note objects (for display)
    pdfs:    [],   // full pdf objects (for display)
  },
};
```

Session Management

```
const loadSessions = async () => {
  const data = await API.chat.getSessions();
  state.sessions = data.sessions || [];
  renderSessionList();

  // Restore last active session from localStorage
  const savedId = Storage.getActiveChatSession();
  if (savedId) {
    const found = state.sessions.find(s => String(s.id) === String(savedId));
    if (found) await activateSession(found, false);
  }
};
```

`String(s.id) === String(savedId)` — converts both to strings before comparing. UUIDs from the database are strings, but stored IDs from localStorage are parsed with `JSON.parse`, which might give a number in some edge cases. Defensive comparison.

```
const activateSession = async (session, scroll = true) => {
  state.activeSession = session;
  Storage.setActiveChatSession(session.id);
  renderSessionList();  // update active state in sidebar
  showMessagesView();   // switch to messages view
  await loadMessages(session.id, scroll);
};
```

Sending a Message

```
const sendMessage = async () => {
  const content = input?.value.trim();
  if (!content || state.isStreaming) return;

  // Auto-create session if none exists
  if (!state.activeSession) {
    await createSession(content.slice(0, 40) || 'New Chat');
    if (!state.activeSession) return;
  }

  input.value = '';
  input.style.height = 'auto';
  state.isStreaming = true;
  sendBtn.disabled = true;

  // Show user message immediately (optimistic)
  appendMessage({ role: 'user', content, created_at: new Date().toISOString() });

  // Show typing indicator (animated skeleton bubbles)
  const typingEl = showTypingIndicator();

  try {
    const response = await API.chat.sendMessage(state.activeSession.id, {
      content,
      context_note_ids: state.context.noteIds,
      context_pdf_ids:  state.context.pdfIds,
    });

    removeTypingIndicator(typingEl);

    const aiContent = response.assistantMessage?.content || 'No response.';
    appendMessage({
      role: 'assistant',
      content: aiContent,
      created_at: new Date().toISOString(),
    });

  } catch (err) {
    removeTypingIndicator(typingEl);
    appendMessage({
```

```
      role: 'assistant',
      content: ` Error: ${err.message}`,
      created_at: new Date().toISOString(),
    });
    toast.error('Message failed: ' + err.message);
  } finally {
    state.isStreaming = false;
    sendBtn.disabled = false;
    input.focus();
  }
};
```

Key design choices:

Optimistic message display — the user's message appears in the chat immediately, before the server confirms it. This feels instant. If sending failed, the error bubble would follow.

`isStreaming` **flag** — prevents double-sending. If the user presses Enter twice, the second press is ignored while waiting for the first response.

`finally` **block** — `state.isStreaming = false` and `sendBtn.disabled = false` run whether the send succeeded or failed. This prevents the UI from getting stuck in a disabled state.

The Typing Indicator

```
const showTypingIndicator = () => {
  const el = document.createElement('div');
  el.className = 'message-wrap assistant typing-wrap';
  el.id = 'typing-indicator';
  el.innerHTML = `
    <div class="message-bubble ai-bubble shimmer-bubble">
      <div class="shimmer-line shimmer-line--long"></div>
      <div class="shimmer-line shimmer-line--mid"></div>
      <div class="shimmer-line shimmer-line--short"></div>
    </div>
  `;
  messagesContainer?.appendChild(el);
  scrollToBottom();
  return el;
};
```

The shimmer lines are CSS-animated "skeleton" lines — they pulse with a shimmer animation while the AI is thinking. Much better UX than a plain "..." text.

Rendering Messages

```
const buildMessageBubble = (msg) => {
  const isUser = msg.role === 'user';
  const wrap = document.createElement('div');
  wrap.className = `message-wrap ${isUser ? 'user' : 'assistant'}`;

  const contentHtml = isUser
    ? `<p>${escHtml(msg.content)}</p>`          // user: escape plain text
    : renderMarkdown(msg.content);               // AI: render markdown

  wrap.innerHTML = `
```

```
    <div class="message-bubble ${isUser ? 'user-bubble' : 'ai-bubble'}">
      ${!isUser ? '<span class="ai-label">Study-Hub AI</span>' : ''}
      <div class="message-content">${contentHtml}</div>
      <span class="message-time">${msg.created_at ? formatDate(msg.created_at) : ''}</span>
    </div>
  `;
  return wrap;
};
```

User messages: escaped as plain text (they wrote it, so no markdown rendering needed, and XSS prevention still applies).

AI messages: rendered as markdown (the AI responds with markdown headings, bullet points, code blocks).

Context Attachment Panel

```
const openAttachMenu = async () => {
  const menu = el('chat-attach-menu');
  const isOpen = !menu.classList.contains('hidden');
  if (isOpen) { hide(menu); return; }  // toggle off if already open

  await populateAttachLists();  // load notes and PDFs
  show(menu);

  // Close when clicking outside
  setTimeout(() => {
    const closeMenu = (e) => {
      if (!menu.contains(e.target) && e.target !== el('chat-attach-btn')) {
        hide(menu);
        document.removeEventListener('click', closeMenu);
      }
    };
    document.addEventListener('click', closeMenu);
  }, 10);  // setTimeout(fn, 10) to avoid the current click triggering closeMenu
};
```

The `setTimeout(fn, 10)` trick: the click that opened the menu would immediately trigger `closeMenu` if the listener were added synchronously. Adding it after 10ms (next event loop tick) lets the opening click finish first.

```
const toggleContextItem = (type, checkbox) => {
  const id = checkbox.value;  // UUID string
  if (type === 'note') {
    if (checkbox.checked) {
      if (!state.context.noteIds.includes(id)) state.context.noteIds.push(id);
    } else {
      state.context.noteIds = state.context.noteIds.filter(x => x !== id);
    }
  } else {
    // same for pdfIds
  }
  updateContextBadge();
};
```

The comment `// IDs are UUIDs - never parseInt them` is a reminder that UUIDs like

"ac2c0621-89c1" would become NaN if parsed as integers, breaking the ID comparison.

Context Badge

```
const updateContextBadge = () => {
  const badge = el('chat-context-badge');
  const label = el('chat-context-label');
  const total = state.context.noteIds.length + state.context.pdfIds.length;

  if (total === 0) { hide(badge); return; }

  show(badge);
  const parts = [];
  if (state.context.noteIds.length)
    parts.push(`${state.context.noteIds.length} note${state.context.noteIds.length !== 1 ? 's' : ''}`);
  if (state.context.pdfIds.length)
    parts.push(`${state.context.pdfIds.length} PDF${state.context.pdfIds.length !== 1 ? 's' : ''}`);
  if (label) label.textContent = '  ' + parts.join(', ');
  // Result examples: "  2 notes, 1 PDF" or "  1 note"
};
```

Plural handling: `note${count !== 1 ? 's' : ''}` — adds "s" unless count is exactly 1. "1 note" not "1 notes".

Keyboard Shortcut

```
input.addEventListener('keydown', (e) => {
  if (e.key === 'Enter' && !e.shiftKey) {
    e.preventDefault();   // don't insert a newline
    if (input.value.trim() && !state.isStreaming) sendMessage();
  }
  // Shift+Enter: e.shiftKey is true → don't intercept → textarea gets a newline
});
```

Enter sends the message. Shift+Enter adds a newline (for multi-paragraph messages). This is the standard messaging app convention.

Cross-Module Communication (PDF → Chat)

When a user clicks "Ask AI" on a PDF in the PDF viewer, it needs to switch to the Chat view and attach that PDF. But `pdfs.js` and `chat.js` don't import each other — they're separate IIFEs.

The solution: custom events.

In `pdfs.js`:

```
const askAI = (pdf) => {
  window.dispatchEvent(new CustomEvent('chat:attach-pdf', { detail: { pdf } }));
  // Switch to chat view
  window.dispatchEvent(new CustomEvent('nav:switch', { detail: { view: 'chat' } }));
};
```

In `chat.js`:

```
window.addEventListener('chat:attach-pdf', (e) => {
  attachPDF(e.detail.pdf);
});
```

In `app.js`:

```
window.addEventListener('nav:switch', (e) => {
  if (e.detail?.view) switchView(e.detail.view);
});
```

Three modules communicate without knowing about each other — decoupled, clean, maintainable.

14.6 The Complete Flow for Sending a Message with Context

```
1. User attaches "Lecture 3.pdf" via the   menu
   → state.context.pdfIds = ['pdf-uuid']
   → context badge shows "  1 PDF"

2. User types "Summarize the key points" and presses Enter

3. sendMessage() runs:
   - User message appears in chat (optimistic)
   - Typing indicator appears
   - POST /api/chat/sessions/{id}/messages
     { content: "Summarize...", context_pdf_ids: ['pdf-uuid'] }

4. Server (chatController.sendMessage):
   - Saves user message to DB
   - buildContext() fetches PDF with id 'pdf-uuid' from DB
     → gets extracted_text (the full text of the PDF)
   - Gets last 20 messages from DB
   - Builds messages array:
     [system + PDF context, ...history, user message]
   - Calls llmService.chat()

5. llmService.chat():
   - Tries openrouter/free model
   - Sends messages to OpenRouter API
   - Receives AI response text
   - Returns response text

6. Server:
   - Checks for [[CREATE_NOTE]] tags (none found)
   - Saves AI message to DB
   - Updates session title if first message
   - Returns { userMessage, assistantMessage }

7. Frontend:
   - Removes typing indicator
   - Renders AI response as markdown
   - Scrolls to bottom
   - Re-enables send button
```

14.7 Key Takeaways

- AI chat uses the OpenAI-compatible chat completions API format via OpenRouter
- The system prompt defines the AI's personality and the `[[CREATE_NOTE]]` protocol
- Three models are tried in order — if one fails, fall back to the next
- Context is built from explicitly attached notes/PDFs, with keyword-based auto-attach as fallback
- Content is truncated before sending to avoid exceeding the AI's token limit
- The `[[CREATE_NOTE]]` detection parses the AI response and saves notes automatically
- The typing indicator (shimmer skeleton) gives feedback while the AI is thinking
- `isStreaming` flag prevents double-sends
- Modules communicate via custom browser events — no direct dependencies on each other
- The entire conversation history (last 20 messages) is sent with every request for AI memory

Section 15 — System Design

Architecture diagrams, data flow, and the thinking behind every major decision

---

15.1 The Big Picture Architecture

```
                    USER'S BROWSER


                frontend/index.html


storage      helpers        api        app.js
  .js          .js          .js


 auth.js     notes.js     chat.js    pdfs.js


           HTTP/JSON (fetch API)

                        port 3000

               SERVER (Node.js)


                 server.js
        helmet · cors · rate-limit ·
        morgan · body-parser · static
```

```
                    routes/index.js

        auth     notes      pdfs       chat

    authController  notes    pdf      chat
                   Ctrl     Ctrl    Controller



                                 llmService.js
                              (OpenRouter API call)



      database.js                uploads/
      (pg Pool)                  pdfs/


              TCP port 5432

                        POSTGRESQL

      users   note_folders   notes   pdf_folders   pdfs
              chat_sessions   chat_messages



         OpenRouter API
         (external service)
         Llama / Gemma / etc.
```

15.2 Request Lifecycle — The Full Journey

Let's trace every step of a request from "user presses Enter to send a chat message" to "AI response appears on screen."

Step 1: User presses Enter in the chat input

```
chat.js: input keydown event fires
→ sendMessage() called
→ state.isStreaming = true (prevents re-send)
→ User message appended to DOM immediately (optimistic)
→ Typing indicator shown
→ API.chat.sendMessage(sessionId, body) called
```

Step 2: Frontend sends HTTP request

```
api.js: request() function called
```

```
→ Storage.getToken() reads JWT from localStorage
→ Headers built: { Content-Type: application/json, Authorization: Bearer eyJ... }
→ body: JSON.stringify({ content, context_note_ids, context_pdf_ids })
→ fetch('http://localhost:3000/api/chat/sessions/{id}/messages', config)
→ Request travels over localhost TCP connection to port 3000
```

Step 3: Express middleware pipeline

```
server.js: request received on port 3000
→ Helmet: adds security headers to the (eventual) response
→ CORS: checks Origin header, adds CORS headers
→ Rate limiter: checks if this IP has exceeded 300 req/min
→ JSON body parser: parses request body string → req.body object
→ Morgan: logs "POST /api/chat/sessions/... 200 342ms"
→ app.use('/api', routes): hands off to routes
```

Step 4: Route matching

```
routes/index.js:
POST /chat/sessions/:sessionId/messages
→ authenticate middleware runs:
   - Reads Authorization header
   - jwt.verify(token, JWT_SECRET) checks signature and expiry
   - Queries DB: SELECT * FROM users WHERE id = decoded.userId
   - req.user = user object
   - next() called
→ chatController.sendMessage runs
```

Step 5: Controller processes request

```
chatController.sendMessage:
→ Parse body: content, context_note_ids, context_pdf_ids
→ Query DB: verify session exists and belongs to req.user.id
→ INSERT user message into chat_messages
→ buildContext():
   - If noteIds provided: SELECT notes WHERE id IN (...)
   - If pdfIds provided: SELECT pdfs WHERE id IN (...)
→ SELECT last 20 messages from this session
→ llmService.chat(history, context, user)
```

Step 6: AI service builds and sends request

```
llmService.js:
→ Build context string from notes/PDFs
→ Append to SYSTEM_PROMPT
→ Build messages array: [system, ...history]
→ Try model 'openrouter/free':
   fetch('https://openrouter.ai/api/v1/chat/completions', {
     body: { model, messages, temperature: 0.7, max_tokens: 2048 }
   })
→ OpenRouter forwards to AI model
→ AI model generates response (takes 2-20 seconds)
→ OpenRouter returns response JSON
```

```
→ Extract text from choices[0].message.content
→ Return text to chatController
```

Step 7: Controller saves and responds

```
chatController.sendMessage:
→ Check aiResponse for [[CREATE_NOTE]] tags
   - If found: INSERT new note, strip tags, append "  Note saved"
→ INSERT aiResponse into chat_messages
→ If session.title === 'New Chat': UPDATE session title
→ res.json({ success, userMessage, assistantMessage, savedNote })
```

Step 8: Response travels back to browser

```
Express sends HTTP 200 with JSON body
→ fetch() in api.js receives response
→ res.json() parses JSON body
→ Returns data to chat.js
```

Step 9: Frontend updates UI

```
chat.js:
→ removeTypingIndicator()
→ buildMessageBubble(aiMessage)
→ renderMarkdown(msg.content) → HTML
→ appendMessage() → DOM updated
→ scrollToBottom()
→ state.isStreaming = false
→ sendBtn.disabled = false
→ input.focus()
```

Total time: typically 3–15 seconds (AI inference is the bottleneck).

15.3 Data Flow Diagram — Notes Feature

```
User types in editor textarea
        ↓
input event fires
        ↓
debouncedSave() (1 second delay)
        ↓  (only if no new keystroke for 1s)
API.notes.update(id, { title, content })
        ↓
PUT /api/notes/:id
        ↓
authenticate middleware → req.user
        ↓
notesController.updateNote
        ↓
Dynamic SQL:
UPDATE notes SET title=$1, content=$2
WHERE id=$3 AND user_id=$4
```

```
RETURNING *
        ↓
PostgreSQL executes query
Auto-trigger fires: updated_at = NOW()
        ↓
Returns updated note row
        ↓
{ success: true, note: {...} }
        ↓
state.notes[idx] = updated.note
        ↓
renderNotes() → DOM updated
```

15.4 Major Design Decisions and Why

Decision 1: Single-Page Application

Why not multiple HTML pages?

Multiple pages would require a full page reload on every navigation. For an application where users switch frequently between Notes, PDFs, and Chat, this would feel slow and jarring. A SPA keeps the UI state (sidebar, theme, scroll position, open note) consistent across navigation.

Trade-off: More complex JavaScript, initial load of all code.

Why not a framework (React, Vue, etc.)?

Study-Hub uses vanilla JavaScript for educational clarity — it's easier to understand what's happening without framework abstractions. A production app with a team would benefit from React's component system.

Decision 2: JWT over Sessions

Alternative: Server-side sessions (store session data in the database, give the user a cookie with a session ID).

Why JWT instead? - **Stateless**: The server doesn't need to look up a session in the database to authenticate a request — just verify the signature. This scales better. - **Works across servers**: With sessions, all servers need to share a database. JWTs are self-contained. - **Mobile-friendly**: JWTs work naturally with HTTP headers; cookies can have mobile/cross-origin complications.

Trade-off: JWTs can't be invalidated before expiry (unless you maintain a blacklist). 7-day expiry is a reasonable compromise.

Decision 3: Files on Disk, Metadata in Database

Alternative: Store file bytes in the database (as `BYTEA` columns).

Why disk instead? - Databases are optimized for structured queries, not streaming binary data - Large files in a database make backups slow and huge - Streaming from disk is faster and uses less memory (`createReadStream().pipe(res)`) - Cloud deployments (like Render.com) often have ephemeral storage — but that's a trade-off accepted here for simplicity

Trade-off: On Render.com's free tier, the disk resets between deploys. For true persistence, you'd use S3 or similar. Study-Hub notes this as a limitation.

Decision 4: Static Serving from the Same Server

Alternative: Host frontend files on a CDN (like Cloudflare, AWS S3, Netlify).

Why same server? - Simpler deployment — one thing to deploy, one URL, one process - No CORS configuration between separate frontend/backend domains - Easy to get started

Trade-off: Less efficient — the Node.js server handles both API requests and file serving. A CDN would be faster for serving static assets and would offload that work from the API server.

Decision 5: `[[CREATE_NOTE]]` Tags for Auto-Note Creation

Alternative: Add a "save as note" button that appears after the AI responds.

Why tags instead? - The AI can proactively decide to create a note ("here's a summary, I'll save it for you") - It's visible in the response before being stripped — users can see what's being saved - No additional UI elements needed — the AI's response IS the UI - Enables a conversational workflow: "Create a study guide for Chapter 3" → AI generates and saves it in one step

Trade-off: The AI must be reliably prompted to use the tags. If it forgets, no note is created. Prompt engineering is required.

Decision 6: Debounced Auto-Save for Notes

Alternative: Explicit "Save" button.

Why auto-save? - Users never lose work by forgetting to save - It feels modern (like Google Docs) - One less thing to think about while writing

Why debounce instead of saving every keystroke? - Saving every keystroke would generate hundreds of database writes per minute - 1-second debounce: save happens when the user stops typing, not while typing

Trade-off: If the browser crashes in the middle of typing, up to 1 second of work can be lost. Acceptable for this use case.

Decision 7: Full-Text Search via PostgreSQL

Alternative: Client-side search (filter in JavaScript) or Elasticsearch.

Why PostgreSQL FTS? - Already have PostgreSQL — no additional service needed - PostgreSQL's FTS is surprisingly capable: stemming ("running" matches "run"), ranking, multiple languages - Works for thousands of notes without performance issues - Client-side search would require loading ALL notes to the browser — not scalable

Trade-off: PostgreSQL FTS doesn't support fuzzy matching (typos). For production with millions of records, Elasticsearch or Meilisearch would be better.

Decision 8: Multiple AI Models with Fallback

Why not just one model?

Free API tiers have rate limits. If `openrouter/free` is rate-limited, the app still works by falling back to a specific model. This improves reliability significantly.

Why OpenRouter instead of OpenAI directly? - Free tier with multiple capable models - The OpenAI-compatible format means you can switch to any provider later - Single integration, many models

---

15.5 Security Model

Study-Hub implements **defense in depth** — multiple layers of security that each protect against different threats.

Layer	Protection	Against
Helmet CSP	Restricts what scripts/styles can load	XSS, code injection
CORS	Restricts which origins can call the API	Cross-site request forgery
Rate limiting	Limits requests per IP	Brute force, DoS
bcrypt (work factor 12)	Slow password hashing	Brute force against stolen DB
JWT	Stateless authentication	Session hijacking
Parameterized SQL	SQL parameters, never interpolated	SQL injection
`escHtml()` in frontend	Escapes user content in DOM	XSS from stored content
Ownership checks	`WHERE user_id = req.user.id`	Unauthorized access to others' data
UUID filenames	Random disk filenames	Guessing file paths
No direct file serving	All file access through API	Bypassing authentication

No single security measure is sufficient. The attacker who gets past one layer faces the next.

15.6 Performance Considerations

Database indexes

Every frequently-queried column has an index: - `notes(user_id)` — all note queries filter by user - `notes(folder_id)` — folder filtering - `notes_fts_idx` — full-text search - `pdfs(user_id)` — same pattern for PDFs - `chat_messages(session_id)` — loading messages for a session

Without indexes, every query would be a full table scan — fine for 100 rows, catastrophic for 100,000.

Connection pooling

`max: 20` connections in the pool. Opening a database connection takes ~50ms. With pooling, connections are reused — requests wait for a connection from the pool instead of creating a new one each time.

Streaming file responses

Large PDF files are streamed from disk directly to the HTTP response rather than loaded entirely into memory first. Memory usage stays constant regardless of file size.

Debounced saves

Auto-save on notes fires at most once per second, not on every keystroke. A fast typist might type 10 characters per second — debouncing reduces database writes by 90%.

`Promise.all()` for parallel loading

```
await Promise.all([loadFolders(), loadNotes()]);
```

Both queries run simultaneously. If each takes 50ms, sequential would take 100ms — parallel takes 50ms.

15.7 The Data Model — Relationship Summary

```
users (1)
    note_folders (many)       notes (many)

                                        (folder_id can be NULL)

    notes (many, directly - no folder)

    pdf_folders (many)      pdfs (many)

    pdfs (many, directly)

    chat_sessions (many)
            chat_messages (many)
```

Every record belongs to a user. The ownership check (`AND user_id = $n`) on every query ensures strict data isolation.

Folders are optional — notes and PDFs can exist without a folder (they show as "Uncategorized"). When a folder is deleted, its contents become uncategorized (`ON DELETE SET NULL` on the folder foreign key).

15.8 Key Takeaways

- The architecture is classic three-tier: browser (presentation) → server (logic) → database (data)
- Every request goes through the middleware pipeline before reaching a controller
- Design decisions involve trade-offs: simplicity vs. scalability, one service vs. many
- Security is layered — many overlapping protections rather than one big lock
- Performance improvements (indexes, connection pooling, debounce) compound significantly at scale
- The data model enforces ownership at the database level with foreign keys and ownership checks

Section 16 — Deployment

Putting Study-Hub on the internet with Render.com

16.1 What Deployment Means

Deployment is the process of taking your application from your local machine and putting it on a server that's always on and accessible from anywhere in the world.

Right now, Study-Hub runs on `http://localhost:3000` — only visible to your own computer. After deployment, it will have a public URL like `https://study-hub.onrender.com` that anyone with the URL can access.

16.2 What Changes in Production

Several things are different in a production deployment:

	Development (your computer)	Production (Render.com)
PORT	3000	Set by Render (usually 10000)
Database	Local PostgreSQL	Render-managed PostgreSQL
NODE_ENV	`development`	`production`
Secrets	In `.env` file	In Render's environment variables UI
Logging	Verbose (morgan `dev`)	Combined format
File uploads	Saved locally	Saved on Render's disk (ephemeral!)

16.3 Setting Up Git

Render deploys from a Git repository. You need your code in Git first.

Initialize Git in your project

```
cd study-hub
git init
git add .
git commit -m "Initial commit: Study-Hub application"
```

`git init` — creates a new Git repository in the current folder.

`git add .` — stages all files for commit. The `.` means "everything" (respecting `.gitignore`).

`git commit -m "message"` — saves a snapshot with a description. The `-m` flag provides the message inline.

Push to GitHub

Create a free account at **https://github.com** if you don't have one.

Create a new repository (click the + icon → New repository). Name it `study-hub`. Don't initialize with README (you already have files).

GitHub will show you commands. Run them:

```
git remote add origin https://github.com/YOUR_USERNAME/study-hub.git
git branch -M main
git push -u origin main
```

`git remote add origin URL` — tells Git where the remote repository is.

`git branch -M main` — renames your branch to `main` (modern convention).

`git push -u origin main` — uploads your code to GitHub. The `-u` sets the "upstream" so future `git push` commands don't need the full arguments.

16.4 Deploying to Render.com

Render.com is a cloud platform that hosts web applications. They have a generous free tier that's perfect for projects like Study-Hub.

Step 1: Create a Render account

Go to **https://render.com** and sign up (you can sign up with GitHub — recommended, as it simplifies connecting repositories).

Step 2: Create a PostgreSQL database

1. In Render's dashboard, click **New** → **PostgreSQL**
2. Name it `study-hub-db`
3. Choose the **Free** plan
4. Click **Create Database**
5. Wait for it to provision (1-2 minutes)
6. Once ready, copy the **Internal Database URL** (you'll need it, though Render can pass it automatically)

Step 3: Deploy the web service using render.yaml

Study-Hub includes a `render.yaml` file that tells Render exactly how to deploy. This is **Infrastructure as Code** — the deployment config lives in your repository.

```yaml
services:
  - type: web
    name: study-hub
    runtime: node
    buildCommand: npm install
    startCommand: node backend/server.js
    envVars:
      - key: NODE_ENV
        value: production
      - key: DATABASE_URL
```

```yaml
      fromDatabase:
        name: study-hub-db
        property: connectionString
    - key: JWT_SECRET
      generateValue: true   # Render generates a random secret
    - key: PORT
      value: 10000

databases:
  - name: study-hub-db
    plan: free
```

Breaking this down: `- buildCommand: npm install` — run this first to install dependencies `- startCommand: node backend/server.js` — run this to start the server `- DATABASE_URL` with `fromDatabase` — Render automatically connects the database URL to the app. You don't type the connection string. `- JWT_SECRET` with `generateValue: true` — Render generates a cryptographically random secret. You never see it — it's injected automatically.

Step 4: Create the service

1. In Render's dashboard, click **New** → **Blueprint** (this reads your `render.yaml`)
2. Connect your GitHub repository
3. Render will read `render.yaml` and show you what it's about to create
4. Click **Apply** (or **Create Resources**)
5. Render starts building and deploying

What Render does automatically:

1. Clones your GitHub repository
2. Runs `npm install` to install dependencies
3. Sets environment variables (including the generated JWT_SECRET)
4. Runs `node backend/server.js`
5. Your app is now live at `https://study-hub-XXXX.onrender.com`

16.5 Environment Variables in Production

You still need to add some variables manually that aren't in `render.yaml`:

1. In Render dashboard, go to your web service
2. Click **Environment** in the left sidebar
3. Add these:

Key	Value
`OPENROUTER_API_KEY`	Your OpenRouter API key (from openrouter.ai)
`ALLOWED_ORIGINS`	`https://your-app-name.onrender.com`
`MAX_FILE_SIZE_MB`	`50`

Why not put these in `render.yaml`? Because `render.yaml` is in your Git repository — secrets committed to Git can be leaked. Environment variables added through the dashboard are stored securely.

16.6 The Deployment Pipeline

After initial setup, every future deployment is automatic:

```
You push code to GitHub:
  git add .
  git commit -m "Add feature X"
  git push

Render detects the push → automatically:
  1. Pulls the new code
  2. Runs npm install (if package.json changed)
  3. Starts new instance with new code
  4. Once healthy, switches traffic to new instance
  5. Stops old instance

Zero downtime deployment
```

This is called **Continuous Deployment (CD)** — code changes automatically flow to production.

16.7 The Free Tier Limitations

Render's free tier has constraints to be aware of:

Web service sleeps after 15 minutes of inactivity. The first request after inactivity takes 30-60 seconds while Render "wakes" the service. Subsequent requests are fast. There are free services like UptimeRobot that can ping your app every 5 minutes to keep it awake.

Free PostgreSQL databases expire after 90 days. Render deletes free databases after 90 days. Upgrade to a paid tier ($7/month) for persistence. Alternatively, export your data regularly.

Ephemeral disk. Files saved to disk (uploaded PDFs) are deleted when the service restarts or re-deploys. For production use with PDFs, you'd need to store files in an object storage service like **Amazon S3** or **Cloudflare R2**.

16.8 Checking Your Deployment

Once deployed:

Health check: Visit `https://your-app.onrender.com/api/health` — should return:

```
{ "success": true, "status": "ok", "timestamp": "..." }
```

View logs: In Render dashboard → your web service → **Logs**. You'll see Morgan's request logs and any console.log output.

Check the database: In Render dashboard → your database → **Connect** → **PSQL Command**. This opens a terminal connected to your production database. You can run SQL queries to verify data.

16.9 Custom Domain (Optional)

Render gives you a `.onrender.com` subdomain for free. For a custom domain like `studyhub.yourdomain.com`:

1. Buy a domain from Namecheap, Google Domains, etc.
2. In Render → your web service → **Custom Domains**
3. Add your domain
4. Render shows you a CNAME record to add to your domain's DNS
5. Add the CNAME in your domain registrar's DNS settings
6. Wait 5-60 minutes for DNS propagation

16.10 Making Your First Code Change After Deploy

Here's the workflow for updating the live app:

```
# Make a change (e.g., fix a typo in server.js)
# Edit the file in VS Code

# Stage the change
git add backend/server.js

# Commit
git commit -m "Fix typo in error message"

# Push to GitHub - this triggers Render to redeploy
git push
```

Render typically deploys in 1-3 minutes. You can watch the build progress in the Render dashboard.

16.11 Key Takeaways

- Deployment puts your app on a public server accessible from anywhere
- Code must be in Git (GitHub) before you can deploy via Render
- `render.yaml` defines the entire deployment config as code
- Environment variables (secrets, API keys) are set through Render's dashboard, never in code
- Render auto-deploys whenever you push to GitHub
- The free tier has limitations: sleeping service, 90-day database expiry, ephemeral disk
- For production PDF storage, use S3/R2 instead of local disk
- Check logs in the Render dashboard when things go wrong

Next: the glossary — every technical term in plain English.

Section 17 — Glossary

Every technical word used in this tutorial, explained in plain English

---

API (Application Programming Interface) A set of rules for how software components communicate. In web development, an API usually means a set of HTTP endpoints a server exposes

so other software can send and receive data. Study-Hub's API is the collection of `/api/...` routes.

`async` / `await` JavaScript syntax for working with asynchronous operations (things that take time, like database queries). `async` marks a function as asynchronous. `await` pauses execution inside that function until the awaited operation completes.

Authentication Proving you are who you say you are. Study-Hub uses email + password to authenticate, then issues a JWT token.

Authorization Proving you have permission to do something specific. After login (authentication), Study-Hub checks `WHERE user_id = req.user.id` on every query to ensure users can only access their own data.

Base64 A way of encoding binary data (like an image or a JWT) as ASCII text. JWTs are base64-encoded. Base64 is not encryption — it's just encoding.

bcrypt A password hashing function designed to be computationally slow (making brute-force attacks impractical). The work factor controls how slow.

Body parser Express middleware that reads the raw request body and parses it (e.g., JSON string → JavaScript object). Without it, `req.body` is undefined.

Boolean A value that is either `true` or `false`. Named after mathematician George Boole.

Brute force attack An attack that tries every possible combination until one works. Rate limiting and bcrypt's slowness defend against this.

Buffer In Node.js, a fixed-size chunk of binary data (bytes). Used when reading files or receiving binary network data.

Callback A function passed as an argument to another function, to be called when something happens. Example: `setTimeout(callback, 1000)` calls `callback` after 1 second. Older pattern largely replaced by async/await.

CDN (Content Delivery Network) A network of servers distributed globally that cache and serve files close to users. Study-Hub loads fonts from Google's CDN and some libraries from cdnjs.

Cascade (SQL) `ON DELETE CASCADE` — when a parent row is deleted, automatically delete all child rows that reference it. Example: deleting a user cascades to delete all their notes.

Character encoding How text is represented as binary data. UTF-8 can represent every character in human language. Always use UTF-8.

Class (CSS) A reusable style label applied to HTML elements with `class="..."`. Multiple elements can share a class. In CSS, targeted with `.classname`.

CORS (Cross-Origin Resource Sharing) A browser security mechanism that controls which websites can make requests to which other websites. Without CORS headers, browsers block cross-origin requests.

Client In web development, the browser (and the code running in it). The "client side" is everything in the browser.

Constraint (SQL) A rule enforced by the database. `NOT NULL`, `UNIQUE`, `FOREIGN KEY`, `CHECK` are constraints. They prevent invalid data from being stored.

Controller A module that handles incoming requests and coordinates the response. In Study-Hub: `authController.js`, `notesController.js`, etc.

Cookie A small piece of data the server sends to the browser, which the browser stores and sends back with every subsequent request. Study-Hub uses JWTs in headers instead of cookies.

CSS (Cascading Style Sheets) The language that describes how HTML elements should look: colors, fonts, layout, animations.

Custom event A browser event you create yourself (not a built-in like `click`). Study-Hub uses `new CustomEvent('auth:expired')` to notify app.js when a session expires.

Data attribute An HTML attribute starting with `data-` that stores custom information. `data-view="notes"` stores the view name. JavaScript reads it with `element.dataset.view`.

Database A program specifically designed to store, organize, and retrieve data reliably and efficiently. Study-Hub uses PostgreSQL.

Debounce A technique that delays executing a function until a certain time has passed since the last call. Used for auto-save and search — so a function doesn't fire on every keystroke but once the user stops typing.

Dependency A package or module that your code relies on. Listed in `package.json`.

Deployment The process of making an application available on a server for public access.

Destructuring JavaScript syntax for unpacking values from objects or arrays into named variables. `const { name, email } = user;` instead of `const name = user.name; const email = user.email;`.

DOM (Document Object Model) The browser's internal representation of the HTML page as a tree of objects. JavaScript uses the DOM API (`getElementById`, `querySelector`, etc.) to read and modify the page.

dotenv An npm package that reads a `.env` file and loads the key-value pairs into `process.env`.

Dynamic SQL Building SQL query strings at runtime based on conditions (e.g., adding `WHERE folder_id = $n` only if a folder filter is specified).

Element An HTML element: a single tag and its content. `<h1>Hello</h1>` is an `h1` element.

Environment variable A configuration value provided to a program by its environment (operating system or deployment platform), not hardcoded. Accessed via `process.env` in Node.js.

Event Something that happens in the browser or in code. `click`, `submit`, `keydown` are DOM events. Custom events can also be created.

Event listener A function registered to run when a specific event occurs. `element.addEventListener('click', fn)`.

Express A Node.js web framework that makes building HTTP servers easier. Provides routing, middleware, and response helpers.

`fetch()` The browser's built-in function for making HTTP requests from JavaScript.

Flexbox A CSS layout model for arranging elements in rows or columns with powerful alignment options.

Foreign key A column in one database table that references the primary key of another table. Enforces relationships between tables. `folder_id REFERENCES note_folders(id)`.

Framework Pre-built code that provides structure and tools for a specific type of application. Express is a framework for Node.js HTTP servers.

Full-text search (FTS) Database feature for searching text content efficiently. PostgreSQL's FTS uses language-aware indexing (stemming, stop words) rather than simple string matching.

Function A named, reusable block of code. It takes input (parameters), does something, and optionally returns output.

Git Software for tracking changes to files over time. The industry standard for code version control.

GitHub A website that hosts Git repositories. Used for storing and sharing code.

Hash / Hashing A one-way mathematical function that converts input into a fixed-length output (the hash). Used for passwords — you can verify a password matches a hash without storing the plain password.

Header (HTTP) Key-value pairs sent with HTTP requests and responses that provide metadata. `Content-Type: application/json`, `Authorization: Bearer eyJ....`

HTML (HyperText Markup Language) The language that describes the structure and content of web pages.

HTTP (HyperText Transfer Protocol) The rules for how browsers and servers communicate. Every request and response on the web uses HTTP.

HTTPS HTTP + TLS encryption. The `S` stands for Secure. All data between browser and server is encrypted.

ID (HTML) A unique identifier for one specific HTML element. `id="login-btn"`. In CSS, targeted with `#login-btn`. Only one element should have a given ID.

IIFE (Immediately Invoked Function Expression) A function that calls itself immediately. `(() => { ... })()`. Used in Study-Hub to create isolated modules.

Index (SQL) A database structure that speeds up queries on specific columns. Like a book's index — instead of scanning every page, jump directly to what you need.

Injection attack An attack where malicious input is interpreted as code. SQL injection inserts SQL into a query. XSS injects JavaScript into a page. Parameterized queries and `escHtml()` prevent these.

`innerHTML` A DOM property that reads or sets the HTML content inside an element. Setting it can be dangerous with user input (XSS risk) unless the content is sanitized.

`textContent` A DOM property that reads or sets the text content of an element, treating everything as plain text (safe — no HTML interpretation).

IP address A numerical identifier for a computer on a network. IPv4: `142.250.80.46`. IPv6: `2001:0db8:....`

JavaScript A programming language that runs in browsers (and on servers via Node.js). The only language browsers understand natively.

JSON (JavaScript Object Notation) A text format for representing structured data. Looks like a JavaScript object: `{ "name": "Alice", "age": 25 }`. Used for sending data between frontend and backend.

JWT (JSON Web Token) A compact, self-contained way to transmit information as a signed JSON object. Study-Hub uses JWTs to authenticate API requests after login.

Library Pre-written code that you can use in your project. npm packages are libraries.

`localhost` A domain name that refers to your own computer. `http://localhost:3000` accesses port 3000 on your machine.

Markdown A lightweight text format with simple syntax for formatting: `**bold**`, `*italic*`, `# Heading`. Study-Hub uses it for notes and renders it to HTML.

Middleware In Express, a function that runs between a request arriving and the route handler responding. Forms a pipeline. `authenticate`, `rateLimit`, `morgan` are middleware.

MIME type A standardized identifier for file types. `application/pdf`, `image/jpeg`, `application/json`. Used in HTTP headers and file filtering.

Module A self-contained unit of code with a defined interface. Study-Hub's JavaScript modules (`Storage`, `Helpers`, `API`, etc.) are IIFEs that expose a public API.

`npm` **(Node Package Manager)** The tool for installing and managing JavaScript packages. Included with Node.js.

Node.js A runtime that lets JavaScript run outside the browser (on a server). Built on Chrome's V8 engine.

`null` A JavaScript value representing intentional absence of a value. Different from `undefined` (uninitialized).

Object (JavaScript) A collection of named properties: `{ name: "Alice", age: 25 }`. The most common data structure in JavaScript.

Optional chaining (`?.`) JavaScript operator that safely accesses nested properties without crashing if an intermediate value is null/undefined. `user?.address?.city` returns `undefined` instead of throwing if `address` is null.

Parameterized query A SQL query where values are passed separately from the SQL text (`WHERE id = $1`, not `WHERE id = '${id}'`). Prevents SQL injection.

Package A bundle of JavaScript code you can install with npm. Also called a library or module.

`package.json` A file in every Node.js project describing the project (name, version) and listing its dependencies.

Password hash The result of running a password through a hash function (bcrypt). Stored instead of the plain password.

Payload (JWT) The second section of a JWT, containing the data. For Study-Hub: `{ userId: "...", iat: ..., exp: ... }`.

Pool (database) A set of pre-established database connections kept ready for use. More efficient than creating a new connection for each query.

Port A number (0-65535) identifying a specific process on a computer. Node.js server listens on port 3000 (dev) or 10000 (Render).

PostgreSQL A powerful open-source relational database. Study-Hub's data store.

Primary key A column (or set of columns) that uniquely identifies each row in a table. Study-Hub uses UUIDs as primary keys.

`process.env` A Node.js object containing all environment variables. `process.env.PORT`, `process.env.DATABASE_URL`, etc.

Promise A JavaScript object representing a future value. Can be pending, resolved (success), or rejected (failure). `async/await` is built on promises.

Prompt engineering The practice of carefully crafting instructions (system prompts) for AI models to get reliable, desired behavior.

Query (SQL) A statement that reads or modifies database data. `SELECT`, `INSERT`, `UPDATE`, `DELETE` are query types.

Query string Parameters in a URL after the `?`: `?search=study&folder=abc`. Accessed via `req.query` in Express.

Rainbow table A pre-computed lookup table mapping common passwords to their hashes. Bcrypt's salting makes rainbow tables useless.

Rate limiting Restricting how many requests a user/IP can make in a time window. Defends against brute force and DoS attacks.

`RETURNING *` **(SQL)** PostgreSQL extension to INSERT/UPDATE/DELETE that returns the affected rows. Gives you the complete row (including auto-generated fields) after writing.

Regex (Regular Expression) A pattern for matching text. `/^[^\s@]+@[^\s@]+$/` is a regex for email validation.

REST (Representational State Transfer) An architectural style for APIs. URLs identify resources, HTTP methods define actions (GET=read, POST=create, PUT=update, DELETE=remove).

Route A combination of HTTP method and URL path that Express maps to a handler function. `app.get('/api/notes', handler)`.

Salt (cryptography) A random value added to a password before hashing, making each hash unique even for identical passwords. bcrypt includes the salt in the hash.

Schema The structure of a database: which tables exist, which columns they have, what types and constraints those columns have.

Selector (CSS) A pattern that identifies which HTML elements a CSS rule applies to. `.btn`, `#login-form`, `[data-theme="dark"]`.

Server A program that listens for incoming network requests and sends back responses. Runs continuously.

Session A period of interaction between a user and an application. Study-Hub chat sessions are groups of related messages. (Not to be confused with server-side auth sessions — Study-Hub uses JWTs instead.)

Signature (JWT) The third section of a JWT. A cryptographic proof that the token was created by the server and hasn't been tampered with.

Single-Page Application (SPA) A web app where all navigation happens via JavaScript without reloading the page. Study-Hub loads `index.html` once and dynamically shows/hides sections.

SQL (Structured Query Language) The language used to communicate with relational databases. `SELECT`, `INSERT`, `UPDATE`, `DELETE`, `CREATE TABLE`.

SQL injection An attack where malicious SQL is inserted into a query through user input. Prevented by parameterized queries.

Static files Files served unchanged: HTML, CSS, JavaScript, images. `express.static()` serves Study-Hub's frontend files.

Status code (HTTP) A three-digit number in HTTP responses indicating the result. `200 OK`, `201 Created`, `400 Bad Request`, `401 Unauthorized`, `404 Not Found`, `500 Internal Server Error`.

Streaming Sending data in chunks rather than all at once. Study-Hub streams PDF files: each chunk goes to the response as it's read from disk, avoiding loading the whole file into memory.

Tag (HTML) The markup element: `<div>`, `<h1>`, `<input>`, etc. Tags come in pairs (open + close) or are self-closing.

Template literal JavaScript string using backticks `` ` `` that can embed expressions with `${variable}`. Much cleaner than string concatenation.

Token A piece of data that represents something (like authentication permission). JWTs are tokens.

Trigger (SQL) A database rule that automatically runs a function when certain events occur (INSERT, UPDATE, DELETE). Study-Hub uses triggers to auto-update `updated_at` columns.

`try/catch` JavaScript error handling. Code in `try` is attempted. If it throws an error, execution jumps to `catch` where you can handle the error gracefully.

Type (SQL) The kind of data a column stores. `UUID`, `VARCHAR(255)`, `TEXT`, `INTEGER`, `BIGINT`, `BOOLEAN`, `TIMESTAMP WITH TIME ZONE`, `TEXT[]`.

TypeScript A superset of JavaScript that adds static type checking. Study-Hub uses plain JavaScript.

`undefined` The default value of a JavaScript variable that has been declared but not assigned a value. Also returned when accessing a non-existent object property.

URL (Uniform Resource Locator) A web address. `https://study-hub.onrender.com/api/notes?search=biolog` has: protocol (`https`), host, path (`/api/notes`), query string (`?search=biology`).

UUID (Universally Unique Identifier) A 128-bit random identifier. Looks like `ac2c0621-89c1-4925-9c4c-40d5a` Practically impossible to guess or collide with another UUID.

Variable A named container for a value. `const name = "Alice"`. Variables can hold any type of data.

Viewport The visible area of a web page. The viewport meta tag configures how mobile browsers render the page.

XHR (XMLHttpRequest) The older browser API for making HTTP requests. Still used when `fetch()` is insufficient — for example, when upload progress tracking is needed.

XSS (Cross-Site Scripting) An attack where malicious JavaScript is injected into a page via user input. Prevented by escaping HTML (`escHtml()`) and Content Security Policy headers.

`z-index` A CSS property controlling stacking order. Higher values appear in front. The toast container has a high z-index (`1000`) so toasts appear above everything.

That's the complete glossary. If you encounter a term not listed here, search it on MDN (developer.mozilla.org) for HTML/CSS/JavaScript terms, or the PostgreSQL documentation (postgresql.org/docs) for database terms.